

第21章: テスト

この章の目標: テストの基礎概念から実践的なテストコードの書き方まで、初心者でもゼロから理解できるように丁寧に解説します。この章を読み終えれば、BON-LOGアプリケーションに対してユニットテスト・インテグレーションテスト・E2Eテストを自信を持って書けるようになります。

21.0 実習手順の進め方と手順マップ

手順に沿って進めると、**どのファイルに何を入力し、何を確認すればよいか** が分かります。形式の説明は [チュートリアルの進め方](#) を参照してください。

手順	主な対象ファイル（例）	完了時に確認すること
テストの考え方	—	テストピラミッド・種類が説明できる
Vitest セットアップ	<code>vitest.config.ts</code> , <code>__tests__/*</code>	<code>npm test</code> が実行できる
ユニット・コンポーネントテスト	<code>__tests__/**/*.test.ts(x)</code>	単体・コンポーネントが通る
E2E (Playwright)	<code>e2e/*.spec.ts</code>	<code>npm run test:e2e</code> が通る

各セクションで **対象ファイル・入力するコード（サンプルコード）・実行方法・実行するようになる・このあとと変わること・確認方法** を確認しながら進めてください。

21.1 テストとは何か？なぜ必要なのか？

▶ このセクションで学ぶこと（クリックで展開）

なぜテストを書くのか？ 「動いているコードにわざわざテストを書くのは無駄」 と思うかもしれませんが、しかし：

1. **変更の安全性:** 投稿機能を修正した時、「いいね」が壊れていないか確認できる
2. **ドキュメント代わり:** テストを読めば「この関数は何をすべきか」がわかる
3. **開発速度の向上:** 手動で「ログイン→投稿→確認」を毎回繰り返す必要がない
4. **チーム開発:** 他の人の変更で自分のコードが壊れたことに即座に気づける

BON-LOGでは8,800以上のテスト（455テストスイート）が3～5分で実行でき、コード変更の安全性を自動的に検証しています。E2Eテストは38ファイルでPlaywright 1.57.0を使用しています。

21.1.1 テストの基本概念

テストとは、「プログラムが期待通りに動作するかを自動的に確認する仕組み」です。

身近な例で考えてみましょう。あなたが盆栽の鉢を作る職人だとします。

【盆栽の鉢を作る工程のたとえ】

手作業での確認（手動テスト）：

毎回、目で見て形を確認 → 水を入れて漏れないか確認 → 強度を叩いて確認

自動テスト：

形状チェック機 → 水漏れ検査機 → 強度テスト機
を通すだけで、品質が自動で保証される！

プログラムも同じです。手動で「ボタンを押して動くか確認する」のではなく、**テストコード**を書いておけば、コマンド一つで全ての機能が正しく動くか確認できます。

21.1.2 テストを書くメリット

メリット	説明
バグの早期発見	コードを変更した直後に問題を検出できる
リファクタリングの安全性	テストがあれば安心してコードを改善できる
ドキュメント代わり	テストコードを読めば「この関数は何をするか」がわかる
開発速度の向上	長期的には手動テストより圧倒的に速い
チーム開発の品質維持	他の人が書いたコードも自動で品質チェックできる

21.1.3 テストの種類

テストには大きく3つの種類があります。

テストの種類	ユニットテスト (単体テスト)	インテグレーションテスト (結合テスト)	E2Eテスト (エンドツーエンド)
対象	関数1つを個別にテスト	複数の部品を組み合わせてテスト	ユーザーの操作をブラウザで丸ごとテスト
例	formatDate() が正しい文字列を返すか	フォーム送信 → バリデーション → DB保存	ログイン → 投稿 → いいね → ログアウトの一連の流れ
速度	速い(ms単位)	中程度(秒単位)	遅い(分単位)
量	大量に書く	適度に書く	少数だけ書く

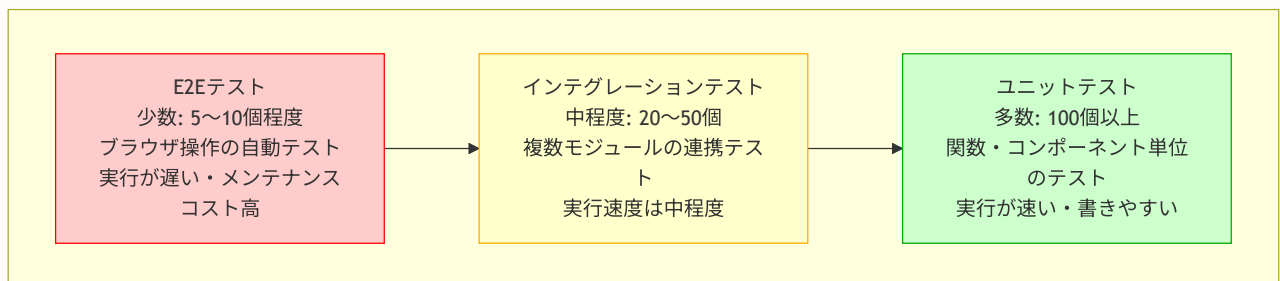
テストの使い分け

テスト種別	何をテストするか	いつ使うか
ユニットテスト	1つの関数・コンポーネント	ロジックが複雑な関数（バリデーション、計算等）
コンポーネントテスト	UIの表示と操作	ボタンクリック、フォーム入力、条件分岐表示
E2Eテスト	ユーザーの操作フロー全体	ログイン→投稿→確認のような重要なフロー

原則: ユニットテストを多く、E2Eテストを少なく（テストピラミッド）。E2Eは遅くて壊れやすいため、重要なフローに限定します。

21.1.4 テストピラミッド

テストピラミッドは、各種テストの**推奨バランス**を示す有名な考え方です。下に行くほど数が多く、上に行くほど数が少なくなります。



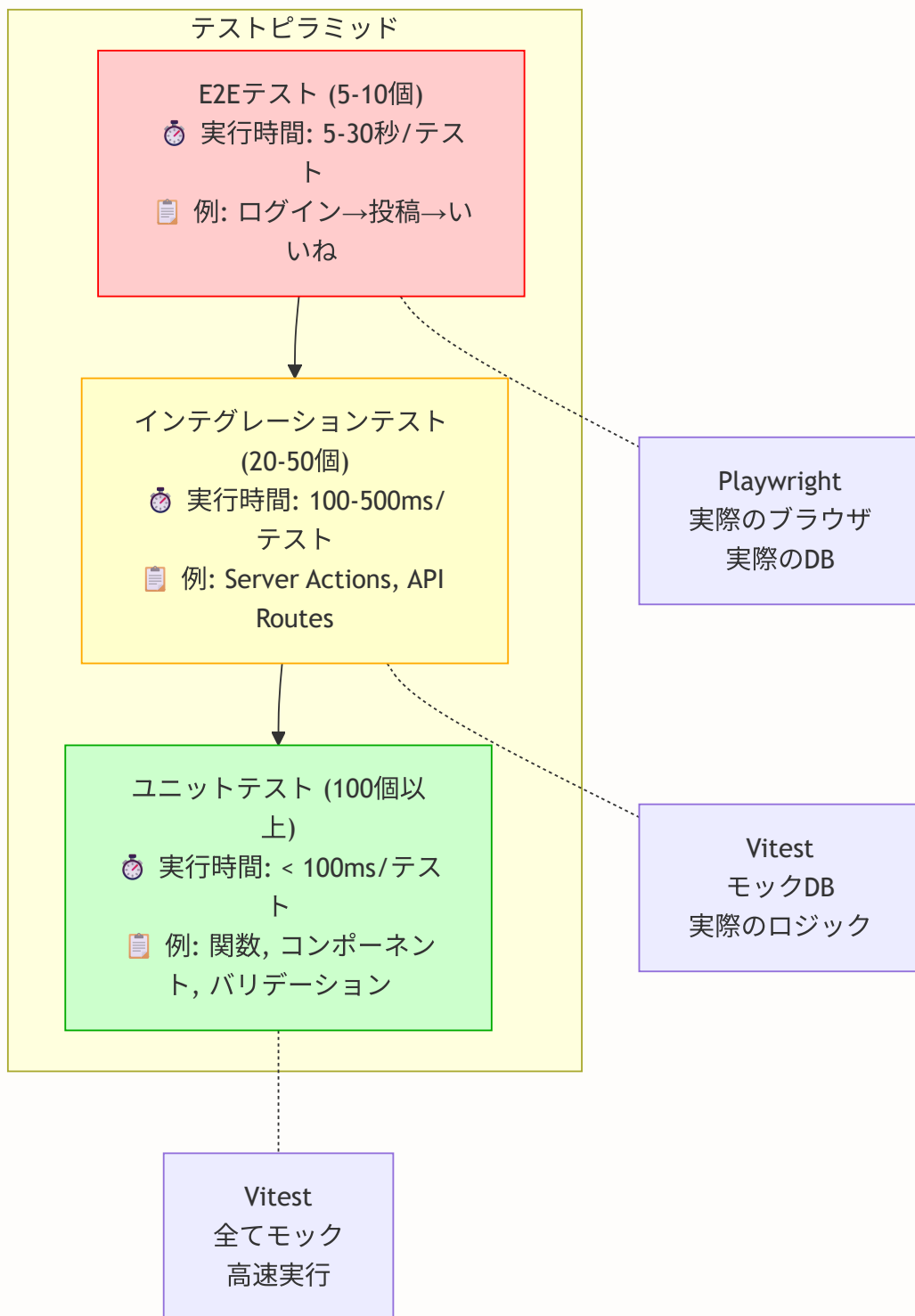
なぜピラミッド型がよいのか？

- **ユニットテスト**は実行が速く、原因の特定も簡単なので、たくさん書く
- **E2Eテスト**は実行が遅くメンテナンスが大変なので、最重要フローだけに絞る
- **インテグレーションテスト**はその中間

▶ 理解度チェック（クリックで回答を確認）

21.1.5 テストピラミッドの詳細構成

BON-LOGプロジェクトにおける具体的なテストの構成を見てみましょう。



テスト数の目安

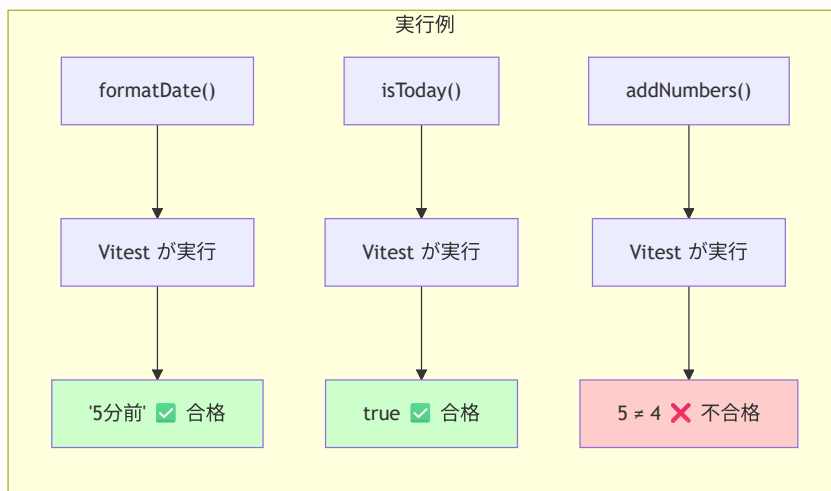
- ユニットテスト: 100個以上（全コードの80%以上をカバー）
- インテグレーションテスト: 20-50個（重要なフローを網羅）
- E2Eテスト: 5-10個（クリティカルなユーザーフローのみ）

21.2 Vitest のセットアップ

▶ このセクションで学ぶこと（クリックで展開）

21.2.1 Vitest とは？

Vitest（ヴィーテスト）は、Vite ベースの高速な JavaScript/TypeScript **テストフレームワーク**です。Jest 互換の API を持ちながら、Vite のビルドパイプラインを活用することで圧倒的に速いテスト実行を実現しています。



21.2.2 インストール

以下のコマンドで、テストに必要なパッケージをまとめてインストールします。

```
# テストフレームワーク本体と React テスト用ライブラリをインストール
npm install -D vitest @vitejs/plugin-react vite-tsconfig-paths @vitest/coverage-v8
npm install -D @testing-library/react @testing-library/jest-dom @testing-library/user-event
npm install -D jsdom
```

各パッケージの役割を確認しましょう。

パッケージ名	役割
<code>vitest</code>	テストフレームワーク本体（Viteベース）。テストの実行・結果表示を行う。型定義も内蔵
<code>@vitejs/plugin-react</code>	React の JSX 変換をViteで行うためのプラグイン
<code>vite-tsconfig-paths</code>	<code>tsconfig.json</code> のパスエイリアス（ <code>@/</code> ）を Vite/Vitest で自動解決
<code>@vitest/coverage-v8</code>	V8 エンジンを使った高速なカバレッジ計測
<code>@testing-library/react</code>	React コンポーネントをテスト環境で描画（レンダリング）するためのツール
<code>@testing-library/jest-dom</code>	DOM要素の状態を確認するための便利なマッチャー（ <code>toBeInTheDocument()</code> など）
<code>@testing-library/user-event</code>	ユーザーの操作（クリック、入力など）をシミュレートするツール
<code>jsdom</code>	ブラウザの DOM 環境を Node.js 上で再現するための環境

21.2.3 vitest.config.ts

Vitest の設定ファイルです。`vite-tsconfig-paths` プラグインを使うことで、`tsconfig.json` のパスエイリアス（`@/`）を自動的に解決できます。

BON-LOGでの使用箇所: プロジェクトルートの `vitest.config.ts` がこの設定です。`npm test` および `npm run test:coverage` コマンドで参照されます。

実装しない場合の影響: `vitest.config.ts` がないとVitestがデフォルト設定で動作し、パスエイリアス（`@/`）の解決に失敗してほぼすべてのテストがエラーになります。カバレッジ閾値が設定されていないと、テストカバレッジが下がってもCIが通ってしまいます。

```
// vitest.config.ts (実際のファイル)
import { defineConfig } from 'vitest/config' // Vitest の設定関数
import react from '@vitejs/plugin-react' // React JSX サポート
import tsconfigPaths from 'vite-tsconfig-paths' // tsconfig のパスエイリアス解決

export default defineConfig({
  // Vite プラグイン
  plugins: [
    react(), // React コンポーネントの JSX 変換
    tsconfigPaths(), // @/ → プロジェクトルートの自動解決
  ],

  // テスト固有の設定
  test: {
    // グローバルAPI: describe, it, expectなどをimportなしで使用可能
    globals: true,

    // テスト環境: ブラウザのDOMを再現するjsdomを使用
    // (Node.jsにはブラウザのdocumentやwindowがないため)
    environment: 'jsdom',

    // テスト実行前に読み込むセットアップファイル
    setupFiles: ['./vitest.setup.tsx'],

    // テストファイルのパターン: どのファイルをテストとして認識するか
    include: [
      '**/__tests__/**/*.test.ts', // __tests__ フォルダ内
      '**/*.test.ts', // *.test.ts や *.spec.ts
    ],

    // テスト対象から除外するパス
    exclude: [
      '**/node_modules/**', // 依存パッケージ
      '**/.next/**', // Next.js ビルド出力
      '**/e2e/**', // E2E テストはPlaywrightで実行するため除外
    ],

    // テストのタイムアウト (10秒)
    testTimeout: 10000,

    // テアダウンのタイムアウト (5秒)
    // テスト後のクリーンアップ処理 (モックのリセットなど) が5秒以内に完了しなければ強制終了
    teardownTimeout: 5000,

    // テスト実行モード: forks で各テストファイルを独立プロセスで実行
    pool: 'forks',

    // カバレッジ (テストでどれだけのコードが実行されたか) の設定
    coverage: {
      provider: 'v8', // V8 エンジンのネイティブカバレッジを使用 (高速)
      include: [
```

```

    'lib/**/*.{ts,tsx}',          // lib ディレクトリ内のファイル
    'components/**/*.{ts,tsx}',   // components ディレクトリ内のファイル
    'app/**/*.{ts,tsx}',          // app ディレクトリ内のファイル
  ],
  exclude: [
    '**/*.d.ts',                  // 型定義ファイルは除外
    '**/node_modules/**',         // node_modules は除外
    '**/__tests__/**',            // テストファイル自体は除外
  ],
  // カバレッジの最低ライン（これを下回るとテストが失敗する）
  thresholds: {
    branches: 30,                 // 条件分岐（if/else）の30%以上をカバー
    functions: 30,                // 関数の30%以上をカバー
    lines: 35,                    // コード行数の35%以上をカバー
    statements: 35,               // 文の35%以上をカバー
  },
},

// ESM パッケージの最適化設定
// CJS/ESM の互換性問題があるパッケージを明示的に列挙
deps: {
  optimizer: {
    web: {
      include: [
        'isomorphic-dompurify',   // HTML サニタイゼーション
        'dompurify',
        '@panva',                  // JWTライブラリ（NextAuth依存）
        'jose',                    // JWTライブラリ
        'nanoid',                  // ユニークID生成
        'uuid',
        'next-auth',               // 認証
        '@auth',
        'otplib',                  // 2FA（TOTP）ライブラリ
        '@otplib',
        '@scure',                  // 暗号化ライブラリ
      ],
    },
  },
},
},
},
},
})

```

21.2.4 vitest.setup.tsx

テスト実行前に毎回読み込まれるセットアップファイルです。全テスト共通の設定やモック（後述）をここに書きます。

BON-LOGでの使用箇所: プロジェクトルートの `vitest.setup.tsx` がこのファイルです。
`vitest.config.ts` の `setupFiles` から参照されており、全テスト実行前に一度だけ読み込まれます。
Prisma、NextAuth、next/navigation、Redisなどのモックがここで一括設定されます。

実装しない場合の影響: `vitest.setup.tsx` がないと、テストファイルごとに個別にPrismaや認証のモックを設定する必要が生じ、冗長なコードが増えます。グローバルモックがないと、実際のDBへの接続を試みてテストが失敗します。TextEncoder/TextDecoderのポリフィルがないと、PrismaクライアントがNode.jsのテスト環境で正しく動作しません。

```
// vitest.setup.tsx

// TextEncoder/TextDecoder ポリフィル (Prisma が内部で使用)
import { TextEncoder, TextDecoder } from 'util'
global.TextEncoder = TextEncoder
global.TextDecoder = TextDecoder as typeof global.TextDecoder

// Testing Library のマッチャーを追加
// これにより toBeInTheDocument() などの DOM チェック関数を使えるようになる
import '@testing-library/jest-dom'

// Prisma クライアントをモック化 (全テスト共通)
vi.mock('@/lib/db', () => ({
  prisma: {
    user: { findUnique: vi.fn(), findMany: vi.fn(), create: vi.fn(), update: vi.fn(), delete: vi.fn() },
    post: { findUnique: vi.fn(), findMany: vi.fn(), create: vi.fn(), update: vi.fn(), delete: vi.fn() },
    follow: { findUnique: vi.fn(), findMany: vi.fn(), create: vi.fn(), delete: vi.fn() },
    $transaction: vi.fn(),
  },
}))

// 認証モック (NextAuth.js)
vi.mock('@/lib/auth', () => ({
  auth: vi.fn().mockResolvedValue({
    user: { id: 'test-user-id', name: 'Test User', email: 'test@example.com' },
  }),
  signIn: vi.fn(),
  signOut: vi.fn(),
  handlers: { GET: vi.fn(), POST: vi.fn() },
}))

// Next.js の navigation 関連フックをモック化
// テスト環境では Next.js のルーターが存在しないため、
// 偽のルーターを用意してエラーを防ぐ
vi.mock('next/navigation', () => ({
  useRouter: () => ({
    push: vi.fn(), replace: vi.fn(), back: vi.fn(),
    forward: vi.fn(), refresh: vi.fn(), prefetch: vi.fn(),
  }),
  useSearchParams: () => new URLSearchParams(),
  usePathname: () => '/',
  redirect: vi.fn(),
  notFound: vi.fn(),
}))
```

21.2.5 package.json のスクリプト設定

テスト実行用のコマンドを `package.json` に追加します。

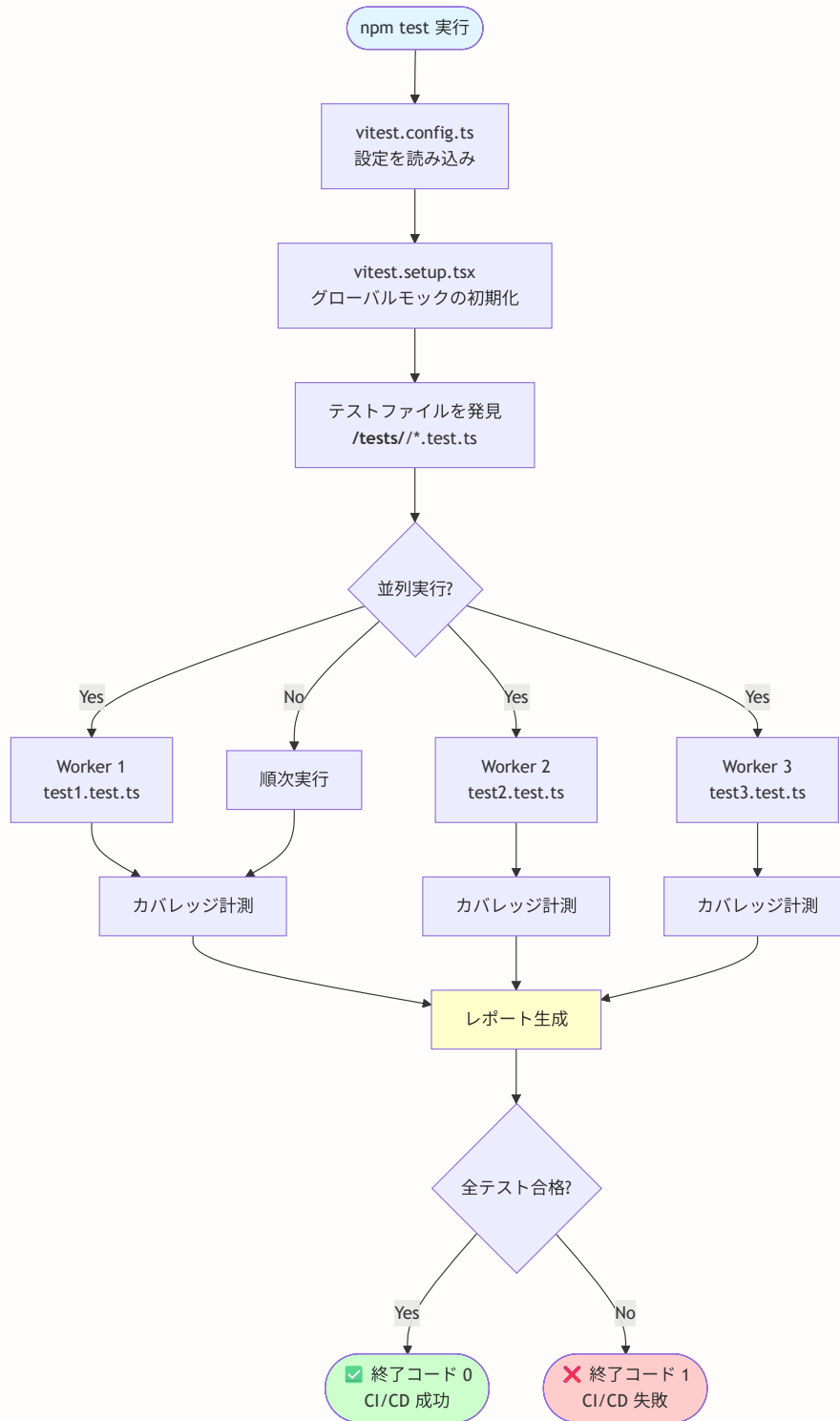
```
{
  "scripts": {
    "test": "vitest run", // 基本のテスト実行
    "test:watch": "vitest", // ファイル変更を監視して自動再実行
    "test:coverage": "vitest run --coverage", // カバレッジレポート付きで実行
    "test:e2e": "playwright test", // E2Eテスト実行（後述）
    "test:all": "npm run test && npm run test:e2e" // 全テスト一括実行
  }
}
```

各コマンドの使い分けを説明します。

コマンド	いつ使うか	説明
<code>npm test</code>	日常の開発中	全ユニットテストを1回実行
<code>npm run test:watch</code>	コーディング中	ファイル保存のたびに関連テストだけ自動再実行（最も便利）
<code>npm run test:coverage</code>	PR作成前	テストがどれだけのコードをカバーしているか確認
<code>npm run test:e2e</code>	リリース前	ブラウザでの実際の操作フローを自動テスト
<code>npm run test:all</code>	CI/CD	ユニットテスト + E2E テストを一括実行

21.2.6 テスト実行パイプライン

Vitestがテストを実行する流れを理解しましょう。



実行の最適化

- テストは並列実行される（複数のワーカーで分散処理）
- 各ワーカーは独立したNode.jsプロセス
- カバレッジは各ワーカーの結果をマージ

よくあるトラブルと解決法（セットアップ編）

トラブル	原因	解決法
<code>Cannot find module '@/lib/ ... '</code>	パスエイリアスが解決できていない	<code>vitest.config.ts</code> で <code>vite-tsconfig-paths</code> プラグインが設定されているか確認
<code>SyntaxError: Cannot use import statement</code>	ESM/CJS の互換性問題	<code>vitest.config.ts</code> の <code>deps.optimizer.web.include</code> に該当パッケージを追加
<code>ReferenceError: document is not defined</code>	テスト環境が Node.js のまま	<code>vitest.config.ts</code> で <code>environment: 'jsdom'</code> を設定
<code>TypeError: fetch is not a function</code>	fetch がモックされていない	<code>vitest.setup.tsx</code> に <code>global.fetch = vi.fn()</code> を追加

▶ 理解度チェック（クリックで回答を確認）

21.3 ユニットテスト

▶ このセクションで学ぶこと（クリックで展開）

21.3.1 ユニットテストの基本構造

ユニットテストは、**1つの関数**や**1つの小さな単位**を対象にテストします。料理のたとえでいえば、「料理全体の味見」ではなく「塩加減だけ確認」「火の通り具合だけ確認」のように、1つずつ確認する方法です。

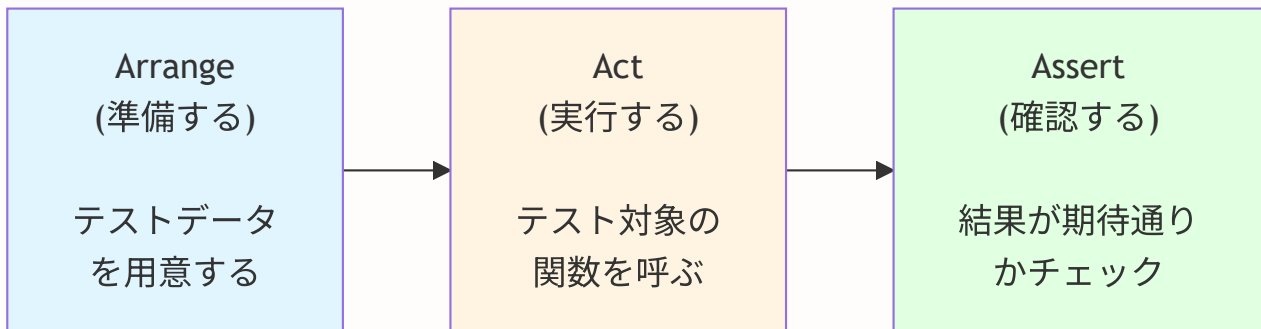
Vitest でのテストの基本構文を見てみましょう。

```
// describe: テストのグループ（まとまり）を定義する
// 「〇〇について」というくくりを作る
describe('テスト対象の名前', () => {

  // it（または test）: 1つのテストケースを定義する
  // 「～すべき」という1つの確認事項を書く
  it('期待する動作の説明', () => {

    // expect: 実際の結果が期待値と一致するか確認する
    // 「この値は〇〇であるべき」というチェック
    expect(実際の値).toBe(期待する値)
  })
})
```

テストの流れを図で示すと以下のようになります。



例: `formatRelativeTime` のテスト



この **Arrange（準備）** → **Act（実行）** → **Assert（検証）** のパターンは「AAAパターン」と呼ばれ、テストを書くときの基本です。

21.3.2 主要なマッチャー（検証関数）一覧

`expect(値)` の後に続く `.toBe()` などを **マッチャー** と呼びます。よく使うものを紹介します。

マッチャー	意味	使用例
<code>.toBe(値)</code>	厳密に等しい (<code>===</code>)	<code>expect(1 + 1).toBe(2)</code>
<code>.toEqual(値)</code>	オブジェクトの中身が等しい	<code>expect({a: 1}).toEqual({a: 1})</code>
<code>.toBeTruthy()</code>	真 (truthy) である	<code>expect('hello').toBeTruthy()</code>
<code>.toBeFalsy()</code>	偽 (falsy) である	<code>expect('').toBeFalsy()</code>
<code>.toBeNull()</code>	null である	<code>expect(null).toBeNull()</code>
<code>.toBeDefined()</code>	undefined でない	<code>expect(value).toBeDefined()</code>
<code>.toContain(値)</code>	配列・文字列に含まれる	<code>expect([1, 2, 3]).toContain(2)</code>
<code>.toThrow()</code>	エラーを投げる	<code>expect(() => fn()).toThrow()</code>
<code>.toHaveLength(数)</code>	配列や文字列の長さ	<code>expect([1, 2]).toHaveLength(2)</code>
<code>.toBeGreaterThan(数)</code>	より大きい	<code>expect(5).toBeGreaterThan(3)</code>
<code>.toBeInTheDocument()</code>	DOM に存在する (Testing Library)	<code>expect(element).toBeInTheDocument()</code>

21.3.3 ユーティリティ関数のテスト

実際にBON-LOGの日付ユーティリティ関数をテストしてみましょう。

まず、テスト対象の関数（実装）を確認します。

lib/utils/date.ts (実装)

```
// 相対時間をフォーマットする関数
// 例: 30秒前 → "たった今"、5分前 → "5分前"、3時間前 → "3時間前"
export function formatRelativeTime(date: Date): string {
  const now = new Date() // 現在時刻を取得
  const diff = now.getTime() - date.getTime() // 現在との差分（ミリ秒）

  const seconds = Math.floor(diff / 1000) // 秒に変換
  if (seconds < 60) return 'たった今' // 60秒未満なら「たった今」

  const minutes = Math.floor(seconds / 60) // 分に変換
  if (minutes < 60) return `${minutes}分前` // 60分未満なら「x分前」

  const hours = Math.floor(minutes / 60) // 時間に変換
  if (hours < 24) return `${hours}時間前` // 24時間未満なら「x時間前」

  // 24時間以上前なら日付を表示（例: "2024/01/15"）
  return date.toLocaleDateString('ja-JP', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit',
  }).replace(/\//g, '/')
}

// 指定された日付が今日かどうかを判定する関数
export function isToday(date: Date): boolean {
  const now = new Date() // 現在時刻を取得
  return (
    date.getDate() === now.getDate() && // 日が同じ
    date.getMonth() === now.getMonth() && // 月が同じ
    date.getFullYear() === now.getFullYear() // 年が同じ
  )
}

// 指定された日付から何日経過したかを計算する関数
export function getDaysSince(date: Date): number {
  const now = new Date() // 現在時刻を取得
  const diffTime = now.getTime() - date.getTime() // 差分（ミリ秒）
  return Math.floor(diffTime / (1000 * 60 * 60 * 24)) // ミリ秒 → 日に変換
}
```

次に、この関数に対するテストコードです。

lib/utils/date.test.ts

```
// テスト対象の関数をインポート
import { formatRelativeTime, isToday, getDaysSince } from './date'

// --- formatRelativeTime のテストグループ ---
describe('date utils', () => {
  describe('formatRelativeTime', () => {

    // テストケース1: 60秒未満の場合「たった今」と表示されるか
    it('should format time within a minute', () => {
      // Arrange (準備): 30秒前の日時を作成
      const now = new Date()
      const date = new Date(now.getTime() - 30000) // 30秒 = 30,000ミリ秒

      // Act + Assert (実行 + 検証): 関数を呼び出して結果を確認
      expect(formatRelativeTime(date)).toBe('たった今')
    })

    // テストケース2: 分単位の表示
    it('should format time in minutes', () => {
      const now = new Date()
      const date = new Date(now.getTime() - 5 * 60000) // 5分 = 5 * 60,000ミリ秒

      expect(formatRelativeTime(date)).toBe('5分前')
    })

    // テストケース3: 時間単位の表示
    it('should format time in hours', () => {
      const now = new Date()
      const date = new Date(now.getTime() - 3 * 3600000) // 3時間 = 3 * 3,600,000ミリ秒

      expect(formatRelativeTime(date)).toBe('3時間前')
    })

    // テストケース4: 24時間以上前は日付表示
    it('should format date for older dates', () => {
      const date = new Date('2024-01-15') // 過去の日付

      expect(formatRelativeTime(date)).toBe('2024/01/15')
    })
  })
})

// --- isToday のテストグループ ---
describe('isToday', () => {

  // テストケース1: 今日の日付なら true を返す
  it('should return true for current date', () => {
    const now = new Date()
    expect(isToday(now)).toBe(true) // 今日 → true
  })
})
```

```
// テストケース2: 昨日の日付なら false を返す
it('should return false for yesterday', () => {
  const yesterday = new Date()
  yesterday.setDate(yesterday.getDate() - 1) // 1日前に設定

  expect(isToday(yesterday)).toBe(false) // 昨日 → false
})

// --- getDaysSince のテストグループ ---
describe('getDaysSince', () => {

  // テストケース1: 7日前の日付なら 7 を返す
  it('should calculate days between dates', () => {
    const date = new Date()
    date.setDate(date.getDate() - 7) // 7日前に設定

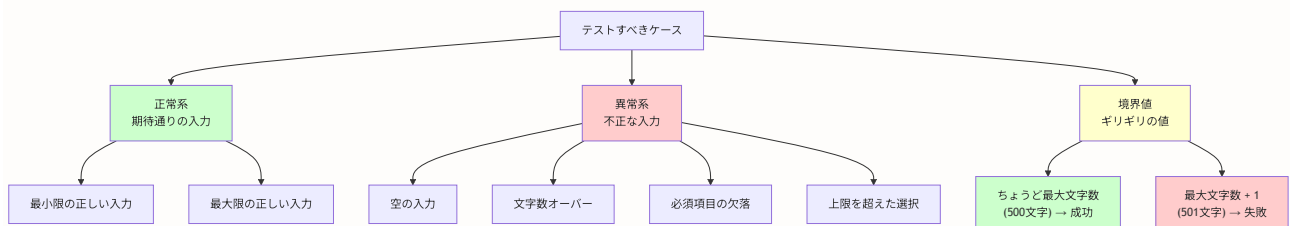
    expect(getDaysSince(date)).toBe(7)
  })

  // テストケース2: 今日の日付なら 0 を返す
  it('should return 0 for same day', () => {
    const now = new Date()
    expect(getDaysSince(now)).toBe(0)
  })
})
})
```

ポイント: テストケースの `it()` の説明文は英語で書くのが慣例ですが、日本語で書いても問題ありません。チームで統一しましょう。

21.3.4 バリデーション関数のテスト

バリデーション（入力値の検証）は、テストが特に重要な領域です。不正な入力データベースに保存されるとセキュリティ問題につながるため、**正常系**と**異常系**の両方をテストします。



lib/validations/post.test.ts

```
// テスト対象のスキーマ（zod で定義されたバリデーションルール）をインポート
import { createPostSchema } from './post'

describe('createPostSchema', () => {

  // --- 正常系テスト ---

  // テストケース1: 正しいデータならバリデーション成功
  it('should validate valid post data', () => {
    // Arrange: 正しい投稿データを用意
    const data = {
      content: 'テスト投稿です',      // 内容あり（1～500文字以内）
      genreIds: ['genre1'],           // ジャンル1つ選択（1～3個以内）
    }

    // Act: safeParse でバリデーション実行（エラーを投げずに結果を返す）
    const result = createPostSchema.safeParse(data)

    // Assert: 成功するはず
    expect(result.success).toBe(true)
  })

  // --- 異常系テスト ---

  // テストケース2: 空のコンテンツは拒否される
  it('should reject empty content', () => {
    const data = {
      content: '',                    // 空文字（不正）
      genreIds: ['genre1'],
    }

    const result = createPostSchema.safeParse(data)

    // バリデーション失敗を確認
    expect(result.success).toBe(false)
    // エラーメッセージの内容も確認
    expect(result.error?.errors[0].message).toBe('内容を入力してください')
  })

  // テストケース3: 500文字を超えるコンテンツは拒否される
  it('should reject content over 500 characters', () => {
    const data = {
      content: 'a'.repeat(501),      // 501文字（上限オーバー）
      genreIds: ['genre1'],
    }

    const result = createPostSchema.safeParse(data)
    expect(result.success).toBe(false)
  })
})
```



```
// テストケース4: ジャンル未選択は拒否される
it('should reject without genre', () => {
  const data = {
    content: 'テスト投稿',
    genreIds: [], // ジャンルなし（最低1つ必要）
  }

  const result = createPostSchema.safeParse(data)
  expect(result.success).toBe(false)
})

// テストケース5: ジャンル4つ以上は拒否される
it('should reject more than 3 genres', () => {
  const data = {
    content: 'テスト投稿',
    genreIds: ['g1', 'g2', 'g3', 'g4'], // 4つ（上限は3つ）
  }

  const result = createPostSchema.safeParse(data)
  expect(result.success).toBe(false)
})
})
```

▶ 理解度チェック（クリックで回答を確認）

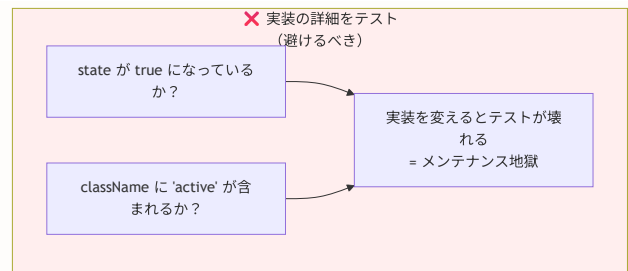
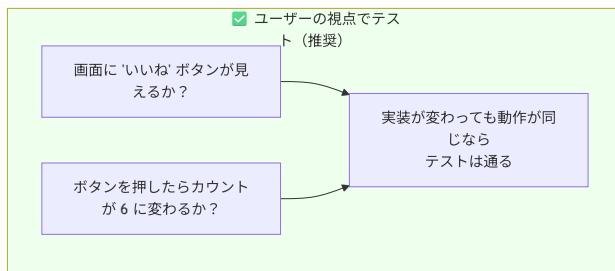
21.4 React コンポーネントのテスト

▶ このセクションで学ぶこと（クリックで展開）

21.4.1 React Testing Library とは？

React Testing Libraryは、React コンポーネントを「**ユーザーの視点**」でテストするためのライブラリです。

「ユーザーの視点」とは何でしょうか？ 以下の比較を見てください。



21.4.2 基本的なAPI

API	役割	使用例
<code>render(<Component />)</code>	コンポーネントを仮想DOMに描画する	<code>render(<Button>OK</Button>)</code>
<code>screen.getByText('文字')</code>	画面上の文字列で要素を取得	<code>screen.getByText('投稿する')</code>
<code>screen.getByRole('button')</code>	役割 (role) で要素を取得	<code>screen.getByRole('button')</code>
<code>screen.getByTestId('id')</code>	data-testid 属性で要素を取得	<code>screen.getByTestId('like-btn')</code>
<code>screen.queryByText('文字')</code>	要素がなくてもエラーにならない	「存在しないこと」の確認に使用
<code>fireEvent.click(要素)</code>	クリックイベントを発火	<code>fireEvent.click(button)</code>
<code>waitFor(() => { ... })</code>	非同期の結果を待つ	API 呼び出し後の表示確認

21.4.3 Button コンポーネントのテスト

シンプルな Button コンポーネントから始めましょう。

components/ui/Button.test.tsx

```
// テストに必要なユーティリティをインポート
import { render, screen, fireEvent } from '@testing-library/react'
// テスト対象のコンポーネントをインポート
import { Button } from './Button'

describe('Button', () => {

  // テスト1: ボタンにテキストが表示されるか
  it('should render button with text', () => {
    // Arrange + Act: ボタンを描画
    render(<Button>クリック</Button>)

    // Assert: 「クリック」というテキストが画面に存在するか確認
    // getByText は該当要素がなければテスト失敗になる
    expect(screen.getByText('クリック')).toBeInTheDocument()
  })

  // テスト2: クリック時にイベントハンドラが呼ばれるか
  it('should call onClick when clicked', () => {
    // Arrange: vi.fn() でモック関数（呼び出しを記録する偽の関数）を作成
    const handleClick = vi.fn()
    render(<Button onClick={handleClick}>クリック</Button>)

    // Act: ボタンをクリック
    fireEvent.click(screen.getByText('クリック'))

    // Assert: handleClick が1回呼ばれたことを確認
    expect(handleClick).toHaveBeenCalledTimes(1)
  })

  // テスト3: disabled 時にボタンが無効化されるか
  it('should be disabled when disabled prop is true', () => {
    render(<Button disabled>クリック</Button>)

    // getByText で取得した要素が disabled 状態か確認
    const button = screen.getByText('クリック')
    expect(button).toBeDisabled()
  })

  // テスト4: variant に応じたスタイルが適用されるか
  it('should apply variant styles', () => {
    // container は描画されたDOMのルート要素
    const { container } = render(<Button variant="destructive">削除</Button>)
    // querySelector で button 要素を直接取得
    const button = container.querySelector('button')

    // destructive クラスが付与されているか確認
    expect(button).toHaveClass('destructive')
  })
})
```

```
// テスト5: disabled 時はクリックしても onClick が呼ばれないか
it('should not call onClick when disabled', () => {
  const handleClick = vi.fn()
  render(<Button onClick={handleClick} disabled>クリック</Button>)

  // disabled なボタンをクリック
  fireEvent.click(screen.getByText('クリック'))

  // onClick は呼ばれないはず
  expect(handleClick).not.toHaveBeenCalled()
})
})
```

21.4.4 PostCard コンポーネントのテスト

より実践的な例として、投稿カードのテストを見てみましょう。

components/post/PostCard.test.tsx

```
import { render, screen } from '@testing-library/react'
import { PostCard } from '../PostCard'

// テスト用のモックデータ（偽のデータ）を用意
// 実際のAPIから取得するデータと同じ構造にする
const mockPost = {
  id: 'post1', // 投稿ID
  content: 'テスト投稿です', // 投稿内容
  createdAt: new Date('2024-01-15'), // 作成日時
  user: { // 投稿者情報
    id: 'user1',
    nickname: 'テストユーザー',
    avatarUrl: '/avatar.jpg',
  },
  _count: { // いいね・コメント数
    likes: 5,
    comments: 3,
  },
}

describe('PostCard', () => {

  // テスト1: 投稿の本文が表示されるか
  it('should render post content', () => {
    render(<PostCard post={mockPost} />)
    // 投稿内容のテキストが画面に存在するか確認
    expect(screen.getByText('テスト投稿です')).toBeInTheDocument()
  })

  // テスト2: ユーザー名が表示されるか
  it('should render user information', () => {
    render(<PostCard post={mockPost} />)
    expect(screen.getByText('テストユーザー')).toBeInTheDocument()
  })

  // テスト3: いいね数が表示されるか
  it('should render like count', () => {
    render(<PostCard post={mockPost} />)
    expect(screen.getByText('5')).toBeInTheDocument()
  })

  // テスト4: コメント数が表示されるか
  it('should render comment count', () => {
    render(<PostCard post={mockPost} />)
    expect(screen.getByText('3')).toBeInTheDocument()
  })

  // テスト5: ユーザーアバター画像が表示されるか
  it('should render user avatar', () => {
```

```

render(<PostCard post={mockPost} />)
// getByRole('img') で img 要素を取得 (name はalt属性の値)
const avatar = screen.getByRole('img', { name: /avatar/i })
// src 属性が正しいか確認
expect(avatar).toHaveAttribute('src', '/avatar.jpg')
})
})

```

よくあるトラブルと解決法（コンポーネントテスト編）

トラブル	原因	解決法
Unable to find an element with the text	テキストが見つからない	<code>screen.debug()</code> を呼んで実際の DOM を確認する
Found multiple elements with the text	同じテキストが複数ある	<code>getAllByText()</code> を使うか、 <code>getByRole()</code> で絞り込む
<code>act()</code> warning	状態更新がテスト外で起きている	<code>waitFor()</code> や <code>act()</code> で非同期処理を待つ
Not wrapped in <code>act(...)</code>	React の状態更新を正しく待てていない	<code>await waitFor(() => { ... })</code> でラップする

21.4.5 モック（Mock）の基礎知識

モック（Mock）とは？ テストでは外部サービス（データベース、API、メール送信等）に実際にアクセスしたくありません。モックは「本物のフリをする偽物」です。

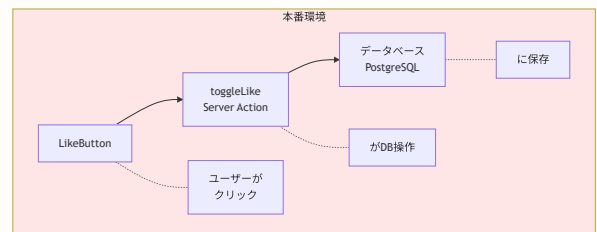
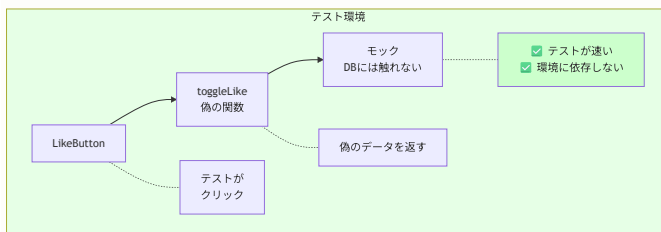
```
// 本物: 実際にデータベースにアクセスする
const user = await prisma.user.findUnique({ where: { id: '123' } })

// モック: データベースにアクセスせず、事前に設定した値を返す
prisma.user.findUnique.mockResolvedValue({ id: '123', name: '太郎' })
```

モックを使う利点：

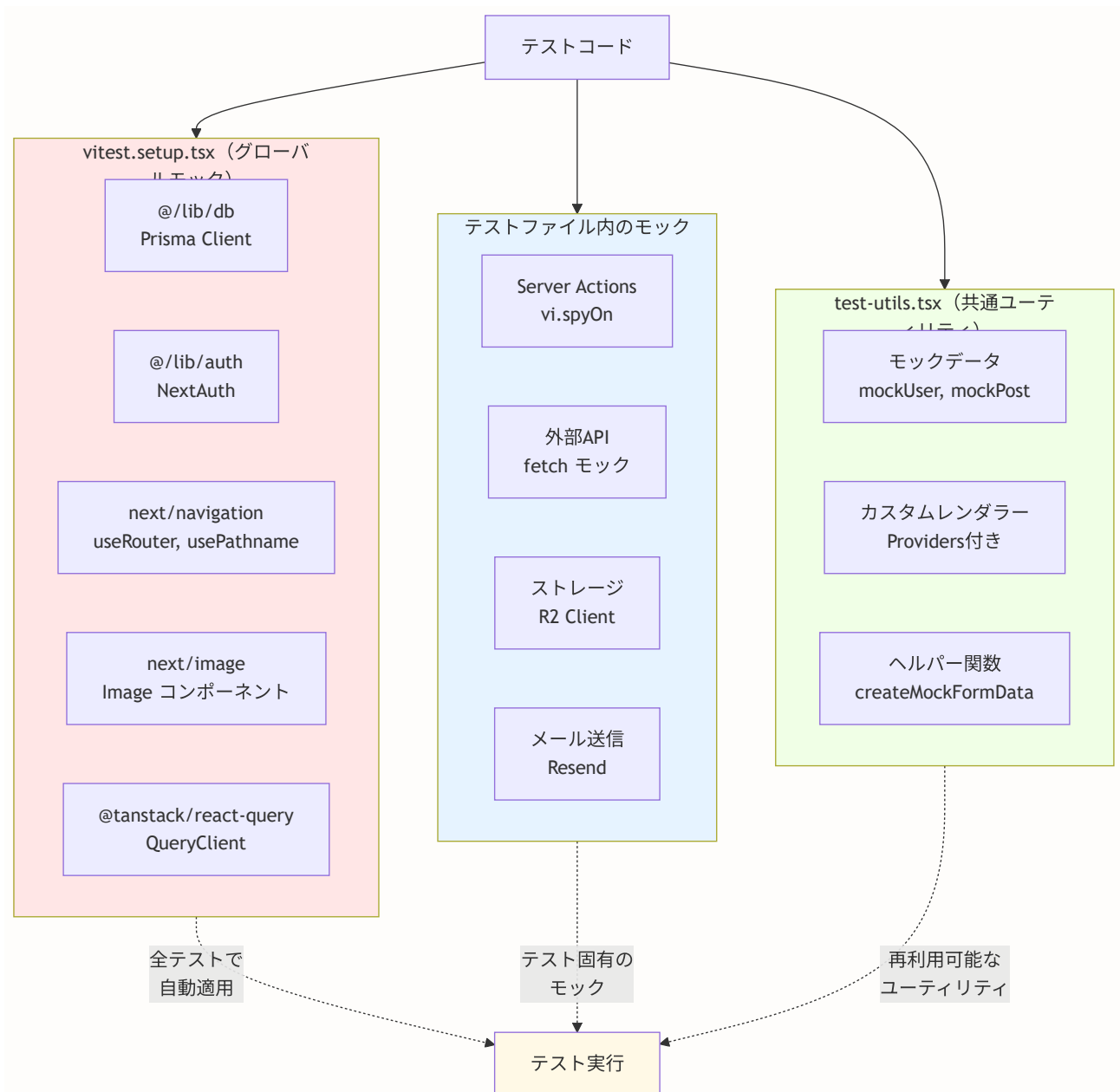
- テストが**高速**（DB接続不要）
- テストが**安定**（外部サービスの状態に依存しない）
- **エラーケースのテスト**が簡単（「DB接続失敗」を再現できる）

モックとは、テスト中に使う「偽物」のことです。テストしたい対象が外部のもの（API、データベースなど）に依存している場合、その外部依存を「偽物」に置き換えることで、テスト対象だけを独立してテストできます。



モックアーキテクチャの全体像

BON-LOGプロジェクトで使用する主要なモックの構成を見てみましょう。



モックの階層

1. **グローバルモック**: 全テストで使われる基本的なモック (vitest.setup.tsx)
2. **テストファイル内モック**: 特定のテストファイルでのみ必要なモック
3. **共通ユーティリティ**: 再利用可能なモックデータとヘルパー関数

Vitest で使える主なモック手法は以下の通りです。

モック手法	用途	使用例
<code>vi.fn()</code>	偽の関数を作る	<code>const onClick = vi.fn()</code>
<code>vi.mock('モジュール名')</code>	モジュール全体を偽物に置換	<code>vi.mock('@lib/db')</code>
<code>vi.spyOn(obj, 'method')</code>	既存の関数を監視+置換	<code>vi.spyOn(actions, 'toggleLike')</code>
<code>.mockResolvedValue(値)</code>	非同期関数の戻り値を指定	<code>fn.mockResolvedValue({ success: true })</code>
<code>.mockReturnValue(値)</code>	同期関数の戻り値を指定	<code>fn.mockReturnValue(42)</code>

components/post/LikeButton.test.tsx

```

import { render, screen, fireEvent, waitFor } from '@testing-library/react'
import { LikeButton } from './LikeButton'
// いいね操作の Server Action をインポート（後でモック化する）
import * as likeActions from '@lib/actions/like'

// vi.mock() でモジュール全体をモック化
// これにより、このテストファイル内では likeActions の全関数が偽物になる
vi.mock('@lib/actions/like')

describe('LikeButton', () => {

  // 各テストの前にモックをリセット
  // 前のテストの呼び出し記録が残らないようにする
  beforeEach(() => {
    vi.clearAllMocks()
  })

  // テスト1: いいね済みの状態で正しいスタイルが適用されるか
  it('should render unlike button when liked', () => {
    // initialLiked=true でいいね済みの状態をシミュレート
    render(<LikeButton postId="post1" initialLiked={true} initialCount={5} />)

    const button = screen.getByRole('button')
    // いいね済みのスタイル (liked クラス) が適用されているか確認
    expect(button).toHaveClass('liked')
  })

  // テスト2: クリック時に toggleLike が正しい引数で呼ばれるか
  it('should toggle like state on click', async () => {
    // spyOn でモック関数を作成し、戻り値を設定
    // mockResolvedValue は Promise.resolve() と同じ（非同期関数の戻り値）
    const mockToggleLike = vi.spyOn(likeActions, 'toggleLike')
    mockToggleLike.mockResolvedValue({ success: true, liked: true })

    render(<LikeButton postId="post1" initialLiked={false} initialCount={5} />)

    // ボタンをクリック
    const button = screen.getByRole('button')
    fireEvent.click(button)

    // waitFor: 非同期処理（Server Action の呼び出し）が完了するまで待つ
    await waitFor(() => {
      // toggleLike が 'post1' を引数に呼ばれたことを確認
      expect(mockToggleLike).toHaveBeenCalledWith('post1')
    })
  })

  // テスト3: いいね成功時にカウントが増えるか
  it('should update like count on successful toggle', async () => {

```

```

const mockToggleLike = vi.spyOn(likeActions, 'toggleLike')
mockToggleLike.mockResolvedValue({ success: true, liked: true })

// 初期カウント: 5、いいねしていない状態
render(<LikeButton postId="post1" initialLiked={false} initialCount={5} />)

const button = screen.getByRole('button')
fireEvent.click(button)

// いいね成功後、カウントが 5 → 6 に増えることを確認
await waitFor(() => {
  expect(screen.getByText('6')).toBeInTheDocument()
})
})

// テスト4: エラー時にカウントが変わらないか (エラーハンドリングのテスト)
it('should handle error gracefully', async () => {
  const mockToggleLike = vi.spyOn(likeActions, 'toggleLike')
  // エラーを返すようにモック設定
  mockToggleLike.mockResolvedValue({ error: 'エラーが発生しました' })

  render(<LikeButton postId="post1" initialLiked={false} initialCount={5} />)

  const button = screen.getByRole('button')
  fireEvent.click(button)

  // エラー時はカウントが変わらない (5のまま) ことを確認
  await waitFor(() => {
    expect(screen.getByText('5')).toBeInTheDocument()
  })
})
})

```

▶ 理解度チェック (クリックで回答を確認)

21.5 Server Action のテスト

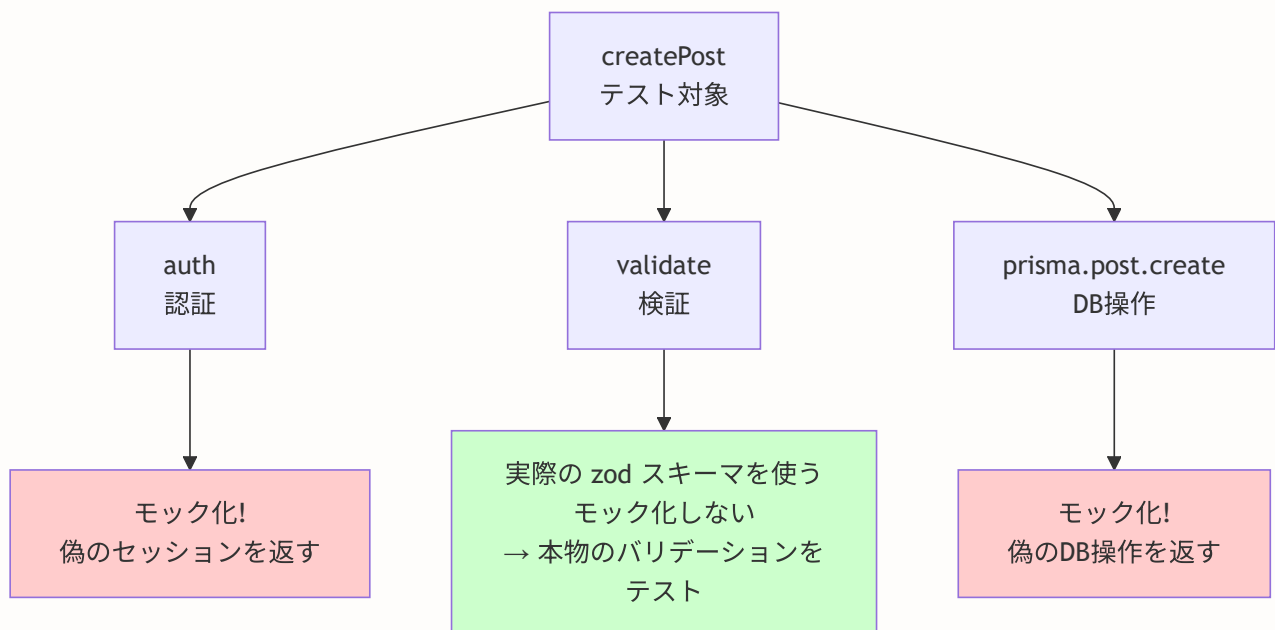
▶ このセクションで学ぶこと (クリックで展開)

BON-LOGでの使用箇所: `__tests__/actions/` ディレクトリ以下にServer Actionのテストファイルが配置されています。`post.test.ts` (投稿作成・削除)、`auth.test.ts` (認証フロー)、`comment.test.ts` (コメント操作) などが実装されています。`vitest.setup.tsx` でPrismaと認証のグローバルモックが設定されており、各テストファイルで `vi.mocked()` を使ってモックの戻り値を上書きします。

実装しない場合の影響: Server Actionのテストがないと、認証チェックの漏れ（誰でも投稿削除できるバグなど）や、バリデーションの抜け（不正な文字列がDBに保存されるバグ）が本番で発見されるリスクがあります。特に認証・認可に関するロジックは、テストによる自動検証が重要です。

21.5.1 Server Action テストの全体像

Server Action は、フォーム送信やデータ変更を処理するサーバーサイドの関数です。テスト時には、データベース（Prisma）と認証（auth）をモック化して、ロジックだけを検証します。



21.5.2 Prisma と auth のモック設定

lib/actions/post.test.ts

```
// テスト対象の Server Action をインポート
import { createPost, deletePost } from './post'
// モック対象のモジュールをインポート
import { prisma } from '@lib/db'
import { auth } from '@lib/auth'

// =====
// Prisma クライアントをモック化
// 実際のデータベース操作を偽物に置き換える
// =====
vi.mock('@lib/db', () => ({
  prisma: {
    post: {
      create: vi.fn(),      // 投稿作成をモック
      delete: vi.fn(),      // 投稿削除をモック
      findUnique: vi.fn(),  // 投稿検索をモック
    },
  },
}))

// =====
// auth() 関数をモック化
// 実際の認証処理を偽物に置き換える
// =====
vi.mock('@lib/auth', () => ({
  auth: vi.fn(),
}))

describe('post actions', () => {

  // 各テストの前にモックをリセット（テスト間の独立性を保証）
  beforeEach(() => {
    vi.clearAllMocks()
  })

  // =====
  // createPost のテスト
  // =====
  describe('createPost', () => {

    // テスト1: 正常系 - 正しいデータで投稿を作成できるか
    it('should create post with valid data', async () => {
      // Arrange: auth() がログイン済みユーザーを返すように設定
      (auth as ReturnType<typeof vi.fn>).mockResolvedValue({
        user: { id: 'user1', email: 'test@example.com' },
      })
    })
  })
})
```

```

// Arrange: prisma.post.create() が作成された投稿を返すように設定
const mockPost = {
  id: 'post1',
  userId: 'user1',
  content: 'テスト投稿',
  createdAt: new Date(),
}
;(prisma.post.create as ReturnType<typeof vi.fn>).mockResolvedValue(mockPost)

// Act: createPost を呼び出し
const result = await createPost({
  content: 'テスト投稿',
  genreIds: ['genre1'],
})

// Assert: 成功結果を確認
expect(result.success).toBe(true)
expect(result.postId).toBe('post1')

// Assert: Prisma が正しい引数で呼ばれたか確認
// expect.objectContaining() は、指定したキーが含まれていればOK
expect(prisma.post.create).toHaveBeenCalledWith({
  data: expect.objectContaining({
    userId: 'user1',
    content: 'テスト投稿',
  }),
})
})

// テスト2: 異常系 - 未認証ユーザーはエラーになるか
it('should return error when not authenticated', async () => {
  // Arrange: auth() が null を返す (= 未ログイン)
  (auth as ReturnType<typeof vi.fn>).mockResolvedValue(null)

  // Act
  const result = await createPost({
    content: 'テスト投稿',
    genreIds: ['genre1'],
  })

  // Assert: エラーメッセージが返る
  expect(result.error).toBe('認証が必要です')
  // Assert: DB操作は行われない (重要!)
  expect(prisma.post.create).not.toHaveBeenCalled()
})

// テスト3: 異常系 - 不正なデータはバリデーションで弾かれるか
it('should return error with invalid data', async () => {
  // Arrange: 認証は通す
  (auth as ReturnType<typeof vi.fn>).mockResolvedValue({
    user: { id: 'user1' },
  })
})

```

```

// Act: 空のコンテンツで投稿を試みる
const result = await createPost({
  content: '', // 空文字は不正
  genreIds: ['genre1'],
})

// Assert: エラーが返る
expect(result.error).toBeDefined()
// Assert: バリデーション失敗時もDB操作は行われない
expect(prisma.post.create).not.toHaveBeenCalled()
})
})

// =====
// deletePost のテスト
// =====
describe('deletePost', () => {

  // テスト4: 正常系 - 投稿者本人なら削除できるか
  it('should delete post when user is owner', async () => {
    // Arrange: user1 としてログイン
    (auth as ReturnType<typeof vi.fn>).mockResolvedValue({
      user: { id: 'user1' },
    })

    // Arrange: 投稿の所有者は user1 (= ログインユーザーと同じ)
    const mockPost = {
      id: 'post1',
      userId: 'user1',
    }
    ;(prisma.post.findUnique as ReturnType<typeof vi.fn>).mockResolvedValue(mockPost)
    ;(prisma.post.delete as ReturnType<typeof vi.fn>).mockResolvedValue(mockPost)

    // Act
    const result = await deletePost('post1')

    // Assert: 削除成功
    expect(result.success).toBe(true)
    expect(prisma.post.delete).toHaveBeenCalledWith({
      where: { id: 'post1' },
    })
  })

  // テスト5: 異常系 - 他人の投稿は削除できないか (権限チェック)
  it('should return error when user is not owner', async () => {
    // Arrange: user2 としてログイン (投稿者は user1)
    (auth as ReturnType<typeof vi.fn>).mockResolvedValue({
      user: { id: 'user2' }, // 別のユーザー
    })

    // Arrange: 投稿の所有者は user1
    const mockPost = {

```

```

    id: 'post1',
    userId: 'user1',          // user2 ≠ user1
  }
;(prisma.post.findUnique as ReturnType<typeof vi.fn>).mockResolvedValue(mockPost)

// Act
const result = await deletePost('post1')

// Assert: 権限エラーが返る
expect(result.error).toBe('削除する権限がありません')
// Assert: 削除操作は行われない（最も重要な確認！）
expect(prisma.post.delete).not.toHaveBeenCalled()
  })
})
})

```

重要なポイント: Server Action のテストでは、「DB操作が呼ばれないこと」の確認

（`not.toHaveBeenCalled()`）が非常に重要です。認証やバリデーションで弾かれた場合に、誤ってデータベースが操作されないことを保証します。

▶ 理解度チェック（クリックで回答を確認）

21.6 E2Eテスト（Playwright）

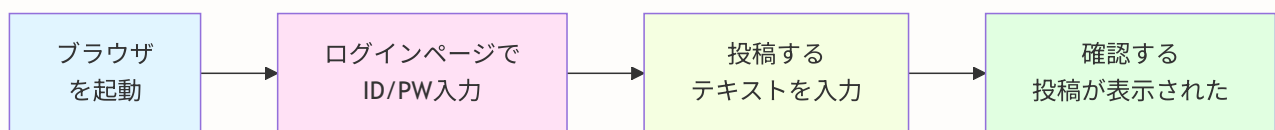
▶ このセクションで学ぶこと（クリックで展開）

BON-LOGでの使用箇所: `playwright.config.ts`（プロジェクトルート）と `e2e/` ディレクトリに実装されています。`chromium`（認証済み）と `chromium-noauth`（未認証）の2プロジェクト構成で、CIでは `npx playwright test --project=chromium --project=chromium-noauth` で実行されます。E2Eジョブは現在手動実行推奨（CIで実質無効化）で、ローカルでは `npm run test:e2e` で実行できます。

実装しない場合の影響: E2Eテストがないと、ログインフロー・投稿フローなどのユーザー操作全体の結合テストが行われません。ユニットテストだけでは検出できない「APIとUIの接続バグ」や「ページ遷移の問題」が本番に混入するリスクがあります。

21.6.1 E2E テストとは？

E2E（エンドツーエンド）テストは、**実際のブラウザ**を自動操作して、ユーザーの操作フローを丸ごとテストする手法です。



ポイント

- 全て自動で実行される（人間がブラウザを触る必要なし）
- 実際のサーバー + 実際のDB + 実際のブラウザ を使う

ユニットテストとの違いを整理しましょう。

項目	ユニットテスト	E2E テスト
テスト対象	関数1つ / コンポーネント1つ	ユーザーの操作フロー全体
実行環境	Node.js（仮想DOM）	実際のブラウザ
実行速度	速い（ミリ秒）	遅い（秒～分）
DB	モック（偽物）	実際のテスト用DB
検出できるバグ	ロジックのバグ	UI / API / DB 連携のバグ

21.6.2 Playwright のセットアップ

Playwright（プレイライト）は、Microsoft が開発した E2E テストフレームワークです。Chrome、Firefox、Safari の3つのブラウザをサポートしています。

```
# Playwright のインストール
npm install -D @playwright/test

# ブラウザエンジンのインストール (Chrome, Firefox, Safari)
npx playwright install
```

21.6.3 playwright.config.ts

Playwright の設定ファイルです。各設定項目を詳しく見ていきましょう。

```

import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  // テストファイルのディレクトリ
  testDir: './e2e',

  // テストを並列実行する（高速化のため）
  fullyParallel: true,

  // CI 環境では .only() の使用を禁止（テストの絞り込み忘れを防ぐ）
  forbidOnly: !!process.env.CI,

  // CI 環境では失敗時に2回リトライ（不安定なテストへの対策）
  retries: process.env.CI ? 2 : 0,

  // CI 環境ではワーカー数を1に制限（安定性のため）
  workers: process.env.CI ? 1 : undefined,

  // テスト結果をHTMLレポートで出力
  reporter: 'html',

  // 全テスト共通の設定
  use: {
    // テスト対象のアプリケーションURL
    baseURL: 'http://localhost:3000',
    // 初回リトライ時にトレース（操作記録）を保存
    // → 失敗原因の調査に使える
    trace: 'on-first-retry',
  },

  // テスト対象のブラウザ（複数ブラウザでテスト）
  projects: [
    {
      name: 'chromium', // Google Chrome
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox', // Mozilla Firefox
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit', // Apple Safari
      use: { ...devices['Desktop Safari'] },
    },
  ],

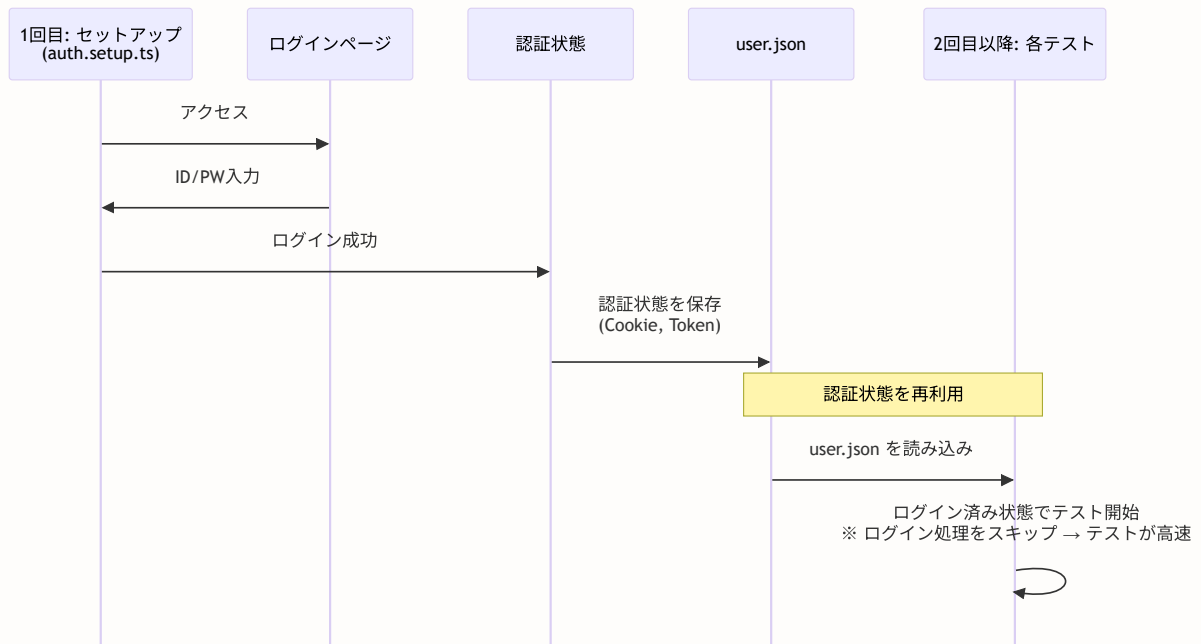
  // テスト実行時に開発サーバーを自動起動
  webServer: {
    command: 'npm run dev', // 起動コマンド
    url: 'http://localhost:3000', // サーバーの URL
    reuseExistingServer: !process.env.CI, // ローカルでは既存サーバーを再利用
  }
})

```

```
} ,  
})
```

21.6.4 認証状態の再利用

E2E テストでは、多くのテストケースがログイン済み状態を前提とします。毎回ログイン処理を行うと遅いため、認証状態を保存して再利用します。



e2e/auth.setup.ts

```
import { test as setup, expect } from '@playwright/test'

// 認証状態の保存先ファイルパス
const authFile = 'playwright/.auth/user.json'

// セットアップ: テスト実行前に1回だけ実行される
setup('authenticate', async ({ page }) => {
  // ログインページにアクセス
  await page.goto('/login')

  // メールアドレスを入力
  await page.fill('input[name="email"]', 'test@example.com')
  // パスワードを入力
  await page.fill('input[name="password"]', 'password123')

  // ログインボタンをクリック
  await page.click('button[type="submit"]')

  // フィードページにリダイレクトされるまで待つ
  await page.waitForURL('/feed')

  // 認証状態 (Cookie やローカルストレージ) をファイルに保存
  // 他のテストはこのファイルを読み込んでログイン済み状態を再現する
  await page.context().storageState({ path: authFile })
})
```

21.6.5 E2Eテストの実例

e2e/post.spec.ts - 投稿関連のテスト

```
import { test, expect } from '@playwright/test'

// 投稿作成のテストグループ
test.describe('Post creation', () => {
  // 保存した認証状態を使う (= ログイン済みの状態でテスト開始)
  test.use({ storageState: 'playwright/.auth/user.json' })

  // テスト1: 新しい投稿を作成できるか
  test('should create a new post', async ({ page }) => {
    // フィードページに移動
    await page.goto('/feed')

    // 投稿フォームにテキストを入力
    await page.fill('textarea[name="content"]', 'テストE2E投稿です')

    // ジャンルを選択 (松柏類ボタンをクリック)
    await page.click('button[data-genre="松柏類"]')

    // 投稿ボタンをクリック
    await page.click('button:has-text("投稿")')

    // 投稿した内容がタイムラインに表示されることを確認
    // toBeVisible() は要素が画面上に見えることを検証
    await expect(page.locator('text=テストE2E投稿です')).toBeVisible()
  })

  // テスト2: いいね機能が動作するか
  test('should like a post', async ({ page }) => {
    await page.goto('/feed')

    // 最初の投稿のいいねボタンを取得
    const likeButton = page.locator('[data-testid="like-button"]').first()
    // 現在のいいね数を取得
    const initialCount = await likeButton.locator('text=/\\d+/').textContent()

    // いいねボタンをクリック
    await likeButton.click()

    // いいね数が1増えることを確認
    await expect(likeButton.locator('text=/\\d+/')).toHaveText(
      String(parseInt(initialCount || '0') + 1)
    )
  })

  // テスト3: コメント機能が動作するか
  test('should comment on a post', async ({ page }) => {
```

```

    await page.goto('/feed')

    // 最初の投稿カードをクリックして詳細ページへ
    await page.locator('[data-testid="post-card"]').first().click()

    // コメント入力欄にテキストを入力
    await page.fill('textarea[name="content"]', 'テストコメントです')
    // コメントボタンをクリック
    await page.click('button:has-text("コメント")')

    // コメントが表示されることを確認
    await expect(page.locator('text=テストコメントです')).toBeVisible()
  })
})

// ユーザープロフィールのテストグループ
test.describe('User profile', () => {
  test.use({ storageState: 'playwright/.auth/user.json' })

  // テスト4: プロフィールを更新できるか
  test('should update profile', async ({ page }) => {
    // 設定ページに移動
    await page.goto('/settings/profile')

    // ニックネームを変更
    await page.fill('input[name="nickname"]', '新しいニックネーム')
    // 自己紹介文を変更
    await page.fill('textarea[name="bio"]', '新しい自己紹介文')

    // 保存ボタンをクリック
    await page.click('button:has-text("保存")')

    // 成功メッセージが表示されることを確認
    await expect(page.locator('text=更新しました')).toBeVisible()
  })

  // テスト5: フォロー機能が動作するか
  test('should follow a user', async ({ page }) => {
    // 他のユーザーのプロフィールページに移動
    await page.goto('/users/user123')

    // フォローボタンをクリック
    await page.click('button:has-text("フォロー")')

    // ボタンの表示が「フォロー」→「フォロー中」に変わることを確認
    await expect(page.locator('button:has-text("フォロー中")')).toBeVisible()
  })
})

```

e2e/search.spec.ts - 検索機能のテスト

```
import { test, expect } from '@playwright/test'

test.describe('Search', () => {

  // テスト1: キーワード検索が動作するか
  test('should search posts by keyword', async ({ page }) => {
    // 検索ページに移動
    await page.goto('/search')

    // 検索キーワードを入力
    await page.fill('input[name="query"]', '盆栽')
    // Enter キーを押して検索実行
    await page.press('input[name="query"]', 'Enter')

    // 検索結果が1件以上表示されることを確認
    await expect(page.locator('[data-testid="post-card"]')).toHaveCount(
      { min: 1 }
    )

    // 検索結果に「盆栽」を含む投稿があることを確認
    const posts = await page.locator('[data-testid="post-content"]').allTextContents()
    expect(posts.some(content => content.includes('盆栽'))).toBe(true)
  })

  // テスト2: ジャンルフィルタが動作するか
  test('should filter by genre', async ({ page }) => {
    await page.goto('/search')

    // 「松柏類」ジャンルフィルタをクリック
    await page.click('button:has-text("松柏類")')

    // フィルタが適用され、松柏類のバッジが表示されることを確認
    await expect(page.locator('[data-testid="genre-badge"]:has-text("松柏類")')).toHaveCount(
      { min: 1 }
    )
  })
})
```


よくあるトラブルと解決法（E2Eテスト編）

トラブル	原因	解決法
<code>Timeout exceeded</code>	要素の表示待ちでタイムアウト	<code>timeout</code> オプションを増やすか、セレクトタを見直す
テストが不安定（たまに失敗）	レースコンディション	<code>waitFor</code> / <code>expect.toBeVisible()</code> を使って明示的に待つ
ログイン状態が保持されない	<code>storageState</code> の設定漏れ	<code>test.use({ storageState: '...' })</code> を確認
開発サーバーが起動しない	ポート競合	<code>webServer.reuseExistingServer</code> を <code>true</code> にして手動でサーバー起動
セレクトタが見つからない	HTML構造が変わった	<code>data-testid</code> 属性を使って安定したセレクトタにする

▶ 理解度チェック（クリックで回答を確認）

21.7 テストカバレッジ

▶ このセクションで学ぶこと（クリックで展開）

BON-LOGでの使用箇所: `vitest.config.ts` のカバレッジ設定（`coverage.thresholds`）で閾値が定義されています。`branches: 30`、`functions: 30`、`lines: 35`、`statements: 35` が最低ラインとして設定されており、これを下回るとテストが失敗します。CIでは `npx vitest run --coverage` でカバレッジを計測し、`coverage/` ディレクトリにHTMLレポートとlcovファイルが生成されます。GitHub Actionsのアーティファクトとして7日間保存されます。

実装しない場合の影響: カバレッジ閾値の設定がないと、テストを追加せずにコードを増やし続けてもCIが通ってしまいます。目標値は低め（30～35%）に設定されていますが、これはプロジェクト初期の現実的な目標です。機能追加と並行してテストを追加していくことで、徐々に引き上げることができます。

21.7.1 テストカバレッジとは？

テストカバレッジとは、「テストによって実行されたコードの割合」を数値で示したものです。

身近なたとえで考えましょう。100ページある本の校正をしているとします。

カバレッジのたとえ：

100ページの本を校正する

- └ 80ページをチェックした → カバレッジ 80%
- └ 残り20ページは未チェック → 誤字が残っているかも ...
- └ 100ページ全てチェック → カバレッジ 100%（理想的）

※ ただし「全ページ読んだ」からといって
「全ての誤字を見つけた」とは限らない
→ カバレッジ100% ≠ バグゼロ

21.7.2 カバレッジの4つの種類

種類	意味	例
ステートメント	文（コードの各行）の実行率	全50行中40行が実行 → 80%
ブランチ	分岐（if/else）の網羅率	全10分岐中7分岐をテスト → 70%
関数	関数の実行率	全20関数中16関数を呼び出し → 80%
ライン	行の実行率	ステートメントとほぼ同じ

ブランチカバレッジの例:

```
function checkAge(age: number) {  
  if (age ≥ 18) {           // ← 分岐1: true の場合  
    return '成人'  
  } else {                 // ← 分岐2: false の場合  
    return '未成年'  
  }  
}
```

テスト: checkAge(20) だけの場合

→ 分岐1 (true) のみテスト → ブランチカバレッジ 50%

テスト: checkAge(20) と checkAge(15) の場合

→ 両方の分岐をテスト → ブランチカバレッジ 100%

21.7.3 カバレッジレポートの生成と確認

```
# カバレッジレポートを生成  
npm run test:coverage  
  
# HTMLレポートを開く（ブラウザで詳細を確認）  
# macOS の場合:  
open coverage/lcov-report/index.html  
# Windows の場合:  
start coverage/lcov-report/index.html
```

21.7.4 カバレッジの目標値

BON-LOG プロジェクトでの推奨目標値です。

カバレッジ種類	目標値	理由
ステートメント	80%以上	主要なコードパスは全て通す
ブランチ	75%以上	重要な分岐（エラーハンドリング等）は網羅
関数	80%以上	公開 API は全てテスト
ライン	80%以上	ステートメントとほぼ同じ基準

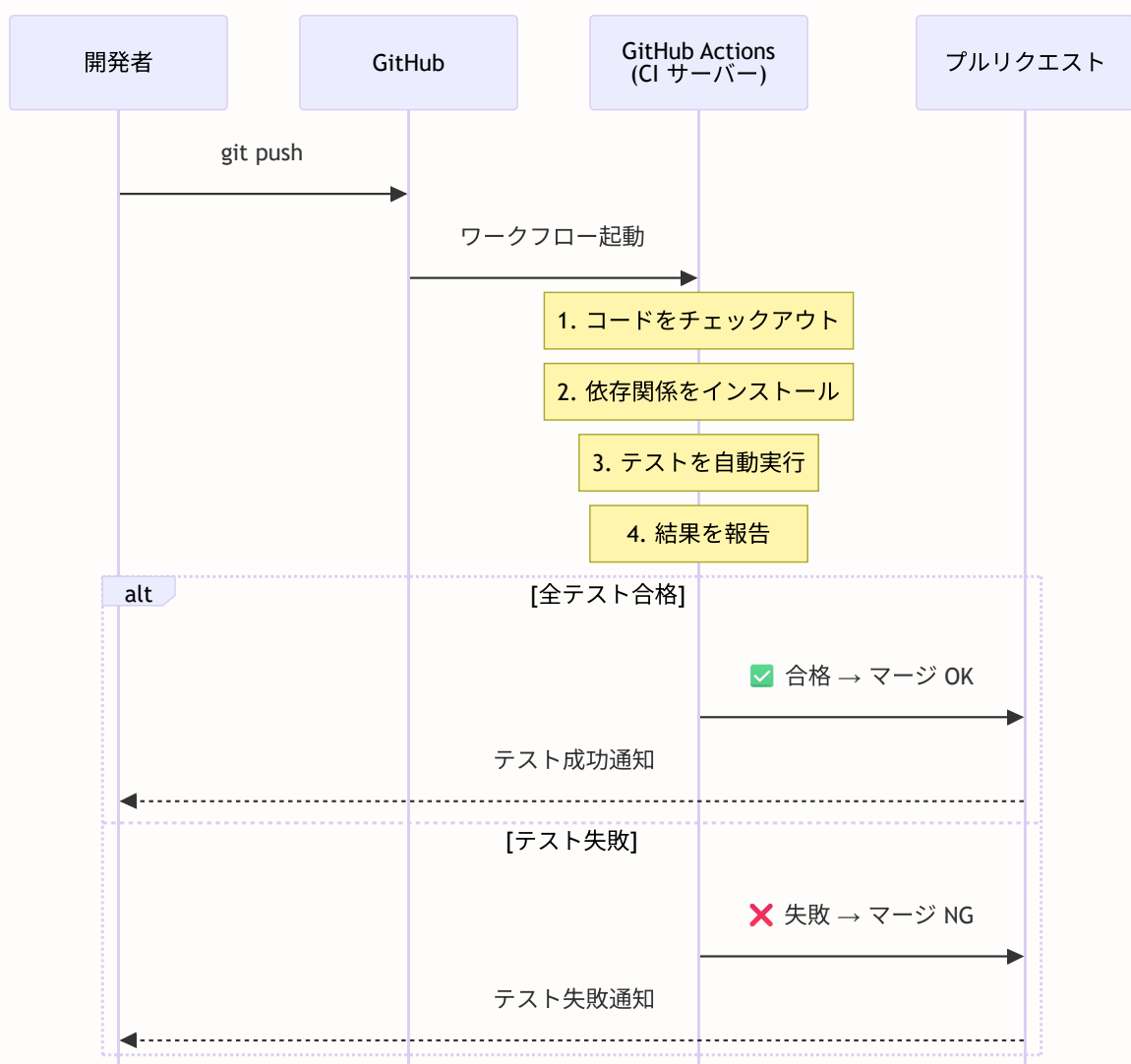
注意: カバレッジ100%を目標にする必要はありません。100%を目指すと、テストのメンテナンスコストが非常に高くなります。重要なビジネスロジック（認証、投稿、いいねなど）を優先的にテストしましょう。

21.8 CI でのテスト自動実行

▶ このセクションで学ぶこと（クリックで展開）

21.8.1 CI とは？

CI（継続的インテグレーション） とは、コードの変更をリポジトリにプッシュするたびに、自動的にテストを実行する仕組みです。



21.8.2 GitHub Actions ワークフロー

`.github/workflows/ci.yml`

```
# ワークフローの名前 (GitHub 上で表示される)
name: CI

# トリガー: いつこのワークフローを実行するか
on:
  push:
    branches: [main, master]          # main, master へのプッシュ時
  pull_request:
    branches: [main, master]          # main, master への PR 作成時

# 同じブランチの古いワークフローをキャンセル
# 例: PRに連続でpushした場合、前のワークフローを止めて最新だけ実行
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

# 全ジョブ共通の環境変数
env:
  NODE_VERSION: '20'
  # Prisma用ダミー環境変数 (ユニットテストでは実際のDB接続は不要)
  DATABASE_URL: "postgresql://dummy:dummy@localhost:5432/dummy"
  DIRECT_URL: "postgresql://dummy:dummy@localhost:5432/dummy"

jobs:
  # =====
  # ジョブ1: lint (コード品質チェック)
  # =====
  lint:
    name: Lint & Type Check
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Generate Prisma Client
        run: npx prisma generate
```

```

# ESLintでコードスタイルをチェック
- name: Run ESLint
  run: npm run lint

# TypeScriptの型エラーがないか確認
- name: Run TypeScript type check
  run: npx tsc --noEmit

# =====
# ジョブ2: test (ユニットテスト)
# =====
# 注意: ユニットテストではDBサービスは不要
# モックを使ってテストするため、ダミーのDATABASE_URLで十分
test:
  name: Unit Tests
  runs-on: ubuntu-latest          # Ubuntu の最新版で実行
  steps:
    # ステップ1: リポジトリのコードを取得
    - name: Checkout
      uses: actions/checkout@v4

    # ステップ2: Node.js をセットアップ
    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'npm'              # npm キャッシュで高速化

    # ステップ3: 依存関係のインストール
    - name: Install dependencies
      run: npm ci                 # ci は install より高速・厳密

    # ステップ4: Prisma クライアントの生成
    - name: Generate Prisma Client
      run: npx prisma generate

    # ステップ5: テスト実行 (カバレッジ付き)
    # --ci: CI環境向けの最適化 (スナップショット自動更新無効化等)
    # --maxWorkers=2: 並列数を制限してメモリ消費を抑える
    - name: Run unit tests
      run: npx vitest run --ci --coverage --maxWorkers=2
      env:
        NODE_OPTIONS: "--max-old-space-size=4096"

    # ステップ6: カバレッジレポートのアップロード
    # アーティファクトとして保存 (GitHubのActionsタブからダウンロード可能)
    - name: Upload coverage report
      uses: actions/upload-artifact@v4
      if: always()
      with:
        name: coverage-report
        path: coverage/
        retention-days: 7

```

```

# =====
# ジョブ3: build (ビルド確認)
# =====
build:
  name: Build
  runs-on: ubuntu-latest
  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Generate Prisma Client
      run: npx prisma generate

    # ビルド時にはDBに接続しないが、環境変数の参照でエラーにならないよう
    # ダミーの値を設定
    - name: Build application
      run: npm run build
      env:
        NEXTAUTH_SECRET: "build-time-secret"
        NEXTAUTH_URL: "http://localhost:3000"
        NODE_OPTIONS: "--max-old-space-size=4096"

# =====
# ジョブ4: E2E テスト
# =====
# lint, test, build が全て成功してから実行
e2e:
  name: E2E Tests
  runs-on: ubuntu-latest
  needs: [lint, test, build]      # 前の3ジョブが成功してから実行
  timeout-minutes: 15            # 最大15分でタイムアウト

  # E2Eテストでは実際のDBが必要
  services:
    postgres:
      image: postgres:16-alpine
      env:
        POSTGRES_USER: postgres
        POSTGRES_PASSWORD: postgres
        POSTGRES_DB: bonsai_sns_test    # E2Eテスト専用のDB名
      ports:
        - 5432:5432
      options: >-

```



```

--health-cmd pg_isready
--health-interval 10s
--health-timeout 5s
--health-retries 5

```

steps:

```

- name: Checkout
  uses: actions/checkout@v4

- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: ${ env.NODE_VERSION }
    cache: 'npm'

- name: Install dependencies
  run: npm ci

- name: Generate Prisma Client
  run: npx prisma generate

# DBスキーマを適用
- name: Setup database
  run: npx prisma db push
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"

# テスト用の初期データ（シードデータ）を投入
- name: Seed test data
  run: npx prisma db seed
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"

# Playwright のブラウザをインストール (chromiumのみで高速化)
- name: Install Playwright browsers
  run: npx playwright install --with-deps chromium

# アプリケーションをビルド
- name: Build application
  run: npm run build
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    NEXTAUTH_SECRET: "ci-e2e-test-secret-key"
    NEXTAUTH_URL: "http://localhost:3000"
    NEXT_PUBLIC_APP_URL: "http://localhost:3000"
    AUTH_TRUST_HOST: "true"
    NODE_OPTIONS: "--max-old-space-size=4096"

# standaloneサーバーの準備
- name: Prepare standalone server

```

```

run: |
  cp -r public .next/standalone/public 2>/dev/null || true
  cp -r .next/static .next/standalone/.next/static 2>/dev/null || true
  cp -r node_modules/.prisma .next/standalone/node_modules/.prisma 2>/dev/null ||
  cp -r node_modules/@prisma .next/standalone/node_modules/@prisma 2>/dev/null ||

# サーバーを起動してヘルスチェック
- name: Start server and check health
  run: |
    node .next/standalone/server.js &
    SERVER_PID=$!
    echo "Waiting for server to start..."
    for i in $(seq 1 30); do
      if curl -s http://localhost:3000/login > /dev/null 2>&1; then
        echo "Server is ready"
        break
      fi
      sleep 2
    done
    kill $SERVER_PID 2>/dev/null || true
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    NEXTAUTH_SECRET: "ci-e2e-test-secret-key"
    NEXTAUTH_URL: "http://localhost:3000"
    AUTH_TRUST_HOST: "true"
    PORT: "3000"
    HOSTNAME: "0.0.0.0"
    NODE_ENV: "production"

# E2E テスト実行 (chromiumプロジェクトのみ)
- name: Run E2E tests
  run: npx playwright test --project=chromium --project=chromium-noauth
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    NEXTAUTH_SECRET: "ci-e2e-test-secret-key"
    NEXTAUTH_URL: "http://localhost:3000"
    AUTH_TRUST_HOST: "true"
    PORT: "3000"
    HOSTNAME: "0.0.0.0"
    NODE_ENV: "production"
  timeout-minutes: 15

# テスト結果レポートをアーティファクトとして保存
# always() により、テスト失敗時もレポートを保存
- name: Upload Playwright report
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: playwright-report

```

```
path: playwright-report/  
retention-days: 7
```

▶ 理解度チェック（クリックで回答を確認）

21.9 演習問題

テストの理解を深めるために、段階的な演習問題に取り組みましょう。

演習1: 基礎 - ユーティリティ関数のテスト

以下の `truncateText` 関数に対するテストを書いてください。

```
// lib/utils/text.ts  
// 指定された長さでテキストを切り詰める関数  
export function truncateText(text: string, maxLength: number): string {  
  if (text.length ≤ maxLength) return text  
  return text.slice(0, maxLength) + ' ... '  
}
```

要件:

- 短いテキスト（maxLength 以下）はそのまま返る
- 長いテキスト（maxLength 超）は切り詰められて `...` が付く
- 空文字列の場合はそのまま返る
- ちょうど maxLength の場合はそのまま返る（境界値テスト）

▶ 解答例（まず自分で書いてから確認しましょう）

演習2: 基礎 - コンポーネントの表示テスト

以下の `UserBadge` コンポーネントに対するテストを書いてください。

```
// components/user/UserBadge.tsx
type Props = {
  nickname: string
  isOnline: boolean
}

export function UserBadge({ nickname, isOnline }: Props) {
  return (
    <div data-testid="user-badge">
      <span>{nickname}</span>
      {isOnline && <span className="online-indicator">オンライン</span>}
    </div>
  )
}
```

要件:

- ニックネームが表示される
- オンライン時に「オンライン」が表示される
- オフライン時に「オンライン」が表示されない

▶ 解答例

演習3: 応用 - フォーム入力のテスト

ユーザー登録フォームの統合テストを作成してください。

要件:

- 何も入力せずに送信するとバリデーションエラーが表示される
- 正しい入力値で送信すると成功する
- ローディング状態の表示をテスト
- メールアドレスの形式チェック

▶ ヒント（行き詰ったら確認）

演習4: 応用 - APIエンドポイントのテスト

`/api/posts` エンドポイントのテストを作成してください。

要件:

- GET: 投稿一覧を返す
- POST: 新しい投稿を作成
- 未認証時は 401 エラーを返す
- バリデーションエラー時は 400 エラーを返す
- ページネーションが正しく動作する

▶ ヒント

演習5: チャレンジ - スナップショットテスト

PostCard コンポーネントのスナップショットテストを作成してください。スナップショットテストは、コンポーネントの出力HTMLを保存し、次のテスト実行時に変更がないか自動で比較する手法です。

```
// スナップショットテストの基本
test('should match snapshot', () => {
  const { container } = render(<PostCard post={mockPost} />)
  // 初回はスナップショットファイルが作成される
  // 2回目以降は前回のスナップショットと比較される
  expect(container).toMatchSnapshot()
})
```

要件:

- 通常の投稿のスナップショット
- 画像付き投稿のスナップショット
- いいね0件の投稿のスナップショット
- スナップショットが変わった場合の更新方法を理解する

ヒント: スナップショットを更新するには `npm test -- -u` を実行します。

演習6: チャレンジ - カスタムフックのテスト

以下の `useDebounce` カスタムフックのテストを書いてください。

```
// hooks/useDebounce.ts
import { useState, useEffect } from 'react'

export function useDebounce<T>(value: T, delay: number): T {
  const [debouncedValue, setDebouncedValue] = useState<T>(value)

  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value)
    }, delay)

    return () => clearTimeout(timer)
  }, [value, delay])

  return debouncedValue
}
```

要件:

- 初期値が即座に返される
- 値が変わってもdelay時間が経過するまでは前の値のまま
- delay時間経過後に新しい値が返される
- 連続して値が変わった場合、最後の値だけが反映される

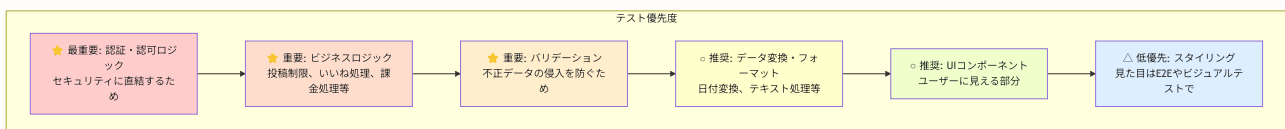
▶ ヒント

21.10 テスト戦略のベストプラクティス

▶ このセクションで学ぶこと（クリックで展開）

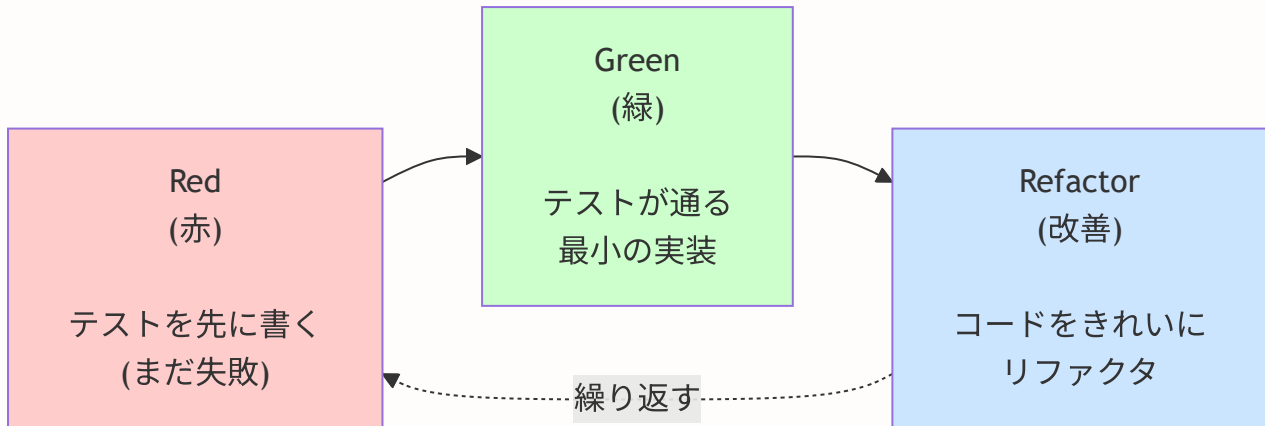
21.10.1 何をテストするか？優先度の考え方

全てのコードにテストを書く必要はありません。**リスクが高い部分**から優先的にテストを書きましょう。



21.10.2 テスト駆動開発（TDD）の基本サイクル

TDD（Test-Driven Development）は、**テストを先に書いてから実装する**開発手法です。



TDDのサイクル

1. **Red**: まずテストを書く（実装がないので失敗する）
2. **Green**: テストが通る最小限のコードを書く
3. **Refactor**: コードをきれいにする（テストは通ったまま）

21.10.3 テストのメンテナンスを楽にするコツ

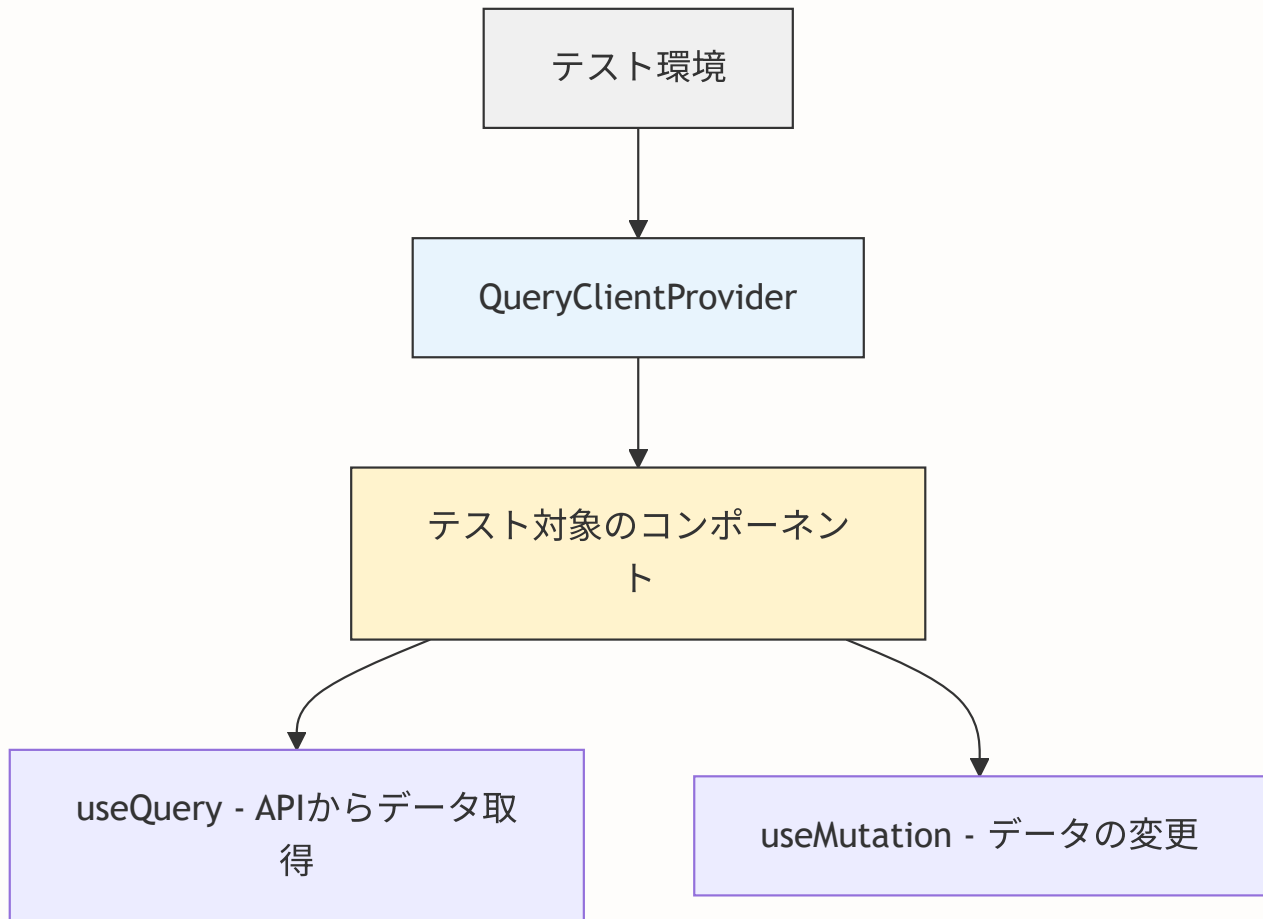
コツ	説明
テストヘルパーを作る	モックデータの生成やセットアップをヘルパー関数にまとめる
実装の詳細に依存しない	CSSクラス名ではなく <code>data-testid</code> や <code>getByRole</code> を使う
1テスト1検証	1つのテストケースで1つのことだけ確認する
テストの説明文を丁寧に	失敗時に何が壊れたかすぐ分かるように書く
共通モックは <code>vitest.setup</code> に	全テスト共通のモックはセットアップファイルに集約

21.11 React Query テスト

▶ このセクションで学ぶこと(クリックで展開)

21.11.1 React Query とテストの関係

BON-LOG では、サーバー状態の管理に **React Query** (**TanStack Query**) を使っています。React Query を使ったコンポーネントのテストでは、`QueryClientProvider` で囲む必要があります。



テスト用 QueryClient の設定ポイント:

- テスト用の QueryClient を使う
- `retry: false` で自動リトライを無効化
- `gcTime: 0` でキャッシュを即座に破棄

21.11.2 テスト用 QueryClient の作成

テスト環境では、本番とは異なる設定の QueryClient を使います。


```
// __tests__/utils/test-utils.tsx に定義されている例

import { QueryClient, QueryClientProvider } from '@tanstack/react-query'

/**
 * テスト用 QueryClient を作成する関数
 *
 * テスト環境では以下の設定が重要:
 * - retry: false → テスト失敗時にリトライしない（即座にエラー検出）
 * - gcTime: 0 → テスト間でキャッシュが残らない（テストの独立性保証）
 * - staleTime: 0 → データを常に最新と判断しない
 */
export function createTestQueryClient() {
  return new QueryClient({
    defaultOptions: {
      queries: {
        retry: false, // 自動リトライを無効化
        gcTime: 0, // ガベージコレクション時間を0に
        staleTime: 0, // キャッシュされたデータを即座にstaleとみなす
      },
      mutations: {
        retry: false, // ミューテーションもリトライしない
      },
    },
    // テスト中のログ出力を抑制
    logger: {
      log: console.log,
      warn: console.warn,
      error: () => {}, // エラーログを抑制（テスト失敗時のノイズ防止）
    },
  })
}

/**
 * テスト用ラッパーコンポーネント
 * React Query を使うコンポーネントのテストで使用
 */
export function createQueryWrapper() {
  const queryClient = createTestQueryClient()
  return function QueryWrapper({ children }: { children: React.ReactNode }) {
    return (
      <QueryClientProvider client={queryClient}>
        {children}
      </QueryClientProvider>
    )
  }
}
```

21.11.3 useQuery を使ったコンポーネントのテスト

投稿一覧を取得する `useQuery` を使ったコンポーネントのテスト例です。

```

import { render, screen, waitFor } from '@testing-library/react'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { PostList } from '@components/post/PostList'

// fetch をモック化してAPIレスポンスを偽装
global.fetch = vi.fn()

// テスト用 QueryClient
function createTestQueryClient() {
  return new QueryClient({
    defaultOptions: {
      queries: { retry: false, gcTime: 0 },
    },
  })
}

// テスト用ラッパー
function renderWithQueryClient(ui: React.ReactElement) {
  const queryClient = createTestQueryClient()
  return render(
    <QueryClientProvider client={queryClient}>
      {ui}
    </QueryClientProvider>
  )
}

describe('PostList (useQuery使用)', () => {
  beforeEach(() => {
    vi.clearAllMocks()
  })

  it('ローディング中にスケルトンが表示される', () => {
    // fetch がまだ解決していない (Pending状態)
    ;(global.fetch as ReturnType<typeof vi.fn>).mockReturnValue(new Promise(() => {}))

    renderWithQueryClient(<PostList />)

    // ローディング中のスケルトンUIが表示される
    expect(screen.getByTestId('post-list-skeleton')).toBeInTheDocument()
  })

  it('投稿一覧が正常に表示される', async () => {
    // fetch が投稿データを返すようにモック
    const mockPosts = [
      { id: '1', content: '黒松の手入れ', user: { nickname: '盆栽太郎' } },
      { id: '2', content: '五葉松の植替え', user: { nickname: '盆栽花子' } },
    ]
    ;(global.fetch as ReturnType<typeof vi.fn>).mockResolvedValueOnce({
      ok: true,
      json: () => Promise.resolve({ posts: mockPosts, nextCursor: null }),
    })
  })
}

```

```

renderWithQueryClient(<PostList />)

// waitFor: 非同期データの取得完了を待つ
await waitFor(() => {
  expect(screen.getByText('黒松の手入れ')).toBeInTheDocument()
  expect(screen.getByText('五葉松の植替え')).toBeInTheDocument()
})

it('エラー時にエラーメッセージが表示される', async () => {
  // fetch がエラーを返すようにモック
  ;(global.fetch as ReturnType<typeof vi.fn>).mockRejectedValueOnce(
    new Error('ネットワークエラー')
  )

  renderWithQueryClient(<PostList />)

  await waitFor(() => {
    expect(screen.getByText(/エラーが発生しました/)).toBeInTheDocument()
  })

  it('空の投稿一覧で空状態メッセージが表示される', async () => {
    ;(global.fetch as ReturnType<typeof vi.fn>).mockResolvedValueOnce({
      ok: true,
      json: () => Promise.resolve({ posts: [], nextCursor: null }),
    })

    renderWithQueryClient(<PostList />)

    await waitFor(() => {
      expect(screen.getByText(/投稿がありません/)).toBeInTheDocument()
    })
  })
})

```

21.11.4 useMutation のテスト

データ変更操作（いいね、投稿作成など）に使う `useMutation` のテストパターンです。

```

import { render, screen, fireEvent, waitFor } from '@testing-library/react'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { LikeButton } from '@components/post/LikeButton'

describe('LikeButton (useMutation使用)', () => {
  let queryClient: QueryClient

  beforeEach(() => {
    queryClient = new QueryClient({
      defaultOptions: {
        queries: { retry: false },
        mutations: { retry: false },
      },
    })
    vi.clearAllMocks()
  })

  it('いいねボタンクリックでAPIが呼ばれる', async () => {
    // いいねAPIのモック
    ;(global.fetch as ReturnType<typeof vi.fn>).mockResolvedValueOnce({
      ok: true,
      json: () => Promise.resolve({ liked: true, likeCount: 6 }),
    })

    render(
      <QueryClientProvider client={queryClient}>
        <LikeButton postId="post-1" initialLiked={false} initialCount={5} />
      </QueryClientProvider>
    )

    // いいねボタンをクリック
    fireEvent.click(screen.getByRole('button'))

    await waitFor(() => {
      // fetch が正しいURLとメソッドで呼ばれたか確認
      expect(global.fetch).toHaveBeenCalledWith(
        expect.stringContaining('/api/posts/post-1/like'),
        expect.objectContaining({ method: 'POST' })
      )
    })
  })

  it('Optimistic Update: クリック直後にUIが更新される', async () => {
    // APIレスポンスをわざと遅延させる
    ;(global.fetch as ReturnType<typeof vi.fn>).mockImplementation(
      () => new Promise(resolve =>
        setTimeout(() => resolve({
          ok: true,
          json: () => Promise.resolve({ liked: true, likeCount: 6 }),
        }), 1000)
      )
    )
  })
})

```

```

    )

    render(
      <QueryClientProvider client={queryClient}>
        <LikeButton postId="post-1" initialLiked={false} initialCount={5} />
      </QueryClientProvider>
    )

    fireEvent.click(screen.getByRole('button'))

    // APIレスポンスを待たずにUI（カウント）が即座に更新される
    // これがOptimistic Update（楽観的更新）
    expect(screen.getByText('6')).toBeInTheDocument()
  })
})

```

21.11.5 vitest.setup.tsx でのグローバルモック

BON-LOG の `vitest.setup.tsx` では、React Query をグローバルにモック化しています。個別テストでモック設定が不要な場合に便利です。

```

// vitest.setup.tsx (抜粋)
vi.mock('@tanstack/react-query', () => ({
  ...vi.importActual('@tanstack/react-query'), // 実際の機能を保持
  useQuery: vi.fn().mockReturnValue({
    data: undefined,
    isLoading: false,
    error: null,
    refetch: vi.fn(),
  }),
  useMutation: vi.fn().mockReturnValue({
    mutate: vi.fn(),
    mutateAsync: vi.fn(),
    isLoading: false,
    error: null,
  }),
  useQueryClient: () => ({
    invalidateQueries: vi.fn(),
    setQueryData: vi.fn(),
  }),
}))

```

注意: グローバルモックを使う場合、個別テストで `useQuery` の戻り値をカスタマイズしたいときは、テストファイル内で `(useQuery as ReturnType<typeof vi.fn>).mockReturnValue({ ... })` と上書きします。

▶ 理解度チェック(クリックで回答を確認)

21.11.6A useMutation を使ったコンポーネントのテスト

`useMutation` はデータの変更（作成・更新・削除）に使うフックです。いいねボタンのような「クリックしたらサーバーにリクエストを送る」コンポーネントのテストを見てみましょう。

#	テストすべきポイント	確認内容
1	ボタンクリック	<code>mutate()</code> が呼ばれるか
2	ローディング中	UIが変化するか
3	成功時	キャッシュが更新されるか
4	失敗時	エラーが表示されるか
5	楽観的更新	Optimistic Update が動作するか

楽観的更新（Optimistic Update）とは？ サーバーの応答を待たずに、先にUIを更新する手法。例: いいねボタンをクリック → 即座にハートが赤くなる → サーバー成功ならそのまま / 失敗なら元に戻す

```
// いいねボタンコンポーネントのテスト例
import { render, screen, fireEvent, waitFor } from '@testing-library/react'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { LikeButton } from '@components/post/LikeButton'

// テスト用のラッパーを作成する関数
function createWrapper() {
  const queryClient = new QueryClient({
    defaultOptions: {
      queries: { retry: false },
      mutations: { retry: false },
    },
  })
  return ({ children }: { children: React.ReactNode }) => (
    <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>
  )
}

// fetch をモック化
global.fetch = vi.fn()

describe('LikeButton', () => {
  beforeEach(() => {
    vi.clearAllMocks()
  })

  it('クリックするとmutateが呼ばれる', async () => {
    // APIレスポンスのモック: いいねが成功する
    ;(global.fetch as ReturnType<typeof vi.fn>).mockResolvedValueOnce({
      ok: true,
      json: () => Promise.resolve({ liked: true, likeCount: 6 }),
    })

    // Arrange: いいね数5、まだいいねしていない状態
    render(
      <LikeButton postId="post-1" initialLiked={false} initialCount={5} />,
      { wrapper: createWrapper() }
    )

    // Act: いいねボタンをクリック
    const button = screen.getByRole('button', { name: /いいね/i })
    fireEvent.click(button)

    // Assert: fetch (API呼び出し) が行われたことを確認
    await waitFor(() => {
      expect(global.fetch).toHaveBeenCalledTimes(1)
    })
  })

  it('いいね済みの場合は解除される', async () => {
    ;(global.fetch as ReturnType<typeof vi.fn>).mockResolvedValueOnce({

```



```

    ok: true,
    json: () => Promise.resolve({ liked: false, likeCount: 4 }),
  })

  // Arrange: いいね済みの状態
  render(
    <LikeButton postId="post-1" initialLiked={true} initialCount={5} />,
    { wrapper: createWrapper() }
  )

  // Act: いいねボタンをクリック（解除）
  const button = screen.getByRole('button')
  fireEvent.click(button)

  // Assert: APIが呼ばれたことを確認
  await waitFor(() => {
    expect(global.fetch).toHaveBeenCalled()
  })
})
})

```

21.11.6B renderHook を使ったカスタムフックのテスト

React Query のカスタムフック（`useQuery` / `useMutation` をラップしたもの）を直接テストする場合は、`renderHook` を使います。

テスト種別	メソッド	検証対象
通常のテスト	<code>render(<Component />)</code>	DOM要素を検証
フックのテスト	<code>renderHook(() => useMyHook())</code>	戻り値を検証

なぜ renderHook が必要？ React のフックは関数コンポーネント内でしか使えないため、直接呼び出すとエラーになります。`renderHook` は内部で仮のコンポーネントを作り、その中でフックを実行してくれます。

BON-LOG の `useToast` フックのテストを例に見てみましょう。

```
// __tests__/hooks/use-toast.test.ts
/**
 * @vitest-environment jsdom
 */
import { renderHook, act, waitFor } from '@testing-library/react'
import { useToast } from '@hooks/use-toast'

describe('useToast', () => {
  // タイマーモック: トーストの自動消去タイマーを制御するため
  beforeEach(() => {
    vi.useFakeTimers() // setTimeoutやsetIntervalを偽物に置き換え
  })
  afterEach(() => {
    vi.useRealTimers() // テスト後に本物のタイマーに戻す
  })

  it('初期状態ではtoasts配列を持つ', () => {
    // renderHook: フックを仮のコンポーネント内で実行
    const { result } = renderHook(() => useToast())

    // result.current: フックの現在の戻り値
    expect(Array.isArray(result.current.toasts)).toBe(true)
  })

  it('toast関数でトーストを追加できる', async () => {
    const { result } = renderHook(() => useToast())

    // act: Reactの状態更新を包む (状態変更はactの中で行う)
    act(() => {
      result.current.toast({ title: 'テスト通知' })
    })

    // waitFor: 非同期の状態更新を待つ
    await waitFor(() => {
      expect(
        result.current.toasts.some(t => t.title === 'テスト通知')
      ).toBe(true)
    })
  })

  it('トーストにvariantを設定できる', async () => {
    const { result } = renderHook(() => useToast())

    act(() => {
      // variant: 'destructive' はエラー表示用の赤いトースト
      result.current.toast({ title: 'エラー', variant: 'destructive' })
    })

    await waitFor(() => {
      expect(
        result.current.toasts.some(t => t.variant === 'destructive')
      )
    })
  })
})
```

```
    ).toBe(true)
  })
})

it('複数のトーストを追加できる', async () => {
  const { result } = renderHook(() => useToast())
  const initialLength = result.current.toasts.length

  act(() => {
    result.current.toast({ title: 'Toast A' })
    result.current.toast({ title: 'Toast B' })
  })

  await waitFor(() => {
    expect(result.current.toasts.length).toBeGreaterThanOrEqual(
      initialLength + 2
    )
  })
})
})
```

21.11.6 グローバルモックとの併用パターン

BON-LOG の `vitest.setup.tsx` では React Query がグローバルにモック化されています。これにより、ほとんどのコンポーネントテストで React Query を意識せずにテストできます。

```
// vitest.setup.tsx での React Query グローバルモック
vi.mock('@tanstack/react-query', () => ({
  // 実際のモジュールの機能を保持しつつ、一部をモック化
  ...vi.importActual('@tanstack/react-query'),

  // useQuery: データ取得フックのモック
  useQuery: vi.fn().mockReturnValue({
    data: undefined,      // 取得したデータ（デフォルトは未取得）
    isLoading: false,    // ローディング中かどうか
    error: null,          // エラー情報
    refetch: vi.fn(),    // 手動での再取得関数
  }),

  // useMutation: データ変更フックのモック
  useMutation: vi.fn().mockReturnValue({
    mutate: vi.fn(),      // 非同期で変更を実行
    mutateAsync: vi.fn(), // Promise を返す版
    isLoading: false,    // 変更処理中かどうか
    error: null,          // エラー情報
  }),

  // useQueryClient: キャッシュ操作フックのモック
  useQueryClient: () => ({
    invalidateQueries: vi.fn(), // キャッシュ無効化
    setQueryData: vi.fn(),     // キャッシュの直接更新
  }),
}))
```

特定のテストで戻り値をカスタマイズしたい場合:

```

import { useQuery } from '@tanstack/react-query'

// テスト内でグローバルモックの戻り値を上書き
it('データ取得成功時にリストが表示される', () => {
  // このテストでだけ useQuery が投稿データを返すように設定
  ;(useQuery as ReturnType<typeof vi.fn>).mockReturnValue({
    data: {
      posts: [
        { id: 'post-1', content: '最初の投稿' },
        { id: 'post-2', content: '2番目の投稿' },
      ],
    },
    isLoading: false,
    error: null,
    refetch: vi.fn(),
  })

  render(<PostList />)

  // 投稿が表示されることを確認
  expect(screen.getByText('最初の投稿')).toBeInTheDocument()
  expect(screen.getByText('2番目の投稿')).toBeInTheDocument()
})

it('ローディング中はスケルトンが表示される', () => {
  ;(useQuery as ReturnType<typeof vi.fn>).mockReturnValue({
    data: undefined,
    isLoading: true,    // ← ローディング中に変更
    error: null,
    refetch: vi.fn(),
  })

  render(<PostList />)

  // スケルトン（ローディングアニメーション）が表示されることを確認
  expect(screen.getByTestId('loading-skeleton')).toBeInTheDocument()
})

it('エラー時にエラーメッセージが表示される', () => {
  ;(useQuery as ReturnType<typeof vi.fn>).mockReturnValue({
    data: undefined,
    isLoading: false,
    error: new Error('データの取得に失敗しました'), // ← エラー
    refetch: vi.fn(),
  })

  render(<PostList />)

  expect(screen.getByText(/データの取得に失敗しました/)).toBeInTheDocument()
})

```

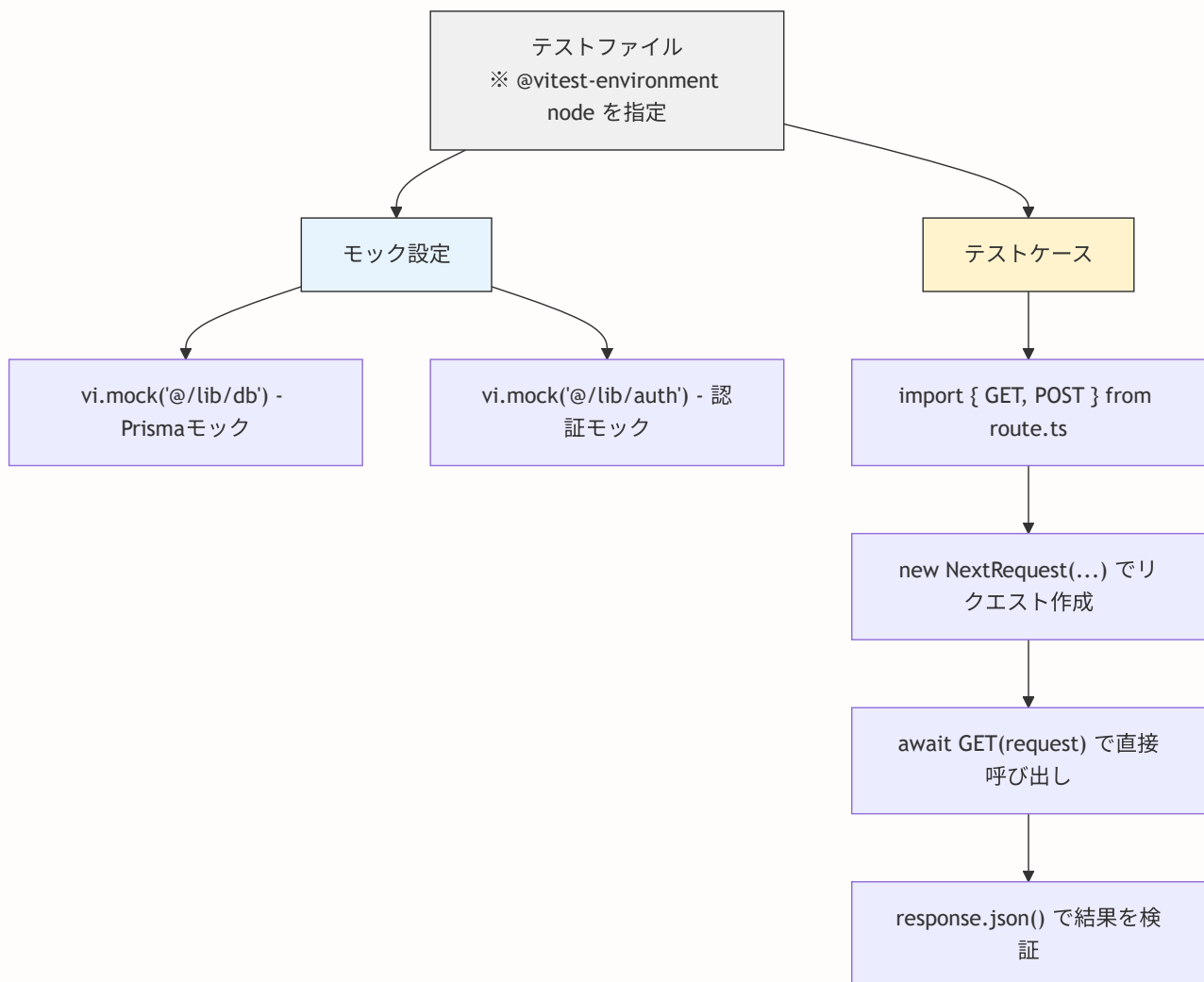
▶ 理解度チェック(クリックで回答を確認)

21.12 Server Action・APIルートテスト

▶ このセクションで学ぶこと(クリックで展開)

21.12.1 APIルートテストの基本構造

Next.js の Route Handler (`app/api/*/route.ts`) は、 `NextRequest` を受け取り `NextResponse` を返す関数です。テストでは実際のHTTPリクエストを飛ばさず、関数を直接呼び出します。



21.12.2 @vitest-environment node

APIルートやServer Actionsはサーバー側で動作するため、テスト環境を `node` に設定する必要があります。ファイルの先頭にコメントで指定します。

```
/**
 * @vitest-environment node
 */
```

この指定がない場合、デフォルトの `jsdom` 環境で実行され、`Request` や `Response` などの Web API が正しく動作しない場合があります。

21.12.3 APIルートテストの実例

BON-LOG の `/api/health` エンドポイントのテストを見てみましょう。実際のプロジェクトの `__tests__/api/health/route.test.ts` に基づいています。

```

/**
 * @vitest-environment node
 */

import { createMockPrismaClient } from '../../utils/test-utils'

// Prisma クライアントをモック化
const mockPrisma = createMockPrismaClient()
vi.mock('@/lib/db', () => ({
  prisma: mockPrisma,
}))

describe('GET /api/health', () => {
  beforeEach(() => {
    vi.clearAllMocks()
  })

  it('正常時に healthy ステータスを返す', async () => {
    // DB接続チェック用のクエリが成功するようにモック
    mockPrisma.$queryRaw.mockResolvedValueOnce([ { '?column?': 1 } ])

    // Route Handler を動的インポート（モック適用後に読み込むため）
    const { GET } = await import('@/app/api/health/route')
    const response = await GET()
    const data = await response.json()

    expect(response.status).toBe(200)
    expect(data.status).toBe('healthy')
    expect(data.database).toBe('connected')
  })

  it('データベースエラー時に unhealthy ステータスを返す', async () => {
    mockPrisma.$queryRaw.mockRejectedValueOnce(new Error('Connection failed'))

    const { GET } = await import('@/app/api/health/route')
    const response = await GET()
    const data = await response.json()

    expect(response.status).toBe(503)
    expect(data.status).toBe('unhealthy')
    expect(data.database).toBe('disconnected')
    expect(data.error).toBe('Connection failed')
  })
})

```


ポイント: `await import()` で動的インポートしているのは、`vi.mock()` でモックを設定した後にルートハンドラを読み込むためです。トップレベルの `import` を使うと、モック適用前にモジュールが読み込まれてしまいます。

21.12.4 認証付きAPIルートのテスト

認証が必要なエンドポイントのテスト例です。

```

/**
 * @vitest-environment node
 */

import { createMockPrismaClient, mockUser } from '../../utils/test-utils'
import { NextRequest } from 'next/server'

// モックの設定
const mockPrisma = createMockPrismaClient()
vi.mock('@/lib/db', () => ({ prisma: mockPrisma }))

const mockAuth = vi.fn()
vi.mock('@/lib/auth', () => ({ auth: mockAuth }))

describe('POST /api/posts', () => {
  beforeEach(() => {
    vi.clearAllMocks()
    // デフォルトで認証済み状態をセット
    mockAuth.mockResolvedValue({
      user: { id: mockUser.id, email: mockUser.email },
    })
  })

  it('認証済みユーザーが投稿を作成できる', async () => {
    const newPost = {
      id: 'new-post-1',
      userId: mockUser.id,
      content: 'テスト投稿',
      createdAt: new Date(),
    }
    mockPrisma.post.create.mockResolvedValueOnce(newPost)
    // 投稿制限チェック用のカウント
    mockPrisma.post.count.mockResolvedValueOnce(0)

    const { POST } = await import('@/app/api/posts/route')
    const request = new NextRequest('http://localhost:3000/api/posts', {
      method: 'POST',
      body: JSON.stringify({
        content: 'テスト投稿',
        genreIds: ['genre-1'],
      }),
      headers: { 'Content-Type': 'application/json' },
    })

    const response = await POST(request)
    const data = await response.json()

    expect(response.status).toBe(201)
    expect(data.id).toBe('new-post-1')
  })
})

```

```

it('未認証時は 401 エラーを返す', async () => {
  // auth() が null を返す (未認証状態)
  mockAuth.mockResolvedValueOnce(null)

  const { POST } = await import('@app/api/posts/route')
  const request = new NextRequest('http://localhost:3000/api/posts', {
    method: 'POST',
    body: JSON.stringify({ content: 'テスト' }),
    headers: { 'Content-Type': 'application/json' },
  })

  const response = await POST(request)

  expect(response.status).toBe(401)
})

it('バリデーションエラー時は 400 エラーを返す', async () => {
  const { POST } = await import('@app/api/posts/route')
  const request = new NextRequest('http://localhost:3000/api/posts', {
    method: 'POST',
    body: JSON.stringify({
      content: '', // 空のコンテンツ (バリデーション違反)
      genreIds: [], // 空のジャンル (バリデーション違反)
    }),
    headers: { 'Content-Type': 'application/json' },
  })

  const response = await POST(request)

  expect(response.status).toBe(400)
})

it('サーバーエラー時は 500 エラーを返す', async () => {
  mockPrisma.post.create.mockRejectedValueOnce(new Error('DB Error'))
  mockPrisma.post.count.mockResolvedValueOnce(0)

  const { POST } = await import('@app/api/posts/route')
  const request = new NextRequest('http://localhost:3000/api/posts', {
    method: 'POST',
    body: JSON.stringify({
      content: 'テスト投稿',
      genreIds: ['genre-1'],
    }),
    headers: { 'Content-Type': 'application/json' },
  })

  const response = await POST(request)

  expect(response.status).toBe(500)
})
})

```

21.12.5 Server Actionテストの詳細パターン

Server Actions は Route Handler と違い、`NextRequest` を使わず直接関数を呼び出します。認証チェック、バリデーション、DB操作の各段階をテストします。

```

/**
 * @vitest-environment node
 */

import { createMockPrismaClient, mockUser } from '../../utils/test-utils'

const mockPrisma = createMockPrismaClient()
vi.mock('@/lib/db', () => ({ prisma: mockPrisma }))

const mockAuth = vi.fn()
vi.mock('@/lib/auth', () => ({ auth: mockAuth }))

// next/cache のモック (revalidatePath / revalidateTag)
vi.mock('next/cache', () => ({
  revalidatePath: vi.fn(),
  revalidateTag: vi.fn(),
}))

describe('投稿関連 Server Actions', () => {
  beforeEach(() => {
    vi.clearAllMocks()
    mockAuth.mockResolvedValue({
      user: { id: mockUser.id },
    })
  })

  describe('toggleLike (いいね切り替え)', () => {
    it('まだいいねしていない場合、いいねを追加する', async () => {
      // いいねがまだ存在しない
      mockPrisma.like.findFirst.mockResolvedValueOnce(null)
      // いいね作成成功
      mockPrisma.like.create.mockResolvedValueOnce({
        id: 'like-1',
        userId: mockUser.id,
        postId: 'post-1',
      })

      const { toggleLike } = await import('@/lib/actions/like')
      const result = await toggleLike('post-1')

      expect(result.liked).toBe(true)
      expect(mockPrisma.like.create).toHaveBeenCalledWith({
        data: expect.objectContaining({
          userId: mockUser.id,
          postId: 'post-1',
        }),
      })
    })

    it('既にいいね済みの場合、いいねを削除する', async () => {
      // 既存のいいねを返す

```

```

mockPrisma.like.findFirst.mockResolvedValueOnce({
  id: 'like-1',
  userId: mockUser.id,
  postId: 'post-1',
})
mockPrisma.like.delete.mockResolvedValueOnce({})

const { toggleLike } = await import('@/lib/actions/like')
const result = await toggleLike('post-1')

expect(result.liked).toBe(false)
expect(mockPrisma.like.delete).toHaveBeenCalled()
})

it('投稿制限を超えた場合はエラーを返す', async () => {
  // 1日の投稿数が上限に達している
  mockPrisma.post.count.mockResolvedValueOnce(20)

  const { createPost } = await import('@/lib/actions/post')
  const result = await createPost({
    content: '上限テスト',
    genreIds: ['genre-1'],
  })

  expect(result.error).toContain('上限')
  // DB操作が行われないことを確認
  expect(mockPrisma.post.create).not.toHaveBeenCalled()
})
})
})

```

21.12.6 FormData を使った Server Action のテスト

Server Actions の中にはフォームデータを受け取るものがあります。テストでは **FormData** オブジェクトを手動で作成します。

```
// テストユーティリティの createMockFormData を使用
import { createMockFormData } from '../../utils/test-utils'

it('フォームデータで投稿を作成できる', async () => {
  mockPrisma.post.count.mockResolvedValueOnce(0)
  mockPrisma.post.create.mockResolvedValueOnce({ id: 'post-1' })

  const { createPostFromForm } = await import('@/lib/actions/post')
  const formData = createMockFormData({
    content: '盆栽の水やり完了',
    genreIds: 'genre-1',
  })

  const result = await createPostFromForm(formData)

  expect(result.success).toBe(true)
})
```

ポイント: `createMockFormData` は `__tests__/utils/test-utils.tsx` に定義されているヘルパー関数です。オブジェクトから FormData を簡単に作成できます。

21.12.7 実践: 投稿アクション (post.test.ts) の完全解説

BON-LOG の実際のテストファイル `__tests__/lib/actions/post.test.ts` の構造を完全に解説します。このファイルは Server Actions のテストパターンの宝庫です。

post.test.ts のテスト構造:

モックのセットアップ:

モック対象	用途
Prisma	DB操作
auth	認証
next/cache	キャッシュ再検証
rate-limit	レート制限
storage	ファイルアップロード
premium	会員制限
sanitize	入力サニタイズ
hashtag / mention	ハッシュタグ・メンション

createPost テスト:

テストケース	期待結果
認証なし	エラー
空の投稿	エラー
500文字超	エラー
1日の上限超	エラー
正常な投稿	成功
アカウント停止中	エラー
レート制限超	エラー
ジャンル4つ以上	エラー
画像5枚以上	エラー
動画2本以上	エラー
メディアのみ投稿	成功
DB エラー	エラーメッセージ

deletePost テスト:

テストケース	期待結果
認証なし	エラー
存在しない投稿	エラー
他人の投稿	権限エラー
自分の投稿	削除成功
DB エラー	エラーメッセージ

getPost / getPosts / その他テスト:

テストケース	期待結果
存在する投稿の取得	成功
存在しない投稿	エラー
ページネーション	正常動作
ブロック/ミュートユーザーの除外	正常動作

モックのセットアップ部分を詳しく見てみましょう。

```

/**
 * @vitest-environment node
 *
 * なぜ node 環境？
 * Server Actions はサーバーサイドで実行されるため、
 * ブラウザ環境（jsdom）ではなく Node.js 環境が必要です。
 */

// テストユーティリティからモックヘルパーとモックデータをインポート
import { createMockPrismaClient, mockUser, mockPost } from '../utils/test-utils'

// —— Prisma モック ——
// createMockPrismaClient() は全テーブルの CRUD メソッドを
// vi.fn() でモック化したオブジェクトを返す
const mockPrisma = createMockPrismaClient()
vi.mock('@lib/db', () => ({
  prisma: mockPrisma,
}))

// —— 認証モック ——
// auth() 関数自体をモック化して、テストごとに戻り値を変えられるように
const mockAuth = vi.fn()
vi.mock('@lib/auth', () => ({
  auth: () => mockAuth(), // 注意: auth: mockAuth ではなく関数でラップ
}))

// —— next/cache モック ——
// revalidatePath: 投稿作成後にフィードページのキャッシュを無効化する関数
// テストでは実際のキャッシュ操作は不要なのでモック化
vi.mock('next/cache', () => ({
  revalidatePath: vi.fn(),
}))

// —— レート制限モック ——
// スпам対策の投稿制限をモック化
const mockCheckUserRateLimit = vi.fn().mockResolvedValue({ success: true })
const mockCheckDailyLimit = vi.fn().mockResolvedValue({
  allowed: true, count: 0, limit: 50
})
vi.mock('@lib/rate-limit', () => ({
  checkUserRateLimit: (...args: unknown[]) => mockCheckUserRateLimit(...args),
  checkDailyLimit: (...args: unknown[]) => mockCheckDailyLimit(...args),
}))

// —— ストレージモック ——
// 画像/動画のアップロードをモック化
const mockUploadFile = vi.fn().mockResolvedValue({
  success: true,
  url: 'https://example.com/file.jpg',
})
vi.mock('@lib/storage', () => ({

```

```
    uploadFile: ( ... args: unknown[]) => mockUploadFile( ... args),  
  })  
  
  // —— 入力サニタイズモック ——  
  // XSS対策のHTMLサニタイズ。テストでは入力をそのまま返す  
  vi.mock('@lib/sanitize', () => ({  
    sanitizePostContent: (content: string) => content,  
  }))
```

各テストケースの解説:

```

describe('Post Actions', () => {
  beforeEach(() => {
    // 各テスト前にすべてのモックの呼び出し記録をクリア
    vi.clearAllMocks()
    // デフォルトの認証状態: ログイン済み
    mockAuth.mockResolvedValue({
      user: { id: mockUser.id },
    })
    // デフォルトの停止状態: アカунトはアクティブ
    mockPrisma.user.findUnique.mockResolvedValue({ isSuspended: false })
  })

  describe('createPost', () => {
    // === 認証テスト ===
    it('認証なしの場合はエラーを返す', async () => {
      // mockAuth が null を返す → 未ログイン状態をシミュレート
      mockAuth.mockResolvedValue(null)

      // 動的インポート: モック設定後にモジュールを読み込む
      const { createPost } = await import('@lib/actions/post')
      const formData = new FormData()
      formData.append('content', 'テスト投稿')

      const result = await createPost(formData)

      // 未認証エラーが返されることを検証
      expect(result).toEqual({ error: '認証が必要です' })
    })

    // === アカウント停止テスト ===
    it('アカウント停止中はエラーを返す', async () => {
      // ユーザーのisSuspendedがtrueの状態
      mockPrisma.user.findUnique.mockResolvedValue({ isSuspended: true })

      const { createPost } = await import('@lib/actions/post')
      const formData = new FormData()
      formData.append('content', 'テスト投稿')

      const result = await createPost(formData)

      expect(result).toEqual({ error: 'アカウントが停止されています' })
    })

    // === バリデーションテスト ===
    it('500文字を超える投稿はエラーを返す', async () => {
      const { createPost } = await import('@lib/actions/post')
      const formData = new FormData()
      // 'a'.repeat(501) で501文字の文字列を生成
      formData.append('content', 'a'.repeat(501))

      const result = await createPost(formData)
    })
  })
})

```

```

    expect(result).toEqual({ error: '投稿は500文字以内で入力してください' })
  })

  // === レート制限テスト ===
  it('レート制限に達した場合はエラーを返す', async () => {
    // レート制限チェックが失敗を返すように設定
    mockCheckUserRateLimit.mockResolvedValueOnce({ success: false })

    const { createPost } = await import('@lib/actions/post')
    const formData = new FormData()
    formData.append('content', 'テスト投稿')

    const result = await createPost(formData)

    expect(result).toEqual({
      error: '投稿が多すぎます。しばらく待ってから再試行してください',
    })
  })

  // === メディア制限テスト ===
  it('画像が上限を超える場合はエラーを返す', async () => {
    const { createPost } = await import('@lib/actions/post')
    const formData = new FormData()
    formData.append('content', 'テスト投稿')
    // 5枚の画像を追加（上限は4枚）
    for (let i = 0; i < 5; i++) {
      formData.append('mediaUrls', `https://example.com/image${i}.jpg`)
      formData.append('mediaTypes', 'image')
    }

    const result = await createPost(formData)

    expect(result).toEqual({ error: '画像は4枚までです' })
  })

  // === 正常系テスト ===
  it('正常な投稿を作成できる', async () => {
    // DB操作のモック：投稿数0件、作成成功
    mockPrisma.post.count.mockResolvedValue(0)
    mockPrisma.post.create.mockResolvedValue({
      ...mockPost,
      id: 'new-post-id',
    })

    const { createPost } = await import('@lib/actions/post')
    const formData = new FormData()
    formData.append('content', 'テスト投稿')
    formData.append('genreIds', 'genre-1')

    const result = await createPost(formData)

    // 成功レスポンスを検証

```

```

    expect(result.success).toBe(true)
    expect(result.postId).toBeDefined()
    // Prisma の create が呼ばれたことを検証
    expect(mockPrisma.post.create).toHaveBeenCalled()
  })

  // ==== DB エラーテスト ====
  it('エラー発生時はエラーメッセージを返す', async () => {
    mockPrisma.post.count.mockResolvedValue(0)
    // mockRejectedValue: 呼び出し時にエラーを投げるモック
    mockPrisma.post.create.mockRejectedValue(new Error('Database error'))

    const { createPost } = await import('@/lib/actions/post')
    const formData = new FormData()
    formData.append('content', 'テスト投稿')

    const result = await createPost(formData)

    expect(result).toEqual({ error: '投稿の作成に失敗しました' })
  })
})

```

21.12.8 認証アクション (auth.test.ts) のパターン

認証関連のテストは、セキュリティに直結するため特に重要です。BON-LOG の

`__tests__/lib/actions/auth.test.ts` から主要なパターンを紹介します。

カテゴリ	検証ポイント
ブルートフォース攻撃対策	ログイン試行回数の制限が機能するか
	失敗記録が正しく保存されるか
	ロックアウト状態が正しく判定されるか
ユーザー登録	パスワードがハッシュ化されてDBに保存されるか
	既存メールアドレスのチェックが機能するか
	入力バリデーションが正しく動作するか
パスワードリセット	存在しないメールでも成功を返すか（列挙攻撃対策）
	トークンの有効期限チェックが機能するか
	使用済みトークンが削除されるか

新規ユーザー登録のテスト例:

```
describe('registerUser', () => {
  it('新規ユーザーを登録できる', async () => {
    // 既存ユーザーが存在しない（メールアドレスが未使用）
    mockPrisma.user.findUnique.mockResolvedValueOnce(null)
    // ユーザー作成が成功する
    mockPrisma.user.create.mockResolvedValueOnce({
      id: 'new-user-id',
      email: 'newuser@example.com',
      nickname: '新規ユーザー',
    })

    const { registerUser } = await import('@/lib/actions/auth')
    const result = await registerUser({
      email: 'newuser@example.com',
      password: 'Password123',
      nickname: '新規ユーザー',
    })

    // 成功を確認
    expect(result).toEqual({ success: true, userId: 'new-user-id' })

    // パスワードがハッシュ化されてDBに保存されることを確認
    // ※ bcrypt.hash のモックが 'hashed-password' を返す設定
    expect(mockPrisma.user.create).toHaveBeenCalledWith({
      data: {
        email: 'newuser@example.com',
        password: 'hashed-password', // 生のパスワードではない！
        nickname: '新規ユーザー',
      },
    })
  })

  it('既存のメールアドレスの場合、エラーを返す', async () => {
    // 既存ユーザーが存在する
    mockPrisma.user.findUnique.mockResolvedValueOnce(mockUser)

    const { registerUser } = await import('@/lib/actions/auth')
    const result = await registerUser({
      email: 'test@example.com',
      password: 'Password123',
      nickname: 'テストユーザー',
    })

    expect(result).toEqual({ error: 'このメールアドレスは既に登録されています' })
    // user.create が呼ばれないことを確認（重複登録されない）
    expect(mockPrisma.user.create).not.toHaveBeenCalled()
  })
})
```

パスワードリセットのセキュリティテスト:

```
describe('requestPasswordReset', () => {
  it('ユーザーが存在しなくても成功を返す（セキュリティ対策）', async () => {
    // ユーザーが存在しない
    mockPrisma.user.findUnique.mockResolvedValueOnce(null)

    const { requestPasswordReset } = await import('@lib/actions/auth')
    const result = await requestPasswordReset('nonexistent@example.com')

    // 重要: エラーではなく成功を返す
    // → 攻撃者に「このメールは未登録」という情報を漏らさない
    expect(result).toEqual({ success: true })

    // トークンは作成されない（実際の処理は行われない）
    expect(mockPrisma.passwordResetToken.create).not.toHaveBeenCalled()
  })
})
```

セキュリティのポイント: パスワードリセットで「ユーザーが見つかりません」というエラーを返すと、攻撃者は「このメールアドレスは登録されていない」という情報を得られます。これは**ユーザー列挙攻撃（User Enumeration Attack）**と呼ばれ、セキュリティ上の脆弱性です。常に成功を返すことで、この攻撃を防ぎます。

▶ 理解度チェック(クリックで回答を確認)

21.13 E2Eテスト詳細

▶ このセクションで学ぶこと(クリックで展開)

21.13.1 playwright.config.ts の詳細解説

BON-LOG の実際の `playwright.config.ts` を詳しく見ていきます。


```

import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  // -----
  // テストファイルの場所
  // -----
  testDir: './e2e', // e2e/ ディレクトリ内のテストファイルを対象

  // -----
  // 並列実行とリトライ
  // -----
  fullyParallel: true, // テストを並列実行（高速化）
  retries: process.env.CI ? 2 : 0, // CI環境では2回リトライ
  workers: process.env.CI ? 1 : undefined, // CI環境ではワーカー1つ（安定性）

  // -----
  // レポーター設定
  // -----
  reporter: [
    ['html', { open: 'never' }], // HTMLレポート（自動で開かない）
    ['list'], // コンソールにリスト表示
  ],

  // -----
  // 共通設定
  // -----
  use: {
    baseURL: 'http://localhost:3000', // アプリケーションのベースURL

    // テスト失敗時のデバッグ支援
    screenshot: 'only-on-failure', // 失敗時のみスクリーンショット保存
    video: 'retain-on-failure', // 失敗時のみビデオ保存
    trace: 'on-first-retry', // 初回リトライ時にトレース保存

    // タイムアウト設定
    actionTimeout: 10000, // 各操作（クリック等）のタイムアウト
    navigationTimeout: 30000, // ページ遷移のタイムアウト
  },

  // -----
  // プロジェクト（ブラウザ）設定
  // -----
  projects: [
    // 1. 認証セットアップ（他のプロジェクトの前に実行）
    {
      name: 'setup',
      testMatch: /\.*\setup\.ts/,
    },

    // 2. デスクトップ Chrome（認証済み状態で実行）
    {

```

```

    name: 'chromium',
    use: {
      ... devices['Desktop Chrome'],
      storageState: 'e2e/.auth/user.json', // 保存済みの認証状態
    },
    dependencies: ['setup'], // setup 完了後に実行
    testIgnore: [/.*\.setup\.ts/, /auth\.spec\.ts/],
  },

// 3. デスクトップ Firefox
{
  name: 'firefox',
  use: {
    ... devices['Desktop Firefox'],
    storageState: 'e2e/.auth/user.json',
  },
  dependencies: ['setup'],
  testIgnore: /.*\.setup\.ts/,
},

// 4. モバイル Chrome (レスポンステスト)
{
  name: 'Mobile Chrome',
  use: {
    ... devices['Pixel 5'],
    storageState: 'e2e/.auth/user.json',
  },
  dependencies: ['setup'],
  testIgnore: /.*\.setup\.ts/,
},

// 5. モバイル Safari
{
  name: 'Mobile Safari',
  use: {
    ... devices['iPhone 12'],
    storageState: 'e2e/.auth/user.json',
  },
  dependencies: ['setup'],
  testIgnore: /.*\.setup\.ts/,
},

// 6. 未認証テスト (ログイン前のテスト用)
{
  name: 'chromium-noauth',
  use: { ... devices['Desktop Chrome'] },
  testMatch: /auth\.spec\.ts/,
},
],

// _____
// 開発サーバー設定
// _____

```

```
webServer: {
  // CI: ビルド済みサーバー / ローカル: 開発サーバー
  command: process.env.CI
    ? 'node .next/standalone/server.js'
    : 'npm run dev',
  url: 'http://localhost:3000',
  reuseExistingServer: !process.env.CI, // ローカルでは既存サーバー再利用
  timeout: 120000,                       // サーバー起動待ちタイムアウト
},

// -----
// グローバルタイムアウト
// -----
timeout: 30000, // 各テストケースのタイムアウト
expect: {
  timeout: 5000, // expect のタイムアウト
},
})
```

21.13.2 認証セットアップの実装

BON-LOG の `e2e/auth.setup.ts` は、テスト実行前にログイン処理を行い、認証状態を保存します。

```
import { test as setup, expect } from '@playwright/test'

const AUTH_FILE = 'e2e/.auth/user.json'

/**
 * 認証セットアップ
 * シードで作成済みのE2Eテスト用ユーザーでログインし、storageState を保存
 */
setup('authenticate', async ({ page }) => {
  const email = 'e2e-test@example.com'
  const password = 'TestPassword123!'

  // ログインページへ遷移
  await page.goto('/login')

  // フォーム入力 (getByLabel でアクセシブルなセレクトを使用)
  await page.getByLabel(/メールアドレス/i).fill(email)
  await page.locator('#password').fill(password)

  // ログインボタンをクリック
  await page.getByRole('button', { name: /ログイン/i }).click()

  // フィードページへのリダイレクトを確認
  await expect(page).toHaveURL(/\/feed/, { timeout: 15000 })

  // 認証状態を JSON ファイルに保存
  // Cookie、localStorage、sessionStorage がすべて保存される
  await page.context().storageState({ path: AUTH_FILE })
})
```

21.13.3 ページオブジェクトパターン

テストコードの再利用性と可読性を高めるために、**ページオブジェクトパターン**を使います。各ページの操作をクラスにまとめます。

```
// e2e/pages/feed-page.ts
import { Page, Locator, expect } from '@playwright/test'

/**
 * フィードページのページオブジェクト
 * フィードページの操作とアサーションをカプセル化
 */
export class FeedPage {
  readonly page: Page
  readonly postForm: Locator
  readonly postContent: Locator
  readonly submitButton: Locator
  readonly postCards: Locator

  constructor(page: Page) {
    this.page = page
    this.postForm = page.locator('[data-testid="post-form"]')
    this.postContent = page.locator('textarea[name="content"]')
    this.submitButton = page.getByRole('button', { name: /投稿/i })
    this.postCards = page.locator('[data-testid="post-card"]')
  }

  /** フィードページに遷移 */
  async goto() {
    await this.page.goto('/feed')
    await expect(this.postForm).toBeVisible()
  }

  /** 新しい投稿を作成 */
  async createPost(content: string) {
    await this.postContent.fill(content)
    await this.submitButton.click()
    // 投稿がタイムラインに表示されるまで待つ
    await expect(this.page.locator(`text=${content}`)).toBeVisible()
  }

  /** 指定番目の投稿にいいね */
  async likePost(index: number = 0) {
    const likeButton = this.postCards
      .nth(index)
      .locator('[data-testid="like-button"]')
    await likeButton.click()
  }

  /** 投稿が表示されていることを確認 */
  async expectPostVisible(content: string) {
    await expect(this.page.locator(`text=${content}`)).toBeVisible()
  }
}
```

ページオブジェクトを使ったテスト:

```
// e2e/feed.spec.ts
import { test, expect } from '@playwright/test'
import { FeedPage } from '../pages/feed-page'

test.describe('フィード機能', () => {
  test('新しい投稿を作成できる', async ({ page }) => {
    const feedPage = new FeedPage(page)
    await feedPage.goto()
    await feedPage.createPost('E2Eテストの投稿です')
    await feedPage.expectPostVisible('E2Eテストの投稿です')
  })

  test('投稿にいいねできる', async ({ page }) => {
    const feedPage = new FeedPage(page)
    await feedPage.goto()
    await feedPage.likePost(0)
    // いいね後のUIの変化を確認
  })
})
```

21.13.4 CI環境でのE2Eテスト

GitHub Actions での E2E テスト設定です。E2Eテストは `needs: [lint, test, build]` により、他の3ジョブが全て成功してから実行されます。

```

# .github/workflows/ci.yml (E2E部分の抜粋)
e2e:
  name: E2E Tests
  runs-on: ubuntu-latest
  needs: [lint, test, build]          # 前の3ジョブが成功してから実行
  timeout-minutes: 15

  services:
    postgres:
      image: postgres:16-alpine
      env:
        POSTGRES_USER: postgres
        POSTGRES_PASSWORD: postgres
        POSTGRES_DB: bonsai_sns_test  # テスト専用のDB名
      ports:
        - 5432:5432
      options: >-
        --health-cmd pg_isready
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Generate Prisma Client
      run: npx prisma generate

    # DBスキーマを適用してシードデータを投入
    - name: Setup database
      run: npx prisma db push
      env:
        DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
        DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"

    - name: Seed test data
      run: npx prisma db seed
      env:
        DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
        DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"

```

```

# Playwright のブラウザをインストール (chromiumのみで高速化)
- name: Install Playwright browsers
  run: npx playwright install --with-deps chromium

# アプリケーションをビルド
- name: Build application
  run: npm run build
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    NEXTAUTH_SECRET: "ci-e2e-test-secret-key"
    NEXTAUTH_URL: "http://localhost:3000"
    NEXT_PUBLIC_APP_URL: "http://localhost:3000"
    AUTH_TRUST_HOST: "true"
    NODE_OPTIONS: "--max-old-space-size=4096"

# standaloneサーバーの準備
- name: Prepare standalone server
  run: |
    cp -r public .next/standalone/public 2>/dev/null || true
    cp -r .next/static .next/standalone/.next/static 2>/dev/null || true
    cp -r node_modules/.prisma .next/standalone/node_modules/.prisma 2>/dev/null || true
    cp -r node_modules/@prisma .next/standalone/node_modules/@prisma 2>/dev/null || true

# サーバーを起動してヘルスチェック
- name: Start server and check health
  run: |
    node .next/standalone/server.js &
    SERVER_PID=$!
    for i in $(seq 1 30); do
      if curl -s http://localhost:3000/login > /dev/null 2>&1; then
        echo "Server is ready"
        break
      fi
      sleep 2
    done
    kill $SERVER_PID 2>/dev/null || true
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    NEXTAUTH_SECRET: "ci-e2e-test-secret-key"
    NEXTAUTH_URL: "http://localhost:3000"
    AUTH_TRUST_HOST: "true"
    PORT: "3000"
    HOSTNAME: "0.0.0.0"
    NODE_ENV: "production"

# E2Eテスト実行
- name: Run E2E tests
  run: npx playwright test --project=chromium --project=chromium-noauth
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"

```



```
NEXTAUTH_SECRET: "ci-e2e-test-secret-key"
NEXTAUTH_URL: "http://localhost:3000"
AUTH_TRUST_HOST: "true"
PORT: "3000"
HOSTNAME: "0.0.0.0"
NODE_ENV: "production"
timeout-minutes: 15
```

```
# レポートのアップロード（失敗時も実行）
- name: Upload Playwright report
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: playwright-report
    path: playwright-report/
    retention-days: 7
```

21.13.5 E2Eテストのベストプラクティス

プラクティス	説明
data-testid を使う	CSSクラスやテキストよりも安定したセレクタ
ページオブジェクトパターン	テストコードの再利用性と可読性を向上
認証状態の再利用	storageState で毎回のログイン処理を省略
CI では1ブラウザに絞る	速度と安定性のバランス（Chromium のみなど）
失敗時のデバッグ情報	screenshot, video, trace を保存
テストデータの独立性	各テストが独自のデータを作成し、他に依存しない

▶ 理解度チェック(クリックで回答を確認)

21.13.6 E2Eテストファイルの全解説

BON-LOG の `e2e/` ディレクトリに存在する全テストファイルを詳しく解説します。

e2e/ ディレクトリの構成:

```
e2e/
├─ .auth/                # 認証状態の保存先
│   └─ user.json         # ログイン後のCookie/セッション情報
├─ auth.setup.ts         # 認証セットアップ（テスト前のログイン処理）
├─ auth.spec.ts          # 認証機能のテスト（未ログイン状態）
├─ feed.spec.ts          # フィード機能のテスト（ログイン状態）
├─ search.spec.ts        # 検索機能のテスト（ログイン状態）
└─ user-profile.spec.ts  # ユーザープロフィール・設定のテスト
```

auth.spec.ts の詳細

認証テストは**未認証状態**で実行されます。`playwright.config.ts` で `chromium-noauth` プロジェクトに設定されているためです。

```

import { test, expect } from '@playwright/test'

test.describe('認証機能', () => {
  // — ログインページのテスト —
  test.describe('ログインページ', () => {
    test('ログインページが表示される', async ({ page }) => {
      await page.goto('/login')

      // WAI-ARIAロールで見出しを検索（アクセシビリティテスト兼用）
      await expect(
        page.getByRole('heading', { name: /ログイン/i })
      ).toBeVisible()

      // ラベルで入力欄を検索（スクリーンリーダー対応の確認）
      await expect(page.getByLabel(/メールアドレス/i)).toBeVisible()
      await expect(page.locator('#password')).toBeVisible()
      await expect(
        page.getByRole('button', { name: /ログイン/i })
      ).toBeVisible()
    })

    test('空のフォームでログインするとバリデーションが発動する', async ({ page }) => {
      await page.goto('/login')

      // 空の状態でログインボタンをクリック
      await page.getByRole('button', { name: /ログイン/i }).click()

      // HTML5バリデーション: required属性により最初の未入力フィールドにフォーカス
      await expect(page.getByLabel(/メールアドレス/i)).toBeFocused()
    })

    test('無効な認証情報でログインするとエラーが表示される', async ({ page }) => {
      await page.goto('/login')

      // 存在しないユーザーの認証情報を入力
      await page.getByLabel(/メールアドレス/i).fill('invalid@example.com')
      await page.locator('#password').fill('wrongpassword')
      await page.getByRole('button', { name: /ログイン/i }).click()

      // エラーメッセージの表示を待機（サーバーへのリクエストがあるため長めに）
      await expect(
        page.getByText(/メールアドレスまたはパスワードが間違っています/i)
      ).toBeVisible({ timeout: 10000 })
    })

    test('新規登録ページへのリンクが動作する', async ({ page }) => {
      await page.goto('/login')
      await page.getByRole('link', { name: /新規登録/i }).click()
      await expect(page).toHaveURL('/register')
    })
  })
})

```

```
// — 新規登録ページのテスト —
test.describe('新規登録ページ', () => {
  test('新規登録ページが表示される', async ({ page }) => {
    await page.goto('/register')

    await expect(
      page.getByRole('heading', { name: /新規登録/i })
    ).toBeVisible()
    await expect(page.getByLabel(/ニックネーム/i)).toBeVisible()
    await expect(page.getByLabel(/メールアドレス/i)).toBeVisible()
    // /^パスワード$/i: 完全一致で検索（「パスワード確認」と区別）
    await expect(page.getByLabel(/パスワード$/i)).toBeVisible()
  })
})

// — 認証ガードのテスト —
test.describe('認証が必要なページへのアクセス', () => {
  test('未ログインでフィードにアクセス → ログインページにリダイレクト', async ({ page }) => {
    await page.goto('/feed')
    // 正規表現: callbackUrl パラメータが付与される可能性があるため
    await expect(page).toHaveURL(/\/login/)
  })

  test('未ログインで設定ページにアクセス → リダイレクト', async ({ page }) => {
    await page.goto('/settings')
    await expect(page).toHaveURL(/\/login/)
  })
})
```

search.spec.ts の詳細

検索機能のテストは**認証済み状態**で実行されます。

```

import { test, expect } from '@playwright/test'

test.describe('検索機能', () => {
  test('検索ページが表示される', async ({ page }) => {
    await page.goto('/search')
    // プレースホルダーテキストで検索バーを特定
    await expect(
      page.getByPlaceholder(/投稿やユーザーを検索/i)
    ).toBeVisible()
  })

  test('検索タブが表示される', async ({ page }) => {
    await page.goto('/search')

    // { exact: true } で「投稿」と「投稿する」ボタンを区別
    await expect(
      page.getByRole('button', { name: '投稿', exact: true })
    ).toBeVisible()
    await expect(
      page.getByRole('button', { name: 'ユーザー', exact: true })
    ).toBeVisible()
    await expect(
      page.getByRole('button', { name: 'タグ', exact: true })
    ).toBeVisible()
  })

  test('キーワードで検索できる', async ({ page }) => {
    await page.goto('/search')
    // ネットワークが安定するまで待機
    await page.waitForLoadState('networkidle')

    const searchInput = page.getByPlaceholder(/投稿やユーザーを検索/i)
    await searchInput.click()
    await searchInput.fill('盆栽')

    // 入力値が反映されたことを確認
    await expect(searchInput).toHaveValue('盆栽')

    // Enterキーで検索実行
    await searchInput.press('Enter')

    // URLにクエリパラメータが追加されることを確認
    await expect(page).toHaveURL(/q=/, { timeout: 15000 })
  })

  test('ユーザータブに切り替えられる', async ({ page }) => {
    await page.goto('/search?q=盆栽')
    await page.waitForLoadState('networkidle')

    await page.getByRole('button', { name: 'ユーザー', exact: true }).click()
  })
})

```

```
// URLに tab=users が追加されることを確認
await expect(page).toHaveURL(/tab=users/, { timeout: 10000 })
})

test('検索結果が0件の場合メッセージが表示される', async ({ page }) => {
  await page.goto('/search?q=存在しないキーワード12345')

  // 複数のメッセージパターンに対応 (正規表現のOR)
  await expect(
    page.getByText(
      /一致する投稿はありません|投稿が見つかりません|見つかりませんでした/i
    )
  ).toBeVisible({ timeout: 10000 })
})
})
```

user-profile.spec.ts の詳細

ユーザープロフィール関連のテストです。

```

import { test, expect } from '@playwright/test'

test.describe('ユーザー設定ページ', () => {
  test('設定ページが表示される', async ({ page }) => {
    await page.goto('/settings')
    await expect(page.getByText(/設定/i).first()).toBeVisible({
      timeout: 10000,
    })
  })
})

test('プロフィール編集ページに遷移できる', async ({ page }) => {
  await page.goto('/settings/profile')
  await expect(page).toHaveURL(/\/settings\/profile/, { timeout: 10000 })
  // プロフィール編集フォームが表示されることを確認
  await expect(page.getByText(/ニックネーム/i)).toBeVisible({
    timeout: 10000,
  })
})
})

test.describe('存在しないユーザー', () => {
  test('404ページが表示される', async ({ page }) => {
    await page.goto('/users/non-existent-user-id-12345')
    // 404関連の複数のメッセージパターンに対応
    await expect(
      page.getByText(
        /見つかりませんでした|存在しません|not found|404|ページが見つかりません/i
      )
    ).toBeVisible({ timeout: 10000 })
  })
})
})

```

21.13.7 Playwright のデバッグテクニック

E2E テストが失敗したときのデバッグ方法を紹介します。

#	デバッグ手法	コマンド	用途
1	UI モードで実行	<code>npx playwright test --ui</code>	ブラウザ操作をリアルタイムで確認
2	ヘッドフルモードで実行	<code>npx playwright test --headed</code>	実際のブラウザ画面を見ながらテスト
3	特定のテストだけ実行	<code>npx playwright test auth.spec.ts</code>	失敗したテストファイルだけを実行
4	トレースビューアで確認	<code>npx playwright show-trace trace.zip</code>	各ステップのスクリーンショットとネットワークログ
5	デバグが付きで実行	<code>npx playwright test --debug</code>	ステップ実行でブレークポイントを設定

最もよく使うデバッグコマンド

UI モードで実行（最も便利）

`npx playwright test --ui`

ブラウザを表示して実行

`npx playwright test --headed`

特定のテストだけ実行

`npx playwright test auth.spec.ts`

特定のテストケースだけ実行（-g でgrepフィルタ）

`npx playwright test -g "ログインページが表示される"`

トレースを有効にして実行

`npx playwright test --trace on`

テストレポートを表示

`npx playwright show-report`

▶ 理解度チェック(クリックで回答を確認)

21.14 テスト組織構造

▶ このセクションで学ぶこと(クリックで展開)

21.14.1 tests/ ディレクトリの全体像

BON-LOG では、テストファイルを `__tests__/` ディレクトリに集約しています。ディレクトリ構造はソースコードの構造をミラーリングしています。

```

__tests__/
├─ api/                                # APIルート (Route Handler) のテスト
│  ├─ health/                          #   /api/health
│  │   └─ route.test.ts
│  ├─ upload/                          #   /api/upload/*
│  │   ├─ avatar/
│  │   ├─ header/
│  │   └─ presigned/
│  ├─ webhooks/                        #   /api/webhooks/*
│  │   └─ stripe/
│  ├─ cron/                            #   /api/cron/*
│  │   ├─ check-subscriptions/
│  │   ├─ cleanup-events/
│  │   └─ publish-scheduled/
│  ├─ admin/                           #   /api/admin/*
│  └─ maintenance/                     #   /api/maintenance/*
├─ app/                                # ページコンポーネントのテスト
│  ├─ auth/                            #   (auth)グループ: ログイン、登録
│  ├─ main/                            #   (main)グループ: メイン機能
│  │   ├─ bookmarks/                  #     ブックマーク
│  │   ├─ drafts/                     #     下書き
│  │   ├─ events/                     #     イベント
│  │   ├─ posts/                      #     投稿詳細・予約投稿
│  │   ├─ search/                     #     検索
│  │   ├─ settings/                   #     設定
│  │   ├─ shops/                       #     盆栽園
│  │   └─ users/                       #     ユーザープロフィール
│  ├─ admin/                           #   管理者ページ
│  └─ maintenance/                     #   メンテナンスページ
├─ components/                          # UIコンポーネントのテスト
│  ├─ auth/                            #   認証コンポーネント
│  ├─ post/                            #   投稿コンポーネント
│  ├─ comment/                          #   コメントコンポーネント
│  ├─ user/                             #   ユーザーコンポーネント
│  ├─ shop/                             #   盆栽園コンポーネント
│  ├─ event/                            #   イベントコンポーネント
│  ├─ feed/                             #   フィードコンポーネント
│  ├─ layout/                           #   レイアウトコンポーネント
│  ├─ common/                           #   共通コンポーネント
│  ├─ ui/                               #   shadcn/ui コンポーネント
│  ├─ notification/                     #   通知コンポーネント
│  ├─ admin/                            #   管理者コンポーネント
│  ├─ subscription/                     #   サブスクリプション
│  ├─ pwa/                              #   PWA関連
│  ├─ seo/                              #   SEO関連
│  └─ ads/                              #   広告関連

```

```

└─ lib/                                # ライブラリ・ユーティリティのテスト
  │
  │ └─ actions/                        #   Server Actions
  │   │ └─ admin/                     #     管理者用Actions
  │   │
  │   └─ validations/                 #     バリデーション
  │   │
  │   └─ services/                    #     サービス層
  │   │
  │   └─ security/                    #     セキュリティ
  │   │
  │   └─ email/                       #     メール送信
  │   │
  │   └─ search/                      #     検索機能
  │   │
  │   └─ storage/                     #     ストレージ
  │   │
  │   └─ constants/                   #     定数
  │   │
  │   └─ scraping/                    #     スクレイピング
  │
  │
  └─ hooks/                            # カスタムフックのテスト
    │ └─ use-toast.test.ts
    │
    │
  └─ coverage-boost/                  # カバレッジ向上のための追加テスト
    │ └─ branch-coverage.test.ts
    │ └─ component-branches.test.tsx
    │ └─ ...
    │
    │
  └─ utils/                           # テストユーティリティ
    │ └─ test-utils.tsx               #   共通モックデータ・ヘルパー関数
    │
    │
  └─ proxy.test.ts                    # Proxyのテスト

```

21.14.2 テストファイルの命名規則

BON-LOG では以下の命名規則を使っています。

パターン	用途	例
<code>*.test.ts</code>	ユニットテスト（関数・ロジック）	<code>auth.test.ts</code> , <code>date.test.ts</code>
<code>*.test.tsx</code>	コンポーネントテスト（JSX含む）	<code>PostCard.test.tsx</code> , <code>Button.test.tsx</code>
<code>*.spec.ts</code>	E2Eテスト（Playwright）	<code>auth.spec.ts</code> , <code>feed.spec.ts</code>
<code>*.setup.ts</code>	E2Eセットアップ（認証等）	<code>auth.setup.ts</code>
<code>*- extended.test.ts</code>	拡張テスト（追加カバレッジ）	<code>auth.extended.test.ts</code>

拡張子	実行ツール	配置場所
<code>.test.ts</code> / <code>.test.tsx</code>	Vitest	<code>__tests__</code> / ディレクトリ内（ユニット・結合テスト）
<code>.spec.ts</code>	Playwright	<code>e2e</code> / ディレクトリ内（E2E テスト）

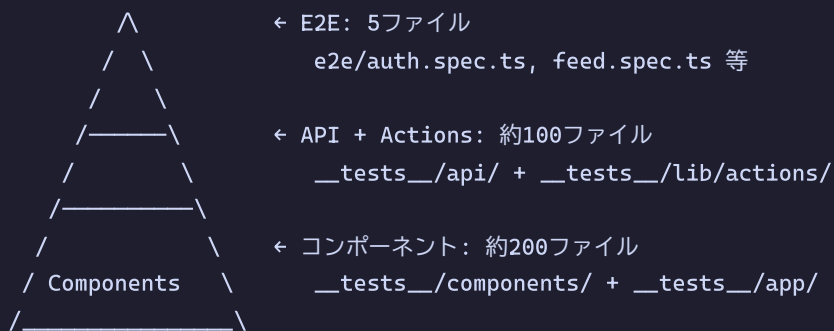
注意: `vitest.config.ts` の `testPathIgnorePatterns` で `e2e/` を除外し、混在を防止しています。

21.14.3 テストカテゴリーの分類

BON-LOG のテストは大きく4つのカテゴリに分類されます。

カテゴリ	ディレクトリ	テスト数	内容
Server Actions	<code>__tests__/lib/actions/</code>	多数	ビジネスロジックの中核テスト
コンポーネント	<code>__tests__/components/</code>	多数	UIコンポーネントのレンダリングテスト
APIルート	<code>__tests__/api/</code>	中程度	HTTP エンドポイントのテスト
E2E	<code>e2e/</code>	少数	ブラウザ操作の統合テスト

テスト数の比率（テストピラミッドに対応）：



合計：455テストスイート、8,800以上のテストケース

21.14.4 テストユーティリティ（`test-utils.tsx`）

`__tests__/utils/test-utils.tsx` は、テスト全体で共有される重要なファイルです。

カテゴリ	内容	説明
1. モックデータ定義	<code>mockUser</code>	テスト用ユーザー
	<code>mockPost</code>	テスト用投稿
	<code>mockComment</code>	テスト用コメント
	<code>mockSession</code>	テスト用セッション
	<code>mockGenres</code>	テスト用ジャンル
	<code>mockNotification</code>	テスト用通知
	<code>mockEvent</code>	テスト用イベント
	<code>mockShop</code>	テスト用盆栽園
	... その他多数	
2. カスタムレンダラー	<code>render()</code>	プロバイダー付き (SessionProvider + QueryClientProvider)
3. ヘルパー関数	<code>createMockFormData()</code>	FormData作成
	<code>waitForLoadingToFinish()</code>	待機
4. Prisma モッククライアント	<code>createMockPrismaClient()</code>	全テーブルのCRUD操作をモック化

テストファイルからの利用例:

```
// __tests__/components/post/ からインポート
import { render, mockPost, mockUser } from '../../../../utils/test-utils'

// __tests__/coverage-boost/ からインポート
import { render, mockPost } from '../../../../utils/test-utils'

// __tests__/lib/actions/ からインポート
import { createMockPrismaClient, mockUser } from '../../../../utils/test-utils'
```

注意: インポートパスは、テストファイルの位置によって異なります。`__tests__/components/xxx/` からは `../../utils/test-utils`、`__tests__/coverage-boost/` からは `../utils/test-utils` となります。

21.14.5 coverage-boost ディレクトリ

`__tests__/coverage-boost/` は、テストカバレッジを向上させるための追加テストを格納するディレクトリです。

`coverage-boost/` の目的:

通常のテスト (`__tests__/components/` など) では
カバーしきれないブランチやエッジケースを
専用のテストファイルで補完する

例:

```
└─ branch-coverage.test.ts      → if/else の未テスト分岐を網羅
└─ component-branches.test.tsx  → コンポーネントの条件分岐を網羅
└─ error-pages-branches.test.tsx → エラーページの分岐をテスト
└─ logger-coverage.test.ts      → ロガーの全パスをテスト
```

▶ 理解度チェック(クリックで回答を確認)

21.15 モックパターン

▶ このセクションで学ぶこと(クリックで展開)

21.15.1 グローバルモック (`vitest.setup.tsx`)

BON-LOG の `vitest.setup.tsx` は、**全テストファイルで共通して適用されるモック**を定義しています。これにより、個々のテストファイルでモック設定を繰り返す必要がなくなります。

vitest.setup.tsx のモック階層 (全テスト共通) :

#	カテゴリ	内容
1	ポリフィル	<code>TextEncoder</code> / <code>TextDecoder</code> (pg / Prisma が内部で使用)
2	Testing Library マッチャー	<code>import '@testing-library/jest-dom' → <code>toBeInTheDocument()</code></code> 等が使える
3	モジュールモック	<code>@/lib/db</code> (Prisma), <code>@/lib/auth</code> (NextAuth), <code>@/lib/logger</code> (ロガー), <code>next/navigation</code> (ルーター), <code>next/image</code> (画像最適化), <code>next/link</code> (リンク), <code>next-auth/react</code> (認証React), <code>@tanstack/react-query</code> (React Query)
4	ブラウザAPIモック	<code>IntersectionObserver</code> , <code>ResizeObserver</code> , <code>matchMedia</code> , <code>scrollTo</code> , <code>localStorage</code>
5	コンソール制御	特定の警告メッセージを抑制

21.15.2 Prisma モック

データベース操作をモック化して、テスト中に実際のDBへの接続を防ぎます。

```
// vitest.setup.tsx (Prismaモック部分)
vi.mock('@/lib/db', () => ({
  prisma: {
    user: {
      findUnique: vi.fn(),
      findMany: vi.fn(),
      create: vi.fn(),
      update: vi.fn(),
      delete: vi.fn(),
    },
    post: {
      findUnique: vi.fn(),
      findMany: vi.fn(),
      create: vi.fn(),
      update: vi.fn(),
      delete: vi.fn(),
    },
    // ... 各テーブル分のモック
    $transaction: vi.fn(),
  },
}))
```

テストファイルでの使い方:

```
import { prisma } from '@lib/db'

// モック関数に戻り値を設定
;(prisma.user.findUnique as ReturnType<typeof vi.fn>).mockResolvedValue({
  id: 'user-1',
  nickname: 'テストユーザー',
})

// テスト後にリセット
afterEach(() => {
  vi.clearAllMocks() // 呼び出し記録をクリア
})
```

重要: グローバルモックでは基本的な構造のみ定義し、各テストで `mockResolvedValue` を使って具体的な戻り値を設定します。

21.15.3 NextAuth モック

認証に関するモックは2種類あります。サーバーサイド用の `@lib/auth` とクライアントサイド用の `next-auth/react` です。


```
// --- サーバーサイド認証モック ---
vi.mock('@lib/auth', () => ({
  auth: vi.fn().mockResolvedValue({
    user: {
      id: 'test-user-id',
      name: 'Test User',
      email: 'test@example.com',
    },
  }),
  signIn: vi.fn(),
  signOut: vi.fn(),
  handlers: {
    GET: vi.fn(),
    POST: vi.fn(),
  },
}))

// --- クライアントサイド認証モック ---
vi.mock('next-auth/react', () => ({
  useSession: () => ({
    data: {
      user: {
        id: 'test-user-id',
        name: 'Test User',
        email: 'test@example.com',
      },
    },
    status: 'authenticated',
  }),
  signIn: vi.fn(),
  signOut: vi.fn(),
  SessionProvider: ({ children }) => children,
}))
```

テストで認証状態を変更する場合:

```
import { auth } from '@lib/auth'

// 未認証状態をシミュレート
;(auth as ReturnType<typeof vi.fn>).mockResolvedValueOnce(null)

// 特定のユーザーとしてログイン
;(auth as ReturnType<typeof vi.fn>).mockResolvedValueOnce({
  user: { id: 'admin-user', role: 'admin' },
})
```

21.15.4 Next.js モジュールのモック

next/navigation

テスト環境には Next.js のルーターが存在しないため、モック化が必須です。

```
vi.mock('next/navigation', () => ({
  useRouter: () => ({
    push: vi.fn(),      // ページ遷移
    replace: vi.fn(),   // ページ置換
    back: vi.fn(),      // 戻る
    forward: vi.fn(),   // 進む
    refresh: vi.fn(),   // 再読み込み
    prefetch: vi.fn(),  // プリフェッチ
  }),
  useSearchParams: () => new URLSearchParams(),
  usePathname: () => '/',
  redirect: vi.fn(),
  notFound: vi.fn(),
}))
```

テストでルーターの呼び出しを検証する場合:

```
import { useRouter } from 'next/navigation'

it('ログインボタンでログインページに遷移する', () => {
  const mockPush = vi.fn()
  ;(useRouter as ReturnType<typeof vi.fn>).mockReturnValue({
    push: mockPush,
    replace: vi.fn(),
    back: vi.fn(),
  })

  render(<LoginButton />)
  fireEvent.click(screen.getByText('ログイン'))

  expect(mockPush).toHaveBeenCalledWith('/login')
})
```

next/image

`next/image` の `<Image>` コンポーネントは、テスト環境では画像最適化が動作しないため、通常の `<img>` タグに置き換えます。

```
vi.mock('next/image', () => ({
  __esModule: true,
  default: (props) => {
    // Next.js Image 固有の props を除外
    const { fill, priority, quality, placeholder,
      blurDataURL, loader, unoptimized, ...imgProps } = props
    // fill プロパティがある場合はスタイルを適用
    if (fill) {
      imgProps.style = {
        ...imgProps.style,
        objectFit: 'cover',
        position: 'absolute',
        width: '100%',
        height: '100%',
      }
    }
    return <img { ...imgProps} />
  },
}))
```

21.15.5 ブラウザAPIモック

`jsdom` 環境には一部のブラウザAPIが存在しないため、モック化が必要です。

```
// IntersectionObserver (スクロール監視)
class MockIntersectionObserver {
  observe = vi.fn()
  unobserve = vi.fn()
  disconnect = vi.fn()
}
Object.defineProperty(window, 'IntersectionObserver', {
  writable: true,
  configurable: true,
  value: MockIntersectionObserver,
})

// matchMedia (メディアクエリ)
Object.defineProperty(window, 'matchMedia', {
  writable: true,
  value: vi.fn().mockImplementation((query) => ({
    matches: false,
    media: query,
    onchange: null,
    addListener: vi.fn(),
    removeListener: vi.fn(),
    addEventListener: vi.fn(),
    removeEventListener: vi.fn(),
    dispatchEvent: vi.fn(),
  })),
})

// localStorage
const localStorageMock = {
  getItem: vi.fn(),
  setItem: vi.fn(),
  removeItem: vi.fn(),
  clear: vi.fn(),
}
Object.defineProperty(window, 'localStorage', {
  value: localStorageMock,
})
```

21.15.6 vi.unmock() の使い方

グローバルモックを無効化して、**実際の実装**をテストしたい場合に `vi.unmock()` を使います。

```
// グローバルモックで @/lib/logger がモック化されているが、
// このテストでは実際の logger の実装をテストしたい
vi.unmock('@lib/logger')

import logger from '@lib/logger'

describe('Logger (実装テスト)', () => {
  it('開発環境でデバッグログが出力される', () => {
    const spy = vi.spyOn(console, 'log')
    process.env.NODE_ENV = 'development'

    logger.debug('テストメッセージ')

    expect(spy).toHaveBeenCalled()
    spy.mockRestore()
  })
})
```

重要な注意点: `vi.unmock()` はファイルの **トップレベル** (`describe` の外) で呼び出す必要があります。`beforeEach` 内で呼び出しても効果がありません。また、`vi.resetModules()` + `require()` を使っても、グローバルモックが優先される場合があります。確実にモックを解除するには `vi.unmock()` を使いましょう。

21.15.7 モックのリセット方法

メソッド	効果	使い分け
<code>vi.clearAllMocks()</code>	全モックの呼び出し記録をクリア	最も頻繁に使う (<code>beforeEach</code> で)
<code>vi.resetAllMocks()</code>	呼び出し記録 + 実装をリセット	モックの戻り値もリセットしたい場合
<code>vi.restoreAllMocks()</code>	<code>spyOn</code> で作った全モックを元に戻す	<code>spyOn</code> を使ったテストで

```
describe('モックのライフサイクル', () => {
  // 各テスト前にモックをクリア
  beforeEach(() => {
    vi.clearAllMocks()
  })

  // テスト1: fetchが1回呼ばれることを確認
  it('テスト1', () => {
    // fetchの呼び出しカウントは0からスタート
    expect(fetch).not.toHaveBeenCalled()
  })

  // テスト2: テスト1の呼び出し記録は残っていない
  it('テスト2', () => {
    // clearAllMocks() によりリセット済み
    expect(fetch).not.toHaveBeenCalled()
  })
})
```

21.15.8 よくあるモックのトラブルと解決法

トラブル	原因	解決法
モックが効かない	<code>vi.mock()</code> の位置が正しくない	ファイルのトップレベル (<code>describe</code> の外) に書く
型エラーが出る	<code>as ReturnType<typeof vi.fn></code> キャストが不足	<code>(fn as ReturnType<typeof vi.fn>).mockResolvedValue(...)</code> と書く
前のテストの結果が残る	<code>clearAllMocks</code> の呼び忘れ	<code>beforeEach(() => vi.clearAllMocks())</code> を追加
<code>vi.unmock()</code> が効かない	<code>describe</code> 内に書いている	ファイルのトップレベルに移動する
グローバルモックを上書きできない	<code>mockReturnValue</code> を使っていない	<code>(useQuery as ReturnType<typeof vi.fn>).mockReturnValue({ ... })</code> で上書き
<code>window.location.reload</code> をモックできない	読み取り専用プロパティ	<code>Object.defineProperty</code> を使う

```
// window.location.reload のモック例
Object.defineProperty(window.location, 'reload', {
  configurable: true,
  writable: true,
  value: vi.fn(),
})
```

21.15.9 fetchモック

Server Actions やコンポーネントで `fetch` を使う場合のモックパターンです。

```
// 成功レスポンスのモック
global.fetch = vi.fn().mockResolvedValue({
  ok: true,
  status: 200,
  json: () => Promise.resolve({ data: 'test' }),
  text: () => Promise.resolve('test'),
  headers: new Headers({ 'Content-Type': 'application/json' }),
})

// エラーレスポンスのモック
global.fetch = vi.fn().mockResolvedValue({
  ok: false,
  status: 500,
  json: () => Promise.resolve({ error: 'Internal Server Error' }),
})

// ネットワークエラーのモック
global.fetch = vi.fn().mockRejectedValue(new Error('Network Error'))

// 複数回の呼び出しで異なるレスポンスを返す
global.fetch = vi.fn()
  .mockResolvedValueOnce({ ok: true, json: () => Promise.resolve({ page: 1 }) })
  .mockResolvedValueOnce({ ok: true, json: () => Promise.resolve({ page: 2 }) })
```

▶ 理解度チェック(クリックで回答を確認)

21.16 まとめ

この章では、テストの基礎から実践までを包括的に学びました。

学んだ内容の一覧

セクション	内容
21.1	テストの基本概念、テストの種類、テストピラミッド
21.2	Vitest のセットアップ、設定ファイルの書き方
21.3	ユニットテストの書き方（ユーティリティ関数、パリデーション）
21.4	React コンポーネントのテスト（React Testing Library）
21.5	Server Action のテスト（Prisma / auth のモック）
21.6	E2E テスト（Playwright の設定と実践）
21.7	テストカバレッジの概念と目標値
21.8	CI（GitHub Actions）でのテスト自動実行
21.9	段階的な演習問題（基礎・応用・チャレンジ）
21.10	テスト戦略のベストプラクティス、TDD
21.11	React Query テスト（QueryClient設定、useQuery/useMutation）
21.12	Server Action・APIルートテスト（NextRequest/NextResponse、認証モック）
21.13	E2Eテスト詳細（playwright.config.ts、ページオブジェクト、CI設定）
21.14	テスト組織構造（ tests /構成、命名規則、テストカテゴリ）
21.15	モックパターン（グローバルモック、vi.unmock、各モジュールモック）

テストを書くときのチェックリスト

テストを書く前に、以下を確認しましょう。

- ☐ テスト対象は明確か？（1つの関数 or コンポーネント）
- ☐ 正常系と異常系の両方をテストしているか？

- ☐ 境界値（ちょうどの値）をテストしているか？
- ☐ モックは必要最小限か？（モックしすぎると実態と乖離する）
- ☐ テストの説明文は分かりやすいか？
- ☐ テスト間は独立しているか？（順番に依存していないか）
- ☐ `@vitest-environment node` は適切に設定されているか？（Server Actions / APIルートの場合）
- ☐ QueryClient のテスト設定は適切か？（React Query を使う場合）
- ☐ グローバルモックの影響を理解しているか？（`vi.unmock()` が必要な場合）

次のステップ

テストの基礎を理解したら、以下のステップでスキルを磨きましょう。

1. **まずはユーティリティ関数のテストから始める**（最も簡単）
2. **次にコンポーネントのテストに挑戦する**（React Testing Library）
3. **Server Action のテストでモックに慣れる**（実践的）
4. **React Query を使ったコンポーネントテストに挑戦する**（QueryClient設定）
5. **最重要フローのE2Eテストを書く**（仕上げ）
6. **CI にテストを組み込む**（自動化で完成）
7. **vitest.setup.tsx のグローバルモックを理解する**（トラブル対応力向上）

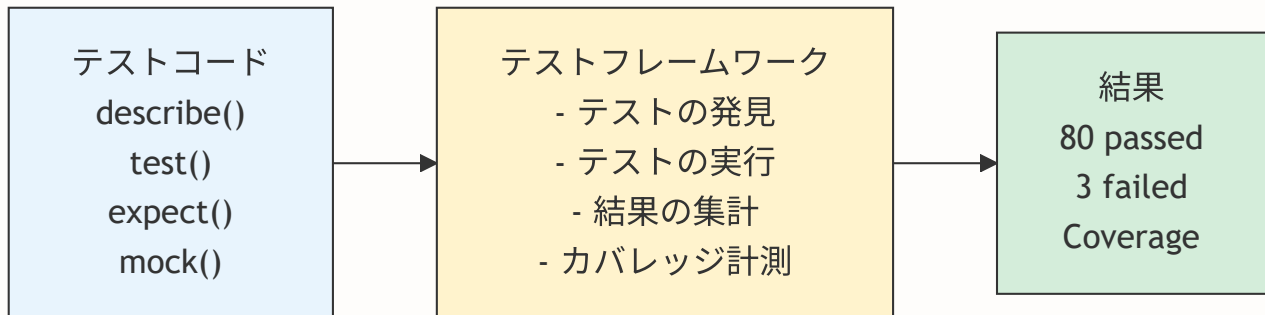
次の章では、CI/CDとデプロイについて学びます。

付録A: 技術選定の背景 -- なぜこの構成を選んだのか

この付録の目的: チュートリアル本編では「Vitestで書く」「Playwrightで書く」という前提で進めましたが、テストツールには多くの選択肢があります。ここでは、BON-LOGのテスト基盤を設計する際に検討した選択肢と判断の理由を、初心者向けに詳しく解説します。

A.1 テストフレームワークの選択肢

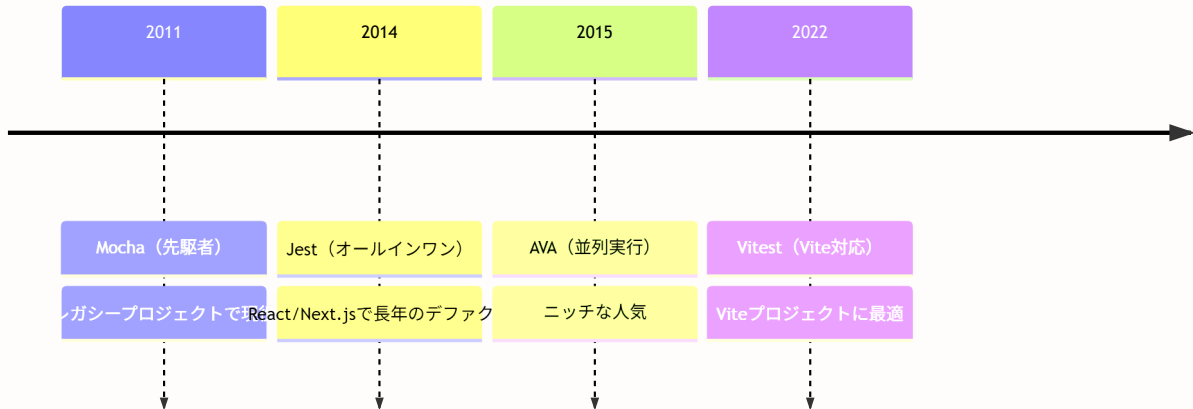
テストフレームワークとは、テストコードを「書く」「実行する」「結果を確認する」ための基盤ツールです。



各フレームワークの比較

項目	Jest	Vitest	Mocha	AVA
開発元	Meta (Facebook)	Vue.jsチーム	TJ Holowaychuk	Sindre Sorhus
初回リリース	2014年	2022年	2011年	2015年
設定の簡単さ	ゼロコンフィグ	ゼロコンフィグ	要設定	少し必要
実行速度	中程度	非常に高速	中程度	高速
スナップショット	標準機能	標準機能	プラグイン必要	プラグイン必要
カバレッジ	標準機能	標準機能	別途設定	別途設定
モック機能	強力（標準）	強力（標準）	別ライブラリ (sinon)	別ライブラリ
ウォッチモード	標準機能	標準機能	標準機能	標準機能
TypeScript	ts-jest / SWC	ネイティブ対応	要設定	要設定
ESM対応	実験的	ネイティブ	対応	対応
コミュニティ	非常に大きい	急成長中	大きい	中程度
npm週間DL	約3000万	約500万	約800万	約30万
Next.js対応	公式サポート	公式サポート + Viteエコシステム	公式サポートなし	公式サポートなし

テストフレームワークの歴史と位置づけ



なぜVitestを選んだのか

BON-LOGでは**Vitest**を採用しています。以下がその理由です。

1. Viteエコシステムとの親和性

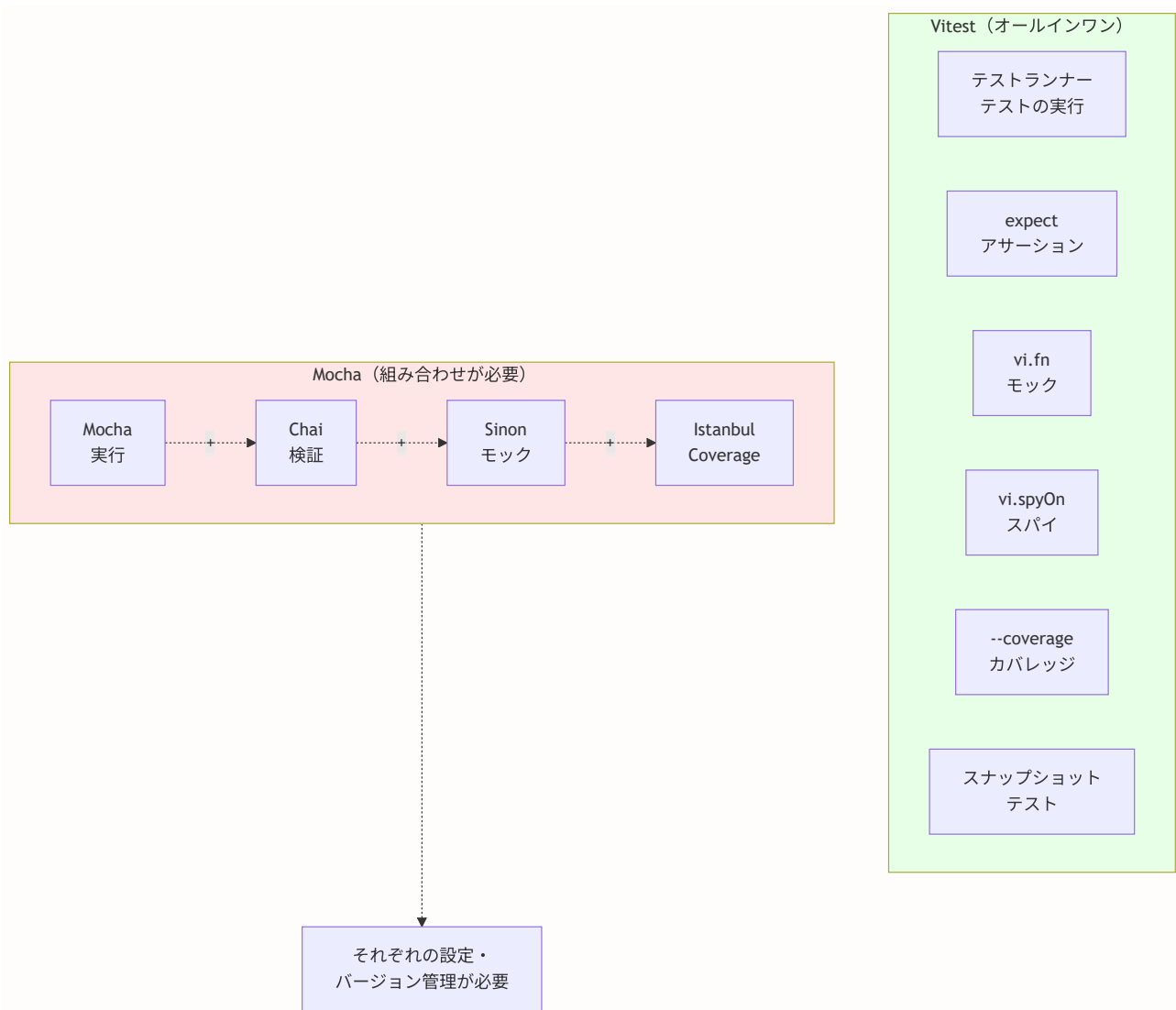
Vitest は Vite のエコシステム上に構築されており、`vite-tsconfig-paths` プラグインで Next.js のパスエイリアス (`@/`) も自動解決できます。

```
// vitest.config.ts - BON-LOGの実際のセットアップ（簡略化）
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'
import tsconfigPaths from 'vite-tsconfig-paths'

export default defineConfig({
  plugins: [react(), tsconfigPaths()],
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: ['./vitest.setup.tsx'],
  },
})
```

2. オールインワンで設定が最小限

Vitest は、テストランナー、アサーションライブラリ、モック機能、カバレッジツールが全て統合されています。



3. スナップショットテストが標準機能

UIコンポーネントの出力が意図せず変わっていないかを簡単に確認できます。

```
// スナップショットテストの例
test('PremiumBadge renders correctly', () => {
  const { container } = render(<PremiumBadge />)
  expect(container).toMatchSnapshot()
  // → 初回実行時にスナップショットファイルを自動生成
  // → 2回目以降、出力が変わるとテストが失敗
  // → 意図的な変更なら `vitest run --updateSnapshot` で更新
})
```

4. ESMネイティブ対応と高速性

VitestはESモジュールにネイティブ対応しており、TypeScriptの変換も高速です。BON-LOGの455テストスイート（8,800以上のテスト）が3～5分で完了します。

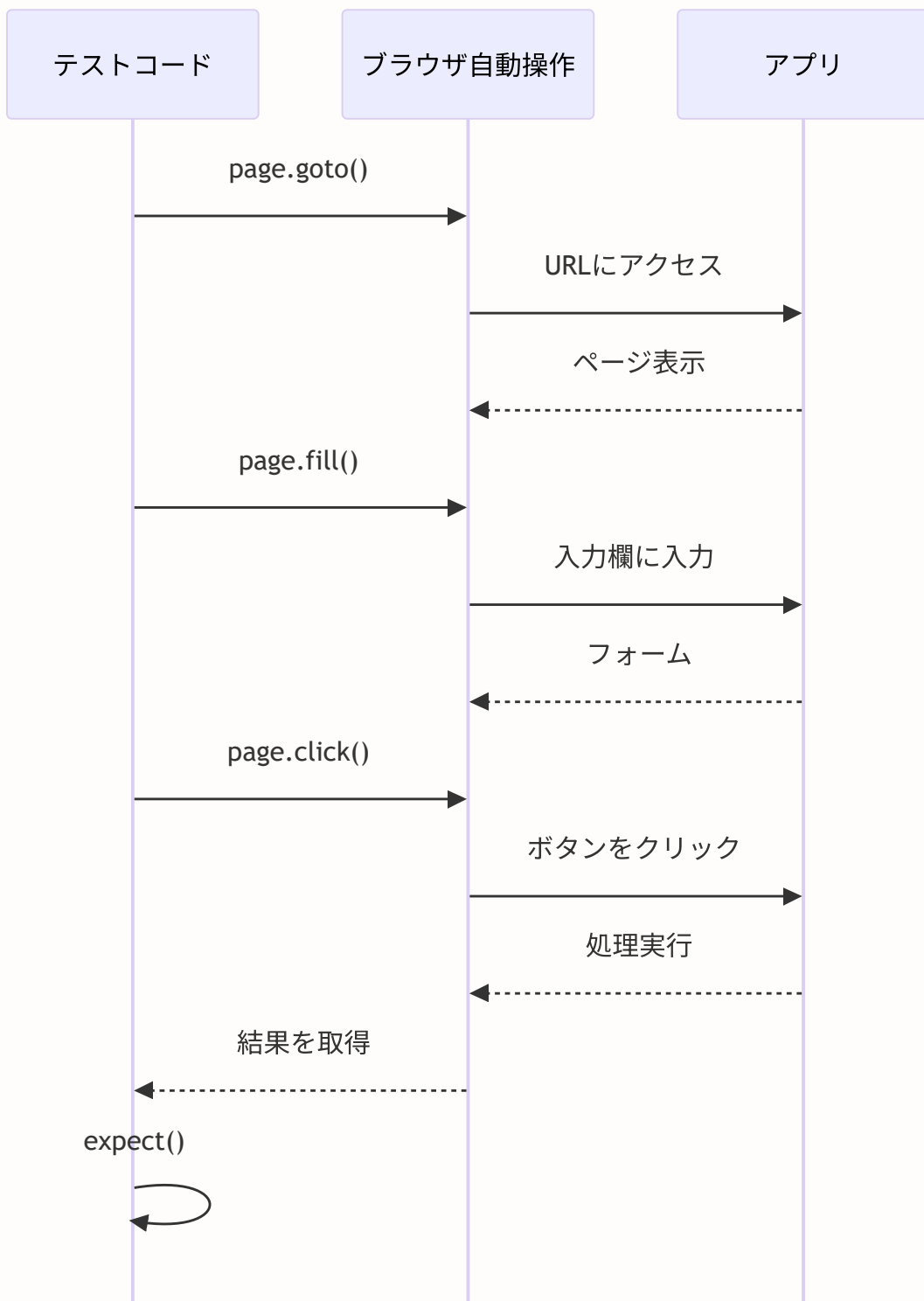
5. Jest互換API

VitestはJest互換のAPIを提供しているため、Jestの知識や学習リソースがそのまま活かれます。`vi.fn()`（Jestの`jest.fn()`相当）、`vi.mock()`（`jest.mock()`相当）など、関数名の接頭辞が`jest.` → `vi.`に変わるだけです。

初心者へのアドバイス: VitestはJest互換APIを持つため、Jestの知識はそのまま役立ちます。新規プロジェクトでは、ESMネイティブ対応と高速性からVitestを第一候補として検討してください。

A.2 E2Eテストフレームワークの選択肢

E2E（End-to-End）テストフレームワークは、実際のブラウザを操作してアプリ全体の動作を検証するツールです。



各フレームワークの比較

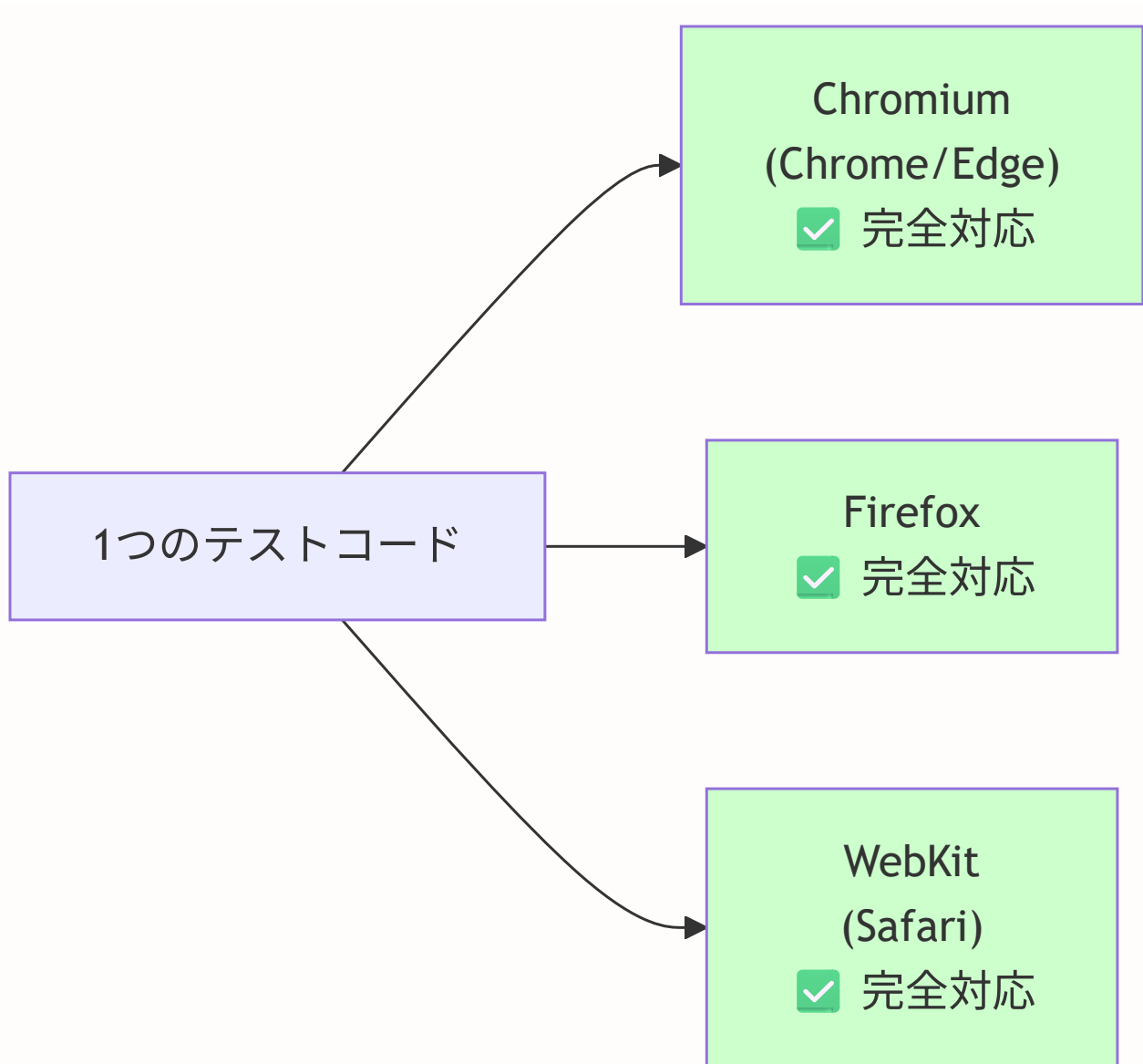
項目	Playwright	Cypress	Selenium	TestCafe
開発元	Microsoft	Cypress社	SeleniumHQ	DevExpress
対応ブラウザ	Chromium, Firefox, WebKit	Chromium, Firefox, (WebKit限定)	全ブラウザ	全ブラウザ
実行速度	非常に高速	高速	遅い	中程度
並列実行	標準機能	有料プラン	要設定	標準機能
自動待機	標準機能	標準機能	手動設定	標準機能
デバッグ	Trace Viewer	Time Travel	ログベース	ログベース
API テスト	標準機能	標準機能	不可	不可
モバイルエミュレーション	標準機能	viewport変更のみ	外部ツール	viewport変更
Next.js公式対応	推奨	対応	非推奨	非推奨
学習コスト	低い	低い	高い	中程度
無料で使える範囲	全機能無料	基本機能無料	全機能無料	基本機能無料

各ツールの特徴を一言で：

Playwright: 「速くて全ブラウザ対応、全機能無料」
 Cypress: 「DXが良く人気だが、マルチブラウザは限定的」
 Selenium: 「歴史が長いけど、モダンな開発には重い」
 TestCafe: 「インストール不要でブラウザ内で実行」

なぜPlaywrightを選んだのか

1. マルチブラウザ対応が標準



メリット

- 1つのテストコードで3つのブラウザを同時テスト可能
- Cypressの場合、WebKit(Safari)対応は実験的

2. 実行が高速

Playwrightはブラウザとの通信にChrome DevTools Protocol (CDP) を直接使用するため、オーバーヘッドが少なく高速です。

テスト100件の実行時間（目安）：

Playwright: 約30秒（並列実行）
Cypress: 約2分（デフォルトは直列）
Selenium: 約5分（セットアップコスト大）

3. Next.jsの公式ドキュメントで推奨

Next.jsのドキュメントでは、E2EテストにPlaywrightを第一候補として紹介しています。`create-next-app`でのセットアップオプションにも含まれています。

4. Trace Viewerによる強力なデバッグ

テストが失敗したとき、Playwright Trace Viewerで「何が起きたか」を視覚的に確認できます。

Playwright Trace Viewer -- テスト失敗時に確認できること:

#	確認できる情報
1	各ステップのスクリーンショット
2	ネットワークリクエストの内容
3	コンソールログ
4	DOMのスナップショット
5	各アクションの実行時間

「なぜテストが失敗したか」をビデオのように巻き戻して確認できる

5. 全機能が無料

Playwrightは全機能がオープンソースで無料です。Cypressの場合、ダッシュボード（テスト結果の記録・共有）や並列実行は有料プランが必要です。

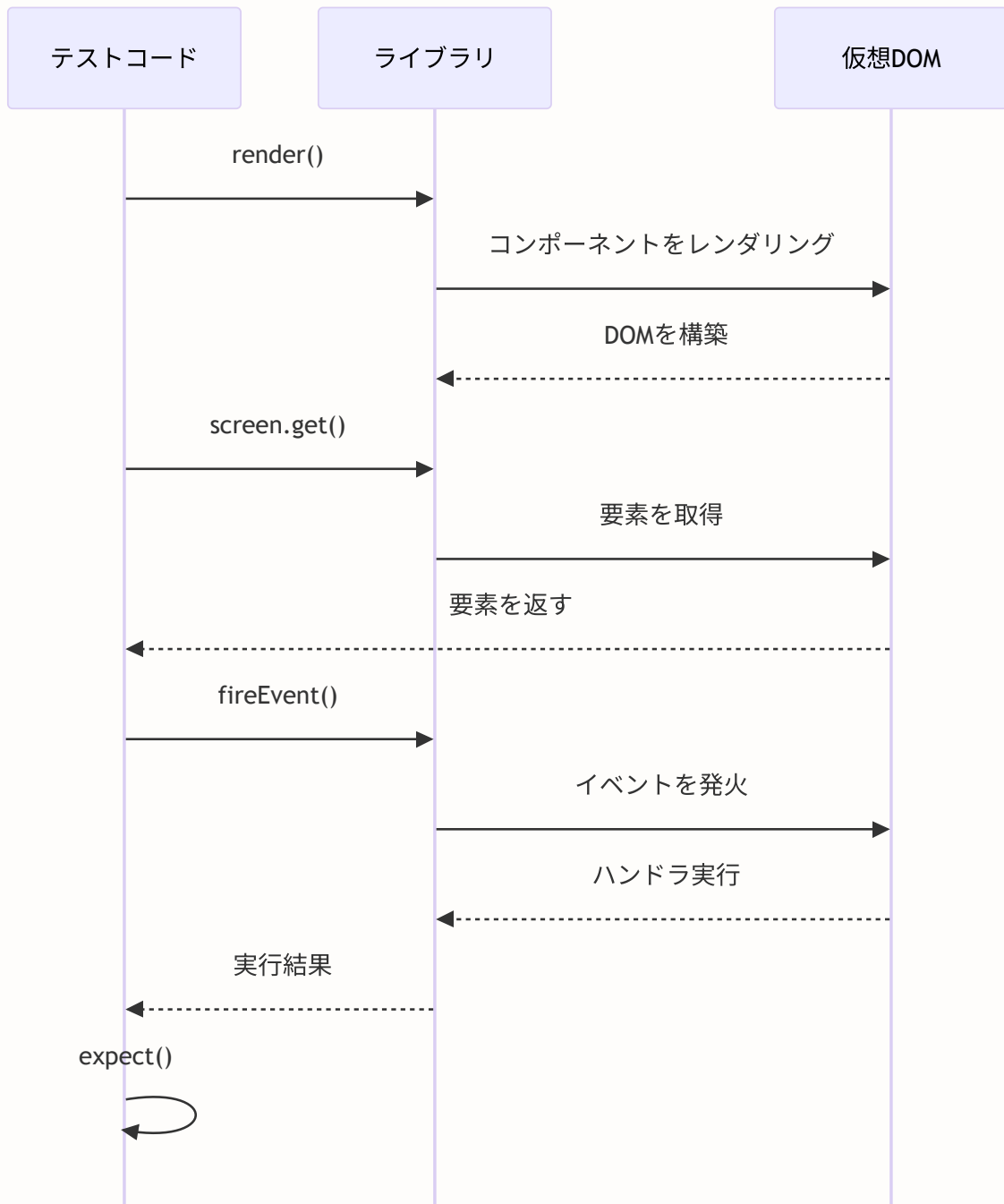
Cypressを選ばなかった理由

Cypressも優れたツールですが、以下の点でPlaywrightを優先しました。

- WebKit（Safari）対応が実験的
- 並列実行が有料プラン
- `iframe` や複数タブのテストに制限がある
- Next.jsの公式推奨がPlaywright

A.3 コンポーネントテストライブラリの選択肢

Reactコンポーネントのテストを書くためのライブラリにも選択肢があります。



各ライブラリの比較

項目	React Testing Library (RTL)	Enzyme	React Test Utils
開発元	Kent C. Dodds	Airbnb	React公式
テスト哲学	ユーザー視点	実装詳細	低レベルAPI
要素の取得方法	テキスト、ラベル、ロール	CSSセレクタ、コンポーネント名	DOM操作
React 19対応	完全対応	非対応（開発停止）	完全対応
Server Components	対応	非対応	対応
学習コスト	低い	中程度	高い
メンテナンス状態	活発	開発停止	公式メンテ
React公式推奨	推奨	非推奨	低レベルとして提供

テスト哲学の違い（最も重要な違い）：

Enzyme（実装詳細をテスト）：

- ✅ `component.state().count === 5` ← 内部stateを直接確認
- ✅ `component.instance().handleClick()` ← 内部メソッドを呼ぶ
- コンポーネントの「中身」を知っている必要がある
- リファクタリングでテストが壊れやすい

RTL（ユーザー視点でテスト）：

- ✅ `screen.getByText('5')` ← ユーザーに見えるテキストで確認
- ✅ `fireEvent.click(button)` ← ユーザーの操作をシミュレート
- コンポーネントの「中身」を知る必要がない
- リファクタリングしてもテストが壊れにくい

なぜReact Testing Libraryを選んだのか

1. ユーザー視点のテストが書ける

RTLの哲学は「**The more your tests resemble the way your software is used, the more confidence they can give you.**」（テストがソフトウェアの実際の使い方に近いほど、テストから得られる信頼性は高い）」です。

```
// RTL でのテスト例（ユーザー視点）
test('いいねボタンをクリックするといいね数が増える', async () => {
  render(<LikeButton postId="1" initialCount={5} />)

  // ユーザーに見えるテキストで要素を取得
  const button = screen.getByRole('button', { name: /いいね/i })
  expect(screen.getByText('5')).toBeInTheDocument()

  // ユーザーの操作をシミュレート
  await userEvent.click(button)

  // ユーザーに見える結果を確認
  expect(screen.getByText('6')).toBeInTheDocument()
})
```

2. アクセシビリティの改善につながる

RTLは `getByRole`、`getByLabelText`、`getByAltText` 等、アクセシビリティ属性を使って要素を取得します。そのため、テストを書くこと自体がアクセシビリティの改善につながります。

RTL の要素取得の優先順位:

1. `getByRole` ← アクセシビリティロール（推奨）
2. `getByLabelText` ← フォームラベル
3. `getByPlaceholderText` ← プレースホルダー
4. `getByText` ← テキスト内容
5. `getByDisplayValue` ← フォームの現在値
6. `getByAltText` ← 画像のalt属性
7. `getByTitle` ← title属性
8. `getByTestId` ← data-testid属性（最終手段）

3. Reactの公式推奨

React公式ドキュメントでRTLが推奨ライブラリとして紹介されています。Enzymeは公式では推奨されていません。

4. Enzymeは開発停止

EnzymeはReact 18以降に対応しておらず（BON-LOGはReact 19を使用）、事実上開発が停止しています。新規プロジェクトでEnzymeを採用する理由はありません。

A.4 専門用語集

テストに関連する専門用語をまとめます。本編を読む前に一読しておくとう理解がスムーズです。

用語	英語表記	説明
ユニットテスト	Unit Test	プログラムの最小単位（関数、クラス、コンポーネント）を個別にテストすること。「部品検査」のようなもの。実行が速く、バグの原因を特定しやすい
統合テスト	Integration Test	複数のモジュールを組み合わせで正しく連携するかテストすること。「組み立て検査」のようなもの。例：フォーム送信 → バリデーション → DB保存の一連の流れ
E2Eテスト	End-to-End Test	ユーザーの操作を実際のブラウザで再現してアプリ全体の動作をテストすること。「完成品検査」のようなもの。例：ログイン → 投稿 → いいね → ログアウト
モック	Mock	テスト対象の依存先を「偽物」に置き換えること。例：実際のデータベースの代わりに「決まったデータを返す偽物」を使う。テストを高速かつ安定にする
スタブ	Stub	特定の呼び出しに対して「決まった値を返す」偽物。モックの一種だが、呼び出されたかどうかの検証はしない。例： <code>getUser()</code> が常に <code>{ name: 'テスト' }</code> を返す
スパイ	Spy	関数の呼び出しを「監視」する仕組み。元の関数の動作は変えずに、「何回呼ばれたか」「どんな引数で呼ばれたか」を記録する。Vitestでは <code>vi.spyOn()</code> で作成
カバレッジ	Coverage	テストによって実行されたコードの割合。ステートメントカバレッジ（文の実行率）、ブランチカバレッジ（条件分岐の実行率）等がある。100%が目標だが、80%以上が現実的な目安
アサーション	Assertion	テストの「期待される結果」を宣言すること。 <code>expect(result).toBe(5)</code> のように書く。アサーションが失敗するとテストが失敗する
テストダブル	Test Double	テストで使う「偽物」の総称。モック、スタブ、スパイ、フェイク、ダミーを含む。映画の「スタントダブル（代役）」が語源
テストランナー	Test Runner	テストファイルを見つけて実行するツール。Vitest自体がテストランナーの機能を持つ
テストスイート	Test Suite	関連するテストをまとめたグループ。Vitestでは <code>describe()</code> ブロックがテストスイートに相当する
フィクスチャ	Fixture	テストに必要な初期データや状態。テストの「前提条件」を作るためのデータ。例：テスト用のユーザーデータ

セットアップ/ティアダウン	Setup / Teardown	テストの前後に実行する処理。Vitestでは <code>beforeEach</code> / <code>afterEach</code> 、 <code>beforeAll</code> / <code>afterAll</code> で定義
スナップショットテスト	Snapshot Test	コンポーネントの出力をファイルに保存し、次回実行時に変更がないか比較するテスト。意図しないUI変更を検出できる
テストピラミッド	Test Pyramid	ユニットテスト（多）、統合テスト（中）、E2Eテスト（少）のバランスを示すモデル。下に行くほど数が多く、上に行くほど少ない
TDD	Test-Driven Development	テストを先に書き、そのテストを通すコードを後から書く開発手法。「Red（テスト失敗）→ Green（テスト成功）→ Refactor（改善）」のサイクルで進める
CI	Continuous Integration	コードの変更を頻繁にメインブランチに統合し、自動テストで品質を確認するプラクティス。GitHub Actionsで自動化する
フレーキーテスト	Flaky Test	同じコードなのに実行するたびに成功したり失敗したりする不安定なテスト。タイミング依存、外部サービス依存が主な原因。CI/CDの信頼性を下げるため、早急な修正が必要

テストダブルの種類（全体像）：

テストダブル（Test Double）

- └─ ダミー（Dummy）
引数を埋めるためだけの値。実際には使われない。
例：テスト対象が引数を3つ取るが、3つ目は使わない場合
- └─ スタブ（Stub）
決まった値を返す偽物。
例：`getUser()` → 常に `{ name: 'テスト' }` を返す
- └─ スパイ（Spy）
呼び出しを監視する。元の動作は変えない。
例：`console.log` が何回呼ばれたか記録
- └─ モック（Mock）
呼び出しの検証も行う偽物。
例：`sendEmail()` が1回だけ呼ばれたことを検証
- └─ フェイク（Fake）
簡易版の実装。
例：実際のDBの代わりにインメモリDBを使う

カバレッジ種別	計測内容	目安
ステートメントカバレッジ	全コード行のうち何%が実行されたか	80%以上
ブランチカバレッジ	全条件分岐のうち何%が実行されたか	70%以上
関数カバレッジ	全関数のうち何%が呼ばれたか	80%以上
行カバレッジ	全行のうち何%が実行されたか	80%以上

BON-LOG の実績: Statements 98.12%, Branches 94.42%, Functions 97.42%, Lines 98.60%

この付録のまとめ: テストツールの選定は「プロジェクトのフレームワークとの相性」が最も重要です。Vitest は Vite ベースの高速なテスト実行、ネイティブ TypeScript/ESM 対応、Jest 互換 API を備えており、Next.js プロジェクトでの利用も成熟しています。BON-LOG では Vitest + React Testing Library + Playwright の組み合わせを採用し、高速かつ安定したテスト環境を実現しています。

付録B: よくある質問（FAQ）

この付録の目的: テストを書き始めると、多くの疑問や壁にぶつかります。ここでは、BON-LOG プロジェクトでの開発経験から集めた「よくある質問」を、初心者がつまずきやすいポイントを中心にまとめました。

B.1 テスト全般に関する質問

Q: テストを書く時間がもったいなく感じます。本当に必要ですか？

A: 短期的には「テストを書かない方が速い」のは事実です。しかし中長期的に見ると、テストがないプロジェクトは以下の問題を抱えます。

開発初期
機能追加が速い → テスト
不要と感じる

テストがある場合
機能追加のたびにテストも
書く
→ コード変更後、数分で全
機能の動作確認完了
→ 安心してリファクタリン
グ・機能追加ができる

開発中期（機能が10個以
上）
「Aを直したらBが壊れた」
が頻発
→ 手動確認に時間がかかる
→ リリース前に不安が募る

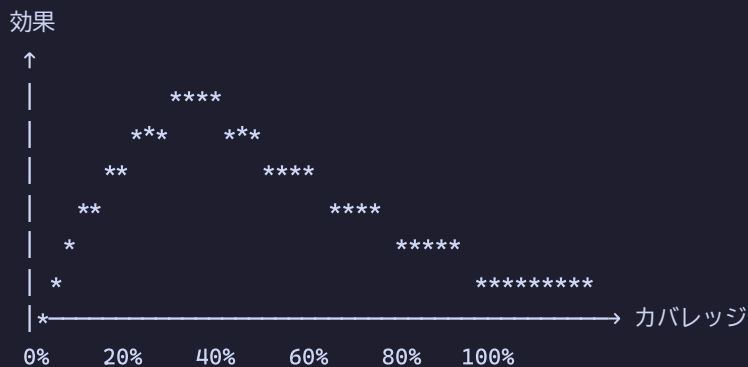
開発後期（機能が30個以
上）
怖くてコードに触れなくな
る
→ 技術的負債が雪だるま式
に増加
→ 最終的にゼロから書き直
し...

BON-LOG プロジェクトでは、テストのおかげで「投稿機能の内部実装を大幅に変更しても、既存のいいね・ブックマーク・コメント機能が壊れていないことを即座に確認できる」という恩恵を受けています。

Q: テストカバレッジ100%を目指すべきですか？

A: いいえ、100%を目指す必要はありません。80%前後が現実的で効果的な目標です。

カバレッジと労力の関係:



注目:

- 0% → 60%: 少ない労力で大きな効果
- 60% → 80%: まだ効果的
- 80% → 95%: 労力が増えるが効果は小さい
- 95% → 100%: 膨大な労力で微小な効果

BON-LOG の実績 (Statements 98.12%, Branches 94.42%) は、`coverage-boost/` ディレクトリの追加テストによって達成されていますが、最初から高カバレッジを目指したわけではなく、段階的に向上させた結果です。

Q: ユニットテストとインテグレーションテスト、どちらから書くべきですか？

A: ユニットテストから書き始めましょう。理由は以下の通りです。

観点	ユニットテスト	インテグレーションテスト
難易度	低い（モックが少ない）	高い（複数の依存関係を管理）
実行速度	速い（ミリ秒）	やや遅い（秒）
デバッグ	簡単（原因が1箇所にある）	難しい（問題の箇所が不明）
フィードバック	即座	やや時間がかかる

具体的には、以下の順番がおすすめです。

- ユーティリティ関数のテスト
 - `formatDate()`, `truncateText()` など
 - モックが不要で最も簡単
- バリデーション関数のテスト
 - `createPostSchema`, `loginSchema` など
 - 正常系・異常系・境界値のパターン学習
- シンプルなコンポーネントのテスト
 - `Button`, `Badge`, `Avatar` など
 - `React Testing Library` の基礎
- 状態を持つコンポーネントのテスト
 - `LikeButton`, `BookmarkButton` など
 - モックと非同期処理の学習
- Server Actionsのテスト
 - `createPost`, `toggleLike` など
 - `Prisma/auth`モックの学習
- E2Eテスト
 - ログイン → 投稿 → いいね の一連フロー
 - `Playwright` の学習

Q: テストファイルはソースコードと同じディレクトリに置くべきですか？別ディレクトリにまとめるべきですか？

A: どちらの方式にもメリット・デメリットがあります。BON-LOG では `__tests__` ディレクトリに集約する方式を採用しています。

方式1: 同じディレクトリに配置 (コロケーション)

```
lib/
├─ utils/
│   ├─ date.ts           ← ソースコード
│   └─ date.test.ts      ← テストファイル (隣に配置)
├─ actions/
│   ├─ post.ts
│   └─ post.test.ts
```

メリット:

- ソースとテストの対応が一目でわかる
- ファイル移動時にテストも一緒に移動される
- IDEの「テストヘジャンプ」機能と相性が良い

デメリット:

- ディレクトリが散らかりやすい
- テスト用のモックデータの共有が面倒

方式2: __tests__/ ディレクトリに集約 (BON-LOGの方式)

```
__tests__/
├─ lib/
│   ├─ actions/
│   │   └─ post.test.ts ← テストファイル
│   └─ utils/
│       └─ date.test.ts
├─ components/
│   └─ post/
│       └─ PostCard.test.tsx
└─ utils/
    └─ test-utils.tsx ← 共通ユーティリティ
```

メリット:

- テスト関連ファイルが1箇所にまとまる
- 共通のモックデータやヘルパーが使いやすい
- ソースコードの可読性が保たれる

デメリット:

- テストとソースの距離が離れる
- ディレクトリ構造の同期が必要

Q: テストを書いているとモックだらけになってしまいます。これは正しいですか？

A: ある程度のモックは必要ですが、モックが多すぎる場合はコード設計を見直すサインかもしれません。

モックが多い = 依存関係が多い

モックが多すぎるケース: <code>createPost()</code>	モック
<code>auth()</code>	モック1
<code>prisma</code>	モック2
<code>rateLimit()</code>	モック3
<code>uploadFile()</code>	モック4
<code>sanitize()</code>	モック5
<code>sendEmail()</code>	モック6
<code>revalidatePath()</code>	モック7

7つのモックが必要 → 関数の責務が大きすぎる可能性 → 関数を分割してテストابلにする

改善後: `createPost()`

依存関数	モック要否
<code>auth()</code>	モック1 (必要)
<code>validatePostInput()</code>	テスト済みなのでモック不要
<code>savePostToDb()</code>	テスト済みなのでモック不要
<code>notifyMentions()</code>	テスト済みなのでモック不要

各関数を個別にテストし、統合部分は最小限のモックで済む

BON-LOG の `vitest.setup.tsx` でグローバルモックを定義しているのは、まさにこの「モックだらけ問題」を軽減するための工夫です。

B.2 Vitest に関する質問

Q: `vi.fn()` と `vi.spyOn()` の使い分けは？

A: 以下のルールで使い分けます。

```
// vi.fn(): 新しいモック関数を作成
// → 存在しない関数の代わりに使う場合
const onClick = vi.fn()
render(<Button onClick={onClick}>クリック</Button>)

// vi.spyOn(): 既存の関数を監視 (+必要に応じて置換)
// → 既存の関数の呼び出しを確認したい場合
const spy = vi.spyOn(console, 'error')
// 何かの処理を実行 ...
expect(spy).toHaveBeenCalled('エラーメッセージ')
spy.mockRestore() // 元に戻る
```

使い分けの判断フロー:

その関数は既に存在する？

```
|
├─ いいえ → vi.fn() で新しいモック関数を作成
|   例: onClick, onSubmit などのイベントハンドラ
|
└─ はい → vi.spyOn() で既存関数を監視
    |
    ├─ 元の関数を実行させたい → spyOn のみ
    |   例: console.error の呼び出しを監視
    |
    └─ 別の動作に置換したい → spyOn + mockReturnValue
        例: Math.random() を常に 0.5 を返すように
```

Q: `mockResolvedValue` と `mockResolvedValueOnce` の違いは？

A: 「何回呼ばれても同じ値を返すか」「1回だけ特定の値を返すか」の違いです。

```
// mockResolvedValue: 何回呼ばれても同じ値を返す
const fetchUser = vi.fn()
fetchUser.mockResolvedValue({ id: '1', name: 'Alice' })

await fetchUser() // → { id: '1', name: 'Alice' }
await fetchUser() // → { id: '1', name: 'Alice' } (同じ値)
await fetchUser() // → { id: '1', name: 'Alice' } (同じ値)

// mockResolvedValueOnce: 1回だけ特定の値を返す (使い捨て)
const fetchData = vi.fn()
fetchData.mockResolvedValueOnce({ page: 1, data: ['a'] })
fetchData.mockResolvedValueOnce({ page: 2, data: ['b'] })

await fetchData() // → { page: 1, data: ['a'] } (1回目)
await fetchData() // → { page: 2, data: ['b'] } (2回目)
await fetchData() // → undefined (設定が切れた)
```

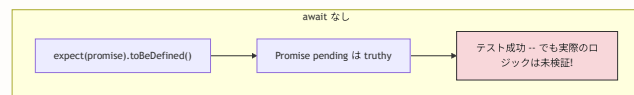
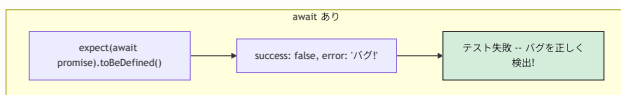
ページネーションのテストなど、呼び出しごとに異なるデータを返したい場合に `mockResolvedValueOnce` を使います。

Q: テストで `async/await` を使い忘れるとどうなりますか？

A: テストが常に成功してしまい、バグを検出できなくなる危険があります。

```
// ❌ 悪い例: await を忘れている
it('投稿を作成できる', () => {
  const result = createPost({ content: 'テスト' })
  // result は Promise オブジェクト (まだ解決していない)
  // → expect が Promise 自体を評価 → テストが無意味に通る
  expect(result).toBeDefined() // Promise は truthy なので常に通る!
})

// ✅ 良い例: await でPromiseを解決してから検証
it('投稿を作成できる', async () => {
  const result = await createPost({ content: 'テスト' })
  // result は解決後の実際の値
  expect(result.success).toBe(true)
})
```



対策: ESLint の `no-floating-promises` ルールを有効にすると、`await` の付け忘れを自動検出できます。

Q: `vi.useFakeTimers()` はどのような場面で使いますか？

A: `setTimeout`, `setInterval`, `Date.now()` など、時間に依存する処理をテストする場合に使います。

```
describe('useDebounce', () => {
  // —— タイマーモック設定 ——
  beforeEach(() => {
    // setTimeoutやsetIntervalを偽物に置き換え
    // → テスト内で時間の進み方を自由にコントロールできる
    vi.useFakeTimers()
  })
  afterEach(() => {
    // テスト後に本物のタイマーに戻る
    // (他のテストに影響を与えないため)
    vi.useRealTimers()
  })

  it('500ms後に値が更新される', () => {
    const { result, rerender } = renderHook(
      ({ value }) => useDebounce(value, 500),
      { initialProps: { value: 'hello' } }
    )

    // 値を変更
    rerender({ value: 'world' })

    // まだ500ms経っていないので古い値のまま
    expect(result.current).toBe('hello')

    // 時間を499ms進める (まだ足りない)
    act(() => {
      vi.advanceTimersByTime(499)
    })
    expect(result.current).toBe('hello') // まだ古い値

    // さらに1ms進める (合計500ms)
    act(() => {
      vi.advanceTimersByTime(1)
    })
    expect(result.current).toBe('world') // 新しい値に更新!
  })
})
```

FakeTimers の仕組み:

通常 (RealTimers) :

テストコード → setTimeout(fn, 500) → 500ms待つ ... → fn実行
→ テストが遅い!

FakeTimers:

テストコード → setTimeout(fn, 500) → タイマーは停止中
→ vi.advanceTimersByTime(500) → 即座にfn実行
→ テストが高速!

よく使うメソッド:

メソッド	動作
`vi.advanceTimersByTime(ms)`	指定ms分時間を進める
`vi.runAllTimers()`	全タイマーを実行
`vi.runOnlyPendingTimers()`	待機中のタイマーのみ実行
`vi.advanceTimersToNextTimer()`	次のタイマーまで進める

B.3 React Testing Library に関する質問

Q: `getByText` と `queryByText` と `findByText` の違いは?

A: 要素が見つからなかったときの動作と、非同期対応の有無が異なります。

メソッド	見つからない時	非同期対応	主な用途
<code>getByText</code>	エラーを投げる	なし (同期)	存在する要素の取得
<code>queryByText</code>	<code>null</code> を返す	なし (同期)	存在しないことの確認
<code>findByText</code>	タイムアウトでエラー	あり (async)	非同期で表示される要素

```
// getByText: 要素が確実に存在するとき
const heading = screen.getByText('ログイン')
expect(heading).toBeInTheDocument()

// queryByText: 要素が存在しないことを確認するとき
const error = screen.queryByText('エラー')
expect(error).not.toBeInTheDocument() // null なので通る

// findByText: 非同期で表示される要素を待つとき
const message = await screen.findByText('投稿しました')
expect(message).toBeInTheDocument()
```

よくある間違い:

✗ `getByText` で「存在しないこと」を確認
`expect(screen.getByText('エラー')).not.toBeInTheDocument()`
→ `getByText` は見つからないと即座にエラーを投げるため、
`not.toBeInTheDocument()` に到達しない！

✓ `queryByText` で「存在しないこと」を確認
`expect(screen.queryByText('エラー')).not.toBeInTheDocument()`
→ `queryByText` は見つからなければ `null` を返すため、
`not.toBeInTheDocument()` が正しく評価される

Q: `fireEvent` と `userEvent` の違いは？どちらを使うべきですか？

A: `userEvent` の方がより実際のユーザー操作に近く、推奨されます。

```
import { fireEvent } from '@testing-library/react'
import userEvent from '@testing-library/user-event'

// ——— fireEvent: 低レベルのイベント発火 ———
// DOMイベントを直接発火する。シンプルだが実際のユーザー操作との乖離がある
fireEvent.click(button) // click イベントだけ発火
fireEvent.change(input, { target: { value: 'test' } }) // change のみ

// ——— userEvent: ユーザー操作のシミュレーション ———
// 実際のユーザー操作に伴う全てのイベントを発火する
const user = userEvent.setup()
await user.click(button) // focus → pointerdown → mousedown → pointerup
// → mouseup → click の順で全て発火
await user.type(input, 'test') // 1文字ずつ focus → keydown → keypress →
// input → keyup → change を発火
```

イベント発火の違い:

```
fireEvent.click(button):
  click
```

```
userEvent.click(button):
  pointerenter → pointerdown → mousedown → pointerup →
  mouseup → click → focus
```

```
fireEvent.change(input, { target: { value: 'abc' } }):
  change (値が一気に変わる)
```

```
userEvent.type(input, 'abc'):
  keydown('a') → keypress('a') → input('a') → keyup('a') →
  keydown('b') → keypress('b') → input('ab') → keyup('b') →
  keydown('c') → keypress('c') → input('abc') → keyup('c')
(1文字ずつ入力される = 実際のユーザー操作と同じ)
```

BON-LOG のテストでは、シンプルなクリックテストには `fireEvent` を、フォーム入力のテストには `userEvent` を使い分けています。

Q: `screen.debug()` の使い方は？

A: テストが失敗したときに、実際のDOM構造を確認するためのデバッグツールです。

```

it('投稿が表示される', () => {
  render(<PostCard post={mockPost} />)

  // DOM構造をコンソールに出力（デバッグ用）
  screen.debug()
  // 出力例:
  // <body>
  //   <div>
  //     <article class="post-card">
  //       <p>テスト投稿です</p>
  //       <span>5</span>
  //       ...
  //     </article>
  //   </div>
  // </body>

  // 特定の要素だけデバッグ
  const card = screen.getByTestId('post-card')
  screen.debug(card)

  // 長いDOMの場合、出力を制限
  screen.debug(undefined, 10000) // 最大10000文字まで表示
})

```

ヒント: テストが `Unable to find an element with the text: ...` で失敗したときは、まず `screen.debug()` を呼んで、実際にどのようなDOMが生成されているかを確認しましょう。多くの場合、テキストの微妙な違い（改行、空白、` ` など）が原因です。

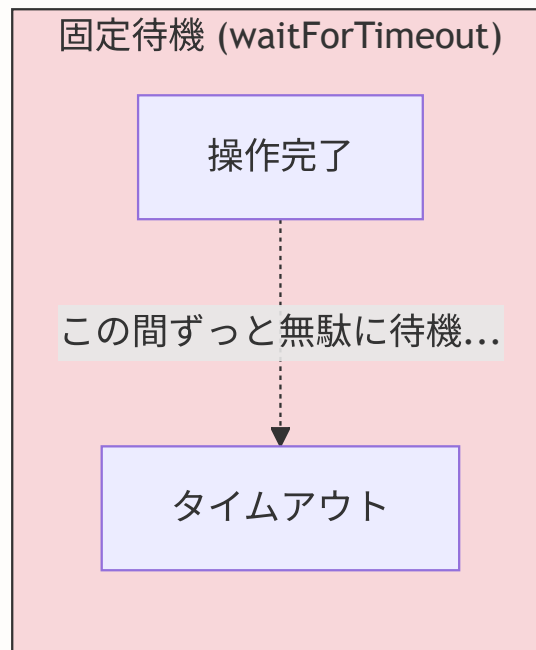
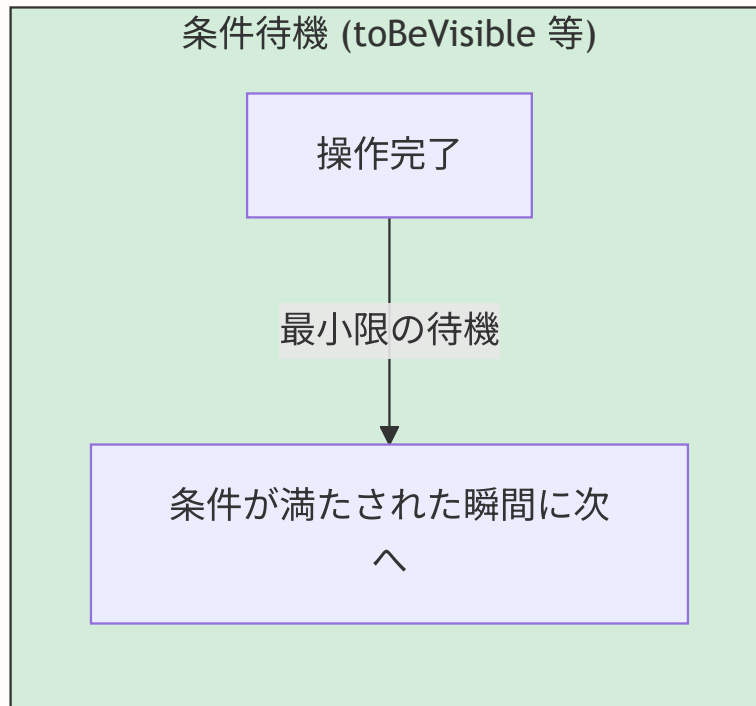
B.4 Playwright（E2E）に関する質問

Q: E2Eテストで `page.waitForTimeout()` を使ってもよいですか？

A: 避けるべきです。固定時間の待機はテストを不安定にします。

```
// ❌ 悪い例: 固定時間の待機
await page.goto('/feed')
await page.waitForTimeout(3000) // 3秒待つ
// → 環境によっては3秒では足りないかもしれない
// → 環境によっては3秒も待つ必要がないかもしれない

// ✅ 良い例: 条件ベースの待機
await page.goto('/feed')
await expect(page.getByText('タイムライン')).toBeVisible()
// → 「タイムライン」テキストが表示されるまで自動で待機
// → 表示された瞬間に次のステップへ進む（無駄な待ちなし）
```

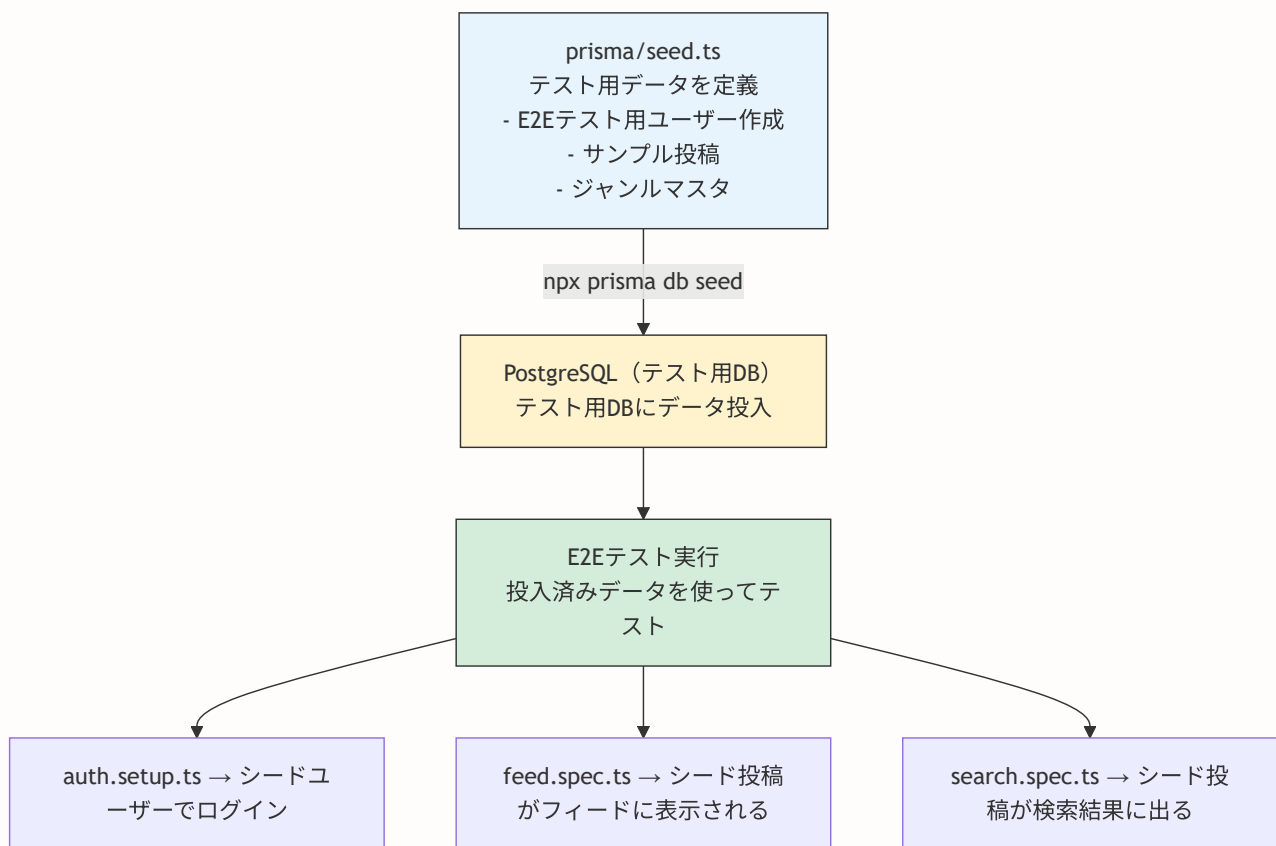


代わりに使うべきメソッド:

メソッド	用途
<code>await expect(locator).toBeVisible()</code>	要素の表示を待つ
<code>await page.waitForLoadState('networkidle')</code>	ネットワークリクエスト完了を待つ
<code>await page.waitForURL(/pattern/)</code>	URL変更を待つ
<code>await page.waitForResponse(url)</code>	特定APIレスポンスを待つ
<code>await page.waitForSelector('.class')</code>	CSSセレクタの要素を待つ

Q: E2Eテストで使うテストデータはどう用意しますか？

A: BON-LOG では、Prisma の **シードデータ** を使ってE2Eテスト用のデータを事前に投入しています。



`e2e/auth.setup.ts` では、シードデータで作成されたテスト用ユーザー `e2e-test@example.com` を使ってログインしています。

Q: Playwrightでスクリーンショットを撮るにはどうしますか？

A: テスト内で明示的にスクリーンショットを撮ることも、失敗時に自動で撮ることもできます。

```
// 明示的にスクリーンショットを撮る
test('フィードページのスクリーンショット', async ({ page }) => {
  await page.goto('/feed')

  // ページ全体のスクリーンショット
  await page.screenshot({ path: 'screenshots/feed-page.png' })

  // 特定の要素だけスクリーンショット
  const postCard = page.locator('[data-testid="post-card"]').first()
  await postCard.screenshot({ path: 'screenshots/post-card.png' })

  // フルページ（スクロールが必要な長いページ）のスクリーンショット
  await page.screenshot({
    path: 'screenshots/feed-full.png',
    fullPage: true,
  })
})
```

失敗時の自動スクリーンショットは `playwright.config.ts` で設定されています:

```
use: {
  screenshot: 'only-on-failure', // テスト失敗時のみ自動撮影
  video: 'retain-on-failure',    // テスト失敗時のみビデオ保存
}
```

B.5 モック・環境に関する質問

Q: `@vitest-environment node` と `@vitest-environment jsdom` の使い分けは？

A: テスト対象がサーバーサイドかクライアントサイドかで使い分けます。

@vitest-environment	用途	利用可能API	テストファイル例
jsdom (デフォルト)	React コンポーネントのテスト	window , document , DOM操作	Button.test.tsx , PostCard.test.tsx , LikeButton.test.tsx
node	Server Actions / API Routes のテスト	Node.js の全API (fetch , crypto 等)	post.test.ts , auth.test.ts , route.test.ts

```
// Server Action のテストでは先頭にこのコメントを書く
/**
 * @vitest-environment node
 */

import { createPost } from '@lib/actions/post'

// コンポーネントテストではデフォルトの jsdom が使われる
// (コメント不要)
import { render, screen } from '@testing-library/react'
import { Button } from '@components/ui/Button'
```

Q: テストで環境変数を使いたい場合はどうしますか？

A: テスト内で `process.env` を直接設定するか、`.env.test` ファイルを使います。

```
describe('環境変数に依存する関数', () => {
  // 元の値を保存
  const originalEnv = process.env

  beforeEach(() => {
    // process.env をクリーンな状態にする
    vi.resetModules()
    process.env = { ...originalEnv }
  })

  afterAll(() => {
    // テスト後に元の環境変数に戻す
    process.env = originalEnv
  })

  it('本番環境ではデバッグログを出力しない', () => {
    process.env.NODE_ENV = 'production'
    // テスト対象の関数を実行 ...
  })

  it('開発環境ではデバッグログを出力する', () => {
    process.env.NODE_ENV = 'development'
    // テスト対象の関数を実行 ...
  })
})
```

注意: Vitest はデフォルトで `process.env.NODE_ENV = 'test'` を設定します。テスト内で `NODE_ENV` を変更する場合は、必ず `afterAll` で元に戻しましょう。

Q: グローバルモックが適用されているか分からないとき、どうデバッグしますか？

A: `console.log` でモック関数かどうかを確認できます。

```
import { prisma } from '@lib/db'
import { auth } from '@lib/auth'

it('デバッグ: モックが適用されているか確認', () => {
  // モック関数には _isMockFunction プロパティがある
  console.log('prisma.user.findUnique はモックか?',
    vi.isMockFunction(prisma.user.findUnique)) // true ならモック

  console.log('auth はモックか?',
    vi.isMockFunction(auth)) // true ならモック

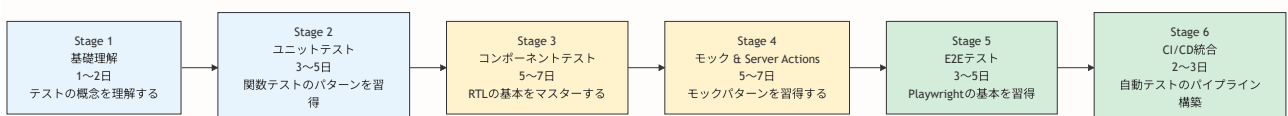
  // モック関数の呼び出し記録を確認
  console.log('auth の呼び出し記録:',
    (auth as ReturnType<typeof vi.fn>).mock.calls)
})
```

▶ 理解度チェック (FAQ) (クリックで回答を確認)

付録C: 学習ロードマップ

この付録の目的: テストの学習は段階的に進めるのが効果的です。ここでは、BON-LOG プロジェクトを題材にした具体的な学習ロードマップを提示します。各ステージに目安期間と達成基準を設けています。

C.1 全体像



C.2 Stage 1: 基礎理解（1～2日）

目標: テストの概念と Vitest の基本構文を理解する。

学習内容

項目	対応セクション	所要時間
テストとは何か	21.1	30分
テストの種類とピラミッド	21.1.3, 21.1.4	30分
Vitest のセットアップ	21.2	1時間
基本構文 (describe, it, expect)	21.3.1	1時間
マッチャーの使い方	21.3.2	30分

達成基準

- ☐ `describe`, `it`, `expect` の3つの関数の役割を説明できる
- ☐ `toBe`, `toEqual`, `toContain` の違いを説明できる
- ☐ `npm test` でテストを実行し、結果を読む
- ☐ AAAパターン (Arrange, Act, Assert) を説明できる

実践課題

```
// 以下の関数に対するテストを書いてみましょう
function add(a: number, b: number): number {
  return a + b
}

function isPositive(n: number): boolean {
  return n > 0
}

function reverseString(str: string): string {
  return str.split('').reverse().join('')
}
```

C.3 Stage 2: ユニットテスト（3～5日）

目標: ユーティリティ関数とバリデーション関数のテストを書けるようになる。

学習内容

項目	対応セクション	所要時間
日付関数のテスト	21.3.3	2時間
バリデーションのテスト	21.3.4	2時間
正常系・異常系・境界値	21.3.4	1時間
演習問題1, 2	21.9	2時間

達成基準

- ☐ 正常系・異常系・境界値の3パターンでテストを書ける
- ☐ zod スキーマの `safeParse` を使ったバリデーションテストが書ける
- ☐ テストケースの説明文を分かりやすく書ける
- ☐ `beforeEach` / `afterEach` の使い方を理解している

実践課題

BON-LOG の以下のファイルに対するテストを書いてみましょう。

テスト対象:

```
└─ lib/utils/date.ts      → formatRelativeTime, isToday
└─ lib/validations/post.ts → createPostSchema
└─ lib/utils/text.ts      → truncateText (演習問題1)
```

C.4 Stage 3: コンポーネントテスト（5～7日）

目標: React Testing Library を使って UI コンポーネントのテストを書けるようになる。

学習内容

項目	対応セクション	所要時間
React Testing Library の基本	21.4.1, 21.4.2	2時間
Buttonコンポーネントのテスト	21.4.3	2時間
PostCardコンポーネントのテスト	21.4.4	2時間
モックの基礎 (vi.fn)	21.4.5	2時間
LikeButtonのテスト (非同期)	21.4.5	3時間
演習問題2, 3	21.9	3時間

達成基準

- ☐ `render`, `screen`, `fireEvent` の3つのAPIを使える
- ☐ `getByText`, `getByRole`, `getByTestId` を使い分けられる
- ☐ `vi.fn()` でモック関数を作成できる
- ☐ `waitFor` で非同期処理の結果を待てる
- ☐ コンポーネントに `props` を渡してテストできる

実践課題

テスト対象:

```
└─ components/ui/Button.tsx      → 表示、クリック、disabled
└─ components/post/PostCard.tsx  → 投稿内容の表示
└─ components/user/UserBadge.tsx → 演習問題2
└─ components/post/LikeButton.tsx → モック使用
```

C.5 Stage 4: モック & Server Actions (5～7日)

目標: Prisma / auth のモック設定と Server Action のテストパターンを習得する。

学習内容

項目	対応セクション	所要時間
Server Action テストの全体像	21.5.1, 21.12	2時間
Prisma モックの設定	21.15.2	2時間
auth モックの設定	21.15.3	2時間
createPost のテスト	21.12.7	3時間
deletePost のテスト	21.5.2	2時間
API ルートのテスト	21.12.3, 21.12.4	3時間
グローバルモックの理解	21.15.1	2時間
React Query のテスト	21.11	3時間

達成基準

- ☐ `vi.mock()` でモジュール全体をモック化できる
- ☐ Prisma のモックで `mockResolvedValue` を使える
- ☐ auth のモックで認証済み/未認証を切り替えられる
- ☐ `@vitest-environment node` の使い方を理解している
- ☐ `expect(fn).not.toHaveBeenCalled()` で副作用の非発生を確認できる
- ☐ React Query のテスト用 QueryClient を作成できる

実践課題

```

テスト対象:
├─ lib/actions/post.ts      → createPost, deletePost
├─ lib/actions/bookmark.ts  → toggleBookmark
├─ lib/actions/like.ts     → toggleLike
├─ app/api/health/route.ts  → GET エンドポイント
└─ app/api/posts/route.ts   → GET, POST エンドポイント
  
```

C.6 Stage 5: E2Eテスト（3～5日）

目標: Playwright を使ってユーザーフローの E2E テストを書けるようになる。

学習内容

項目	対応セクション	所要時間
E2Eテストの概念	21.6.1	30分
Playwright のセットアップ	21.6.2	1時間
playwright.config.ts の設定	21.13.1	2時間
認証セットアップ	21.13.2	2時間
実際のテストファイルの解説	21.13.6	3時間
ページオブジェクトパターン	21.13.3	2時間
デバッグテクニック	21.13.7	1時間

達成基準

- ☐ `npx playwright test` でテストを実行できる
- ☐ `page.goto()`, `page.fill()`, `page.click()` を使える
- ☐ `expect(page).toHaveURL()` でナビゲーションを検証できる
- ☐ 認証状態の保存と再利用ができる
- ☐ `--headed` モードでデバッグできる

実践課題

テスト対象:

```
└─ e2e/auth.spec.ts      → ログイン/登録ページの表示確認
└─ e2e/feed.spec.ts     → フィードページの表示確認
└─ e2e/search.spec.ts   → 検索機能のテスト
└─ 新規: e2e/post-flow.spec.ts → 投稿作成→表示→いいねの一連のフロー
```

C.7 Stage 6: CI/CD統合（2～3日）

目標: GitHub Actions でテストを自動実行するパイプラインを構築できるようになる。

学習内容

項目	対応セクション	所要時間
CIの概念	21.8.1	30分
GitHub Actions ワークフロー	21.8.2	2時間
CI環境でのE2Eテスト	21.13.4	2時間
テスト結果の確認方法	21.8	1時間
カバレッジレポート	21.7	1時間

達成基準

- ☐ GitHub Actions の YAML ファイルを読める
- ☐ PR 作成時にテストが自動実行される仕組みを理解している
- ☐ テスト失敗時のデバッグ手順を知っている
- ☐ カバレッジレポートの見方を理解している

C.8 学習のヒント

#	ヒント	詳細
1	完璧を目指さない	最初は「テストが動く」ことだけを目標にする。リファクタリングは後からで OK
2	既存のテストを読む	BON-LOG の <code>__tests__</code> ディレクトリには380以上のテストファイルがあります。「良いテスト」の実例を読むのが最も効果的な学習
3	テスト駆動開発 (TDD) を試してみる	新機能を追加するときに、先にテストを書いてみる。テストが通るように実装 → リファクタリング
4	小さく始める	最初から Server Actions のモックに挑戦しない。まずは <code>1 + 1 = 2</code> のようなシンプルなテストから
5	エラーメッセージを読む	Vitest のエラーメッセージは非常に親切です。expected / received の差分を注意深く読みましょう

C.9 おすすめリソース

リソース	種類	内容
Vitest 公式ドキュメント	公式	Vitest の設定・API リファレンス
Testing Library 公式	公式	React Testing Library のガイド
Playwright 公式	公式	E2Eテストのチュートリアル
Kent C. Dodds のブログ	ブログ	Testing Library 作者のテスト哲学
BON-LOG <code>__tests__</code> / ディレクトリ	実コード	380以上の実践的テストの実例
BON-LOG <code>vitest.setup.tsx</code>	実コード	グローバルモックのパターン集
BON-LOG <code>e2e</code> / ディレクトリ	実コード	E2Eテストの実例

C.10 達成レベルの自己チェック

各レベルのスキルチェックリストです。自分がどのレベルにいるか確認しましょう。

Level 1: テスト初心者

- ☐ `npm test` コマンドでテストを実行できる
- ☐ テスト結果（成功/失敗）を読んで理解できる
- ☐ `describe`, `it`, `expect` の3つの関数の役割を説明できる
- ☐ 基本的なマッチャー（`toBe`, `toEqual`）を使える

Level 2: ユニットテスト中級

- ☐ 正常系・異常系・境界値の3パターンでテストを書ける
- ☐ `beforeEach`, `afterEach` でテストのセットアップ/クリーンアップができる
- ☐ `vi.fn()` でモック関数を作成し、呼び出しを検証できる
- ☐ zod のバリデーションテストが書ける

Level 3: コンポーネントテスト中級

- ☐ React Testing Library の `render`, `screen`, `fireEvent` を使える
- ☐ `getByText`, `getByRole`, `queryByText` を適切に使い分けられる

- ☐ 非同期コンポーネントのテストで `waitFor` を使える
- ☐ `screen.debug()` でDOMをデバッグできる

Level 4: モック・Server Actions 上級

- ☐ `vi.mock()` でモジュール全体をモック化できる
- ☐ Prisma / auth のモックを設定してServer Actionsをテストできる
- ☐ `@vitest-environment node` の必要性を理解し、適切に設定できる
- ☐ グローバルモックと個別モックの関係を理解している

Level 5: E2E・CI 上級

- ☐ Playwright でログイン → 操作 → 検証のフローを書ける
- ☐ 認証状態の保存と再利用（`storageState`）を設定できる
- ☐ GitHub Actions でテストを自動実行する YAML を書ける
- ☐ テスト失敗時にトレースやスクリーンショットでデバッグできる

Level 6: テストマスター

- ☐ テスト戦略（何をテストし、何をテストしないか）を設計できる
- ☐ テストカバレッジの目標値を設定し、達成方法を計画できる
- ☐ チームメンバーにテストの書き方を教えられる
- ☐ フレーキーテストの原因を特定し、修正できる
- ☐ テスト駆動開発（TDD）で新機能を開発できる

▶ 理解度チェック（学習ロードマップ）（クリックで回答を確認）

付録D: 高度なテストパターン

この付録の目的: 基本的なテストパターンを習得した後に挑戦する、より高度なテクニックを紹介します。BON-LOG プロジェクトの実際のコードから抽出したパターンを、詳細な解説付きで掲載しています。

D.1 動的インポートによるモック制御

Server Actions やAPIルートのテストでは、モック設定**後に**テスト対象モジュールを読み込む必要があります。これを**動的インポートパターン**と呼びます。

```

/**
 * @vitest-environment node
 */

// === モック設定（テスト対象より先に定義） ===
const mockPrisma = createMockPrismaClient()
vi.mock('@lib/db', () => ({ prisma: mockPrisma }))

const mockAuth = vi.fn()
vi.mock('@lib/auth', () => ({ auth: () => mockAuth() }))

vi.mock('next/cache', () => ({
  revalidatePath: vi.fn(),
}))

describe('toggleBookmark', () => {
  beforeEach(() => {
    vi.clearAllMocks()
    // テストごとにモジュールキャッシュをリセット
    // → 各テストで新鮮なモジュールを使う
    vi.resetModules()

    mockAuth.mockResolvedValue({
      user: { id: 'test-user-id' },
    })
  })

  it('ブックマークを追加できる', async () => {
    // モックの戻り値を設定
    mockPrisma.bookmark.findFirst.mockResolvedValue(null)
    mockPrisma.bookmark.create.mockResolvedValue({
      id: 'bookmark-1',
      postId: 'post-1',
      userId: 'test-user-id',
    })

    // === 動的インポート ===
    // vi.mock() が適用された後にモジュールを読み込む
    const { toggleBookmark } = await import('@lib/actions/bookmark')
    const result = await toggleBookmark('post-1')

    expect(result).toEqual({ success: true, bookmarked: true })
    expect(mockPrisma.bookmark.create).toHaveBeenCalled()
  })
})

```

なぜ動的インポートが必要か:

—— 通常のインポート (トップレベル) ——

```
import { toggleBookmark } from '@lib/actions/bookmark'
// ↑ この時点でモジュールが評価される
//   → モジュール内の import { prisma } from '@lib/db' も評価
//   → vi.mock() より先にモジュールがロードされる可能性あり
//   → モックが適用されないかもしれない!
```

—— 動的インポート ——

```
// vi.mock() が確実に適用された後
const { toggleBookmark } = await import('@lib/actions/bookmark')
// ↑ この時点でモジュールが評価される
//   → vi.mock() が既に適用済み
//   → モックが確実に使われる!
```

D.2 createMockPrismaClient パターン

BON-LOG の `test-utils.tsx` で定義されている `createMockPrismaClient()` は、Prisma クライアントの全メソッドをモック化するヘルパー関数です。

```

/**
 * テスト用の Prisma モッククライアントを作成する
 *
 * 実際の Prisma クライアントと同じインターフェースを持つが、
 * 全てのメソッドが vi.fn() に置き換えられている
 */
export function createMockPrismaClient() {
  // 各テーブルの共通メソッドを生成するヘルパー
  const createMockModel = () => ({
    findUnique: vi.fn(),
    findFirst: vi.fn(),
    findMany: vi.fn(),
    create: vi.fn(),
    update: vi.fn(),
    upsert: vi.fn(),
    delete: vi.fn(),
    deleteMany: vi.fn(),
    count: vi.fn(),
    aggregate: vi.fn(),
    groupBy: vi.fn(),
  })

  return {
    user: createMockModel(),
    post: createMockModel(),
    comment: createMockModel(),
    like: createMockModel(),
    bookmark: createMockModel(),
    follow: createMockModel(),
    followRequest: createMockModel(),
    notification: createMockModel(),
    genre: createMockModel(),
    postGenre: createMockModel(),
    postMedia: createMockModel(),
    bonsaiShop: createMockModel(),
    shopReview: createMockModel(),
    event: createMockModel(),
    report: createMockModel(),
    block: createMockModel(),
    mute: createMockModel(),
    hashtag: createMockModel(),
    passwordResetToken: createMockModel(),
    subscription: createMockModel(),
    // Prisma のトランザクションメソッド
    $transaction: vi.fn((callback) => {
      // コールバック形式のトランザクションに対応
      if (typeof callback === 'function') {
        return callback(mockPrisma)
      }
      // 配列形式のトランザクション
      return Promise.all(callback)
    })
  }
}

```



```
    }},  
    $queryRaw: vi.fn(),  
    $executeRaw: vi.fn(),  
  }  
}
```

`createMockPrismaClient()` の構造:

テーブル / メソッド	モック化されるメソッド
user	<code>findUnique</code> , <code>findFirst</code> , <code>findMany</code> , <code>create</code> , <code>update</code> , <code>delete</code> , <code>count</code> (すべて <code>vi.fn()</code>)
post	user と同じ構造
comment, like, bookmark, ...	全テーブル分、同じ構造
\$transaction	<code>vi.fn()</code>
\$queryRaw	<code>vi.fn()</code>

このパターンを使うことで、各テストファイルで `vi.mock('@lib/db')` を毎回書く必要がなくなり、一貫したモック環境を提供できます。

D.3 トランザクションのテスト

Prisma のトランザクション（`$transaction`）をモック化してテストするパターンです。

```

it('フォロー解除でフォロー関係と通知を同時に削除する', async () => {
  // $transaction のモック:
  // コールバック関数を受け取り、その中でモックPrismaを渡す
  mockPrisma.$transaction.mockImplementation(async (callback) => {
    // callback にモックPrismaを渡して実行
    return callback(mockPrisma)
  })

  // トランザクション内で実行されるDB操作のモック
  mockPrisma.follow.delete.mockResolvedValue({
    id: 'follow-1',
    followerId: 'user-1',
    followingId: 'user-2',
  })
  mockPrisma.notification.deleteMany.mockResolvedValue({ count: 1 })

  const { unfollowUser } = await import('@lib/actions/follow')
  const result = await unfollowUser('user-2')

  expect(result.success).toBe(true)
  // トランザクションが呼ばれたことを確認
  expect(mockPrisma.$transaction).toHaveBeenCalled()
  // トランザクション内で両方の操作が行われたことを確認
  expect(mockPrisma.follow.delete).toHaveBeenCalled()
  expect(mockPrisma.notification.deleteMany).toHaveBeenCalled()
})

```

```
テスト対象のコード（実際の実装）  
await  
prisma.$transaction(async tx  
=> {  
  await tx.follow.delete({...})  
  await  
tx.notification.deleteMany({...})  
})
```

モックに到達

```
mockPrisma.$transaction =  
  callback => {  
    // callback に  
    mockPrisma を渡す  
    // → tx.follow.delete は  
    モックが呼ばれる  
    return  
    callback(mockPrisma)  
  }
```

D.4 エラーバウンダリのテスト

Next.js の `error.tsx` コンポーネント（エラーバウンダリ）のテストパターンです。

```

/**
 * error.tsx は 'use client' 必須のコンポーネントで、
 * error オブジェクトと reset 関数を受け取ります。
 */
import { render, screen, fireEvent } from '@testing-library/react'
import ErrorPage from '@app/(main)/feed/error'

describe('FeedErrorPage', () => {
  it('エラーメッセージが表示される', () => {
    const error = new Error('データの取得に失敗しました')
    const reset = vi.fn()

    render(<ErrorPage error={error} reset={reset} />)

    expect(
      screen.getByText(/エラーが発生しました/)
    ).toBeInTheDocument()
  })

  it('再試行ボタンをクリックすると reset が呼ばれる', () => {
    const error = new Error('テストエラー')
    const reset = vi.fn()

    render(<ErrorPage error={error} reset={reset} />)

    // 「再試行」ボタンをクリック
    fireEvent.click(screen.getByRole('button', { name: /再試行/ }))

    // reset 関数が呼ばれたことを確認
    expect(reset).toHaveBeenCalledTimes(1)
  })

  it('エラーの digest が表示される（開発環境）', () => {
    const error = Object.assign(new Error('テスト'), {
      digest: 'abc123',
    })
    const reset = vi.fn()

    render(<ErrorPage error={error} reset={reset} />)

    // digest はエラーの一意識別子（Next.js が付与）
    expect(screen.getByText(/abc123/)).toBeInTheDocument()
  })
})

```

D.5 ミドルウェアのテスト

Next.js の `proxy.ts` のテストパターンです。認証ガードが正しく機能しているかを検証します。

```

/**
 * @vitest-environment node
 */

describe('Middleware', () => {
  beforeEach(() => {
    vi.clearAllMocks()
    vi.resetModules()
  })

  it('認証が必要なパスで未ログイン時にリダイレクトする', async () => {
    // auth が null を返す（未ログイン）ようにモック
    const mockAuth = vi.fn().mockImplementation((handler) => {
      return async (req) => {
        req.auth = null // 未認証状態
        return handler(req)
      }
    })
    vi.mock('@lib/auth', () => ({ auth: mockAuth }))

    const { default: proxy } = await import('@proxy')

    // /feed へのリクエストをシミュレート
    const req = {
      nextUrl: new URL('http://localhost:3000/feed'),
      auth: null,
    }

    const response = proxy(req)

    // ログインページへのリダイレクトを確認
    expect(response).toBeDefined()
    expect(response.headers.get('location')).toContain('/login')
  })

  it('公開パスでは認証チェックをスキップする', async () => {
    const mockAuth = vi.fn().mockImplementation((handler) => {
      return async (req) => {
        req.auth = null
        return handler(req)
      }
    })
    vi.mock('@lib/auth', () => ({ auth: mockAuth }))

    const { default: proxy } = await import('@proxy')

    // /login へのリクエスト（公開パス）
    const req = {
      nextUrl: new URL('http://localhost:3000/login'),
      auth: null,
    }
  })

```

```
const response = proxy(req)

// リダイレクトされないことを確認
expect(response).toBeUndefined()
})
})
```

D.6 レート制限のテスト

スパム対策のレート制限ロジックのテストパターンです。

```

/**
 * @vitest-environment node
 */

describe('Rate Limiter', () => {
  beforeEach(() => {
    vi.clearAllMocks()
    vi.resetModules()
  })

  it('制限内のリクエストは許可される', async () => {
    // Redis モックの設定
    const mockRedis = {
      get: vi.fn().mockResolvedValue(null),
      set: vi.fn().mockResolvedValue('OK'),
      incr: vi.fn().mockResolvedValue(1),
      expire: vi.fn().mockResolvedValue(1),
    }
    vi.mock('@lib/redis', () => ({ redis: mockRedis }))

    const { checkUserRateLimit } = await import('@lib/rate-limit')
    const result = await checkUserRateLimit('user-1', 'post')

    expect(result.success).toBe(true)
  })

  it('制限を超えたリクエストは拒否される', async () => {
    const mockRedis = {
      get: vi.fn().mockResolvedValue('50'), // 既に50回
      incr: vi.fn().mockResolvedValue(51), // 51回目
      expire: vi.fn().mockResolvedValue(1),
    }
    vi.mock('@lib/redis', () => ({ redis: mockRedis }))

    const { checkUserRateLimit } = await import('@lib/rate-limit')
    const result = await checkUserRateLimit('user-1', 'post')

    expect(result.success).toBe(false)
    expect(result.message).toContain('制限')
  })
})

```

D.7 カスタムレンダラーパターン

複数のプロバイダーでラップされたコンポーネントをテストする際の、カスタムレンダラーのパターンです。


```
// __tests__/utils/test-utils.tsx から抜粋

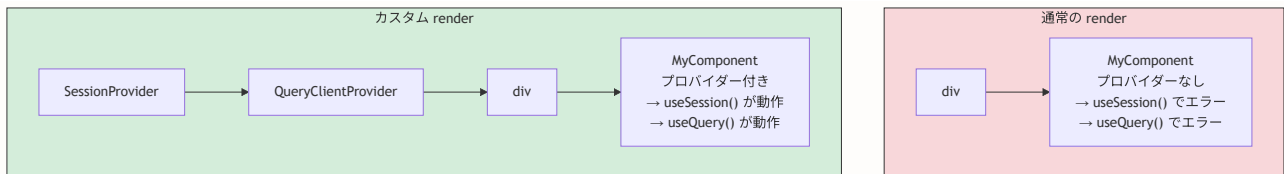
import React, { ReactElement } from 'react'
import { render, RenderOptions } from '@testing-library/react'
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'
import { SessionProvider } from 'next-auth/react'

/**
 * テスト用のカスタムレンダラー
 *
 * 以下のプロバイダーで自動的にラップします:
 * 1. SessionProvider → 認証状態の提供
 * 2. QueryClientProvider → React Query の提供
 */
function AllProviders({ children }: { children: React.ReactNode }) {
  const queryClient = new QueryClient({
    defaultOptions: {
      queries: { retry: false, gcTime: 0 },
      mutations: { retry: false },
    },
  })

  return (
    <SessionProvider session={null}>
      <QueryClientProvider client={queryClient}>
        {children}
      </QueryClientProvider>
    </SessionProvider>
  )
}

/**
 * カスタム render 関数
 *
 * 使い方:
 * import { render } from '../utils/test-utils'
 * render(<MyComponent />)
 * // → SessionProvider + QueryClientProvider でラップされた状態でレンダリング
 */
const customRender = (
  ui: ReactElement,
  options?: Omit<RenderOptions, 'wrapper'>
) => render(ui, { wrapper: AllProviders, ...options })

// Testing Library の render を上書きエクスポート
export { customRender as render }
// その他のユーティリティはそのまま re-export
export * from '@testing-library/react'
```



▶ 理解度チェック（高度なテストパターン）（クリックで回答を確認）

付録E: テストコードレビューチェックリスト

この付録の目的: チームでテストコードをレビューする際に使えるチェックリストです。プルリクエストのレビューで、テストの品質を一定に保つために活用してください。

E.1 基本チェック

- ☐ **テスト対象が明確:** 1つのテストケースで1つの振る舞いだけをテストしている
- ☐ **テスト名が具体的:** 何をテストしているか、テスト名だけで分かる
- ☐ **AAAパターン:** Arrange（準備）、Act（実行）、Assert（検証）が明確に分離されている
- ☐ **テストの独立性:** テスト間に実行順序の依存がない
- ☐ **クリーンアップ:** `beforeEach` でモックがリセットされている

E.2 カバレッジチェック

- ☐ **正常系:** 期待通りの入力で正しく動作するか
- ☐ **異常系:** 不正な入力でエラーが適切に処理されるか
- ☐ **境界値:** ちょうどの値（最大/最小/ゼロ）でテストされているか
- ☐ **エッジケース:** null, undefined, 空配列, 空文字列 など
- ☐ **エラーハンドリング:** DB エラー、ネットワークエラー時の動作

E.3 セキュリティチェック

- ☐ **認証テスト:** 未認証ユーザーがアクセスできないことを確認
- ☐ **認可テスト:** 権限のないユーザーが操作できないことを確認
- ☐ **入力バリデーション:** 不正なデータがDBに保存されないことを確認
- ☐ **副作用の非発生:** 認証/認可失敗時にDB操作が行われないことを確認

E.4 パフォーマンスチェック

- ☐ **テスト実行速度:** 個々のテストが1秒以内に完了する
- ☐ **不要な待機なし:** `waitForTimeout` のような固定待機を使っていない
- ☐ **モックの最小化:** 必要最小限のモックだけ使っている
- ☐ **テストデータの最小化:** テストに必要な最小限のデータだけ用意している

E.5 メンテナンス性チェック

- ☐ **DRY原則:** テスト間で重複するセットアップがヘルパーに抽出されている
- ☐ **脆くないセレクタ:** CSSクラスではなく `data-testid` や `getByRole` を使っている
- ☐ **マジックナンバーなし:** `expect(count).toBe(5)` ではなく定数や変数を使っている
- ☐ **テストヘルパーの活用:** `test-utils.tsx` のモックデータを活用している

付録F: トラブルシューティングガイド

この付録の目的: テストを書いているときに遭遇しやすいエラーや問題と、その解決方法をまとめました。エラーメッセージで検索して、素早く解決できるようになっています。

F.1 Vitest 関連のエラー

エラー: `Cannot find module '@/lib/xxx'`

```
FAIL  __tests__/lib/actions/post.test.ts
  • Test suite failed to run

  Cannot find module '@/lib/db' from 'lib/actions/post.ts'
```

原因: Vitest がパスエイリアス (`@/`) を解決できていません。

解決方法: `vitest.config.ts` で `vite-tsconfig-paths` プラグインが設定されているか確認してください。

```
// vitest.config.ts
import { defineConfig } from 'vitest/config'
import tsconfigPaths from 'vite-tsconfig-paths' // ← これが必要

export default defineConfig({
  plugins: [tsconfigPaths()], // ← tsconfig.json のパスエイリアスを自動解決
  test: {
    // ...
  },
})
```

パスエイリアスの解決フロー:

ソースコード	vite-tsconfig-paths	実際のパス
<code>import '@/lib/db' →</code>	<code>tsconfig.json の paths →</code>	<code>'./lib/db'</code>
	↑	
	<code>"@/*": [".//*"]</code>	
	<code>@ をプロジェクトルートに置換</code>	

ポイント: `vite-tsconfig-paths` プラグインを使っている場合、`tsconfig.json` の `paths` 設定を自動的に読み取ります。`vitest.config.ts` の `plugins` 配列にプラグインが含まれているか確認しましょう。

エラー: `SyntaxError: Cannot use import statement outside a module`

```
SyntaxError: Cannot use import statement outside a module
```

```
> 1 | import { something } from 'some-esm-package'
```

原因: ESM (ECMAScript Modules) 形式のパッケージを Vitest が変換できていません。

解決方法: `vitest.config.ts` の `deps.optimizer.web.include` に該当パッケージを追加します。

```
// vitest.config.ts
export default defineConfig({
  test: {
    deps: {
      optimizer: {
        web: {
          include: [
            'some-esm-package', // ← ESM パッケージを追加
            'another-package',
          ],
        },
      },
    },
  },
})
```

デフォルトの動作:

```
node_modules/
├─ react/           → 事前バンドル済み
├─ some-esm-package/ → バンドルされない ← ここが問題！
└─ next/           → 事前バンドル済み
```

`deps.optimizer.web.include` 修正後:

```
node_modules/
├─ react/           → 事前バンドル済み
├─ some-esm-package/ → 事前バンドルに含める ← 解決！
└─ next/           → 事前バンドル済み
```

よくあるESMパッケージ: `uuid`, `nanoid`, `isomorphic-dompurify`, `jose`, `otplib` など。エラーメッセージにパッケージ名が表示されるので、それを `deps.optimizer.web.include` に追加してください。

エラー: `ReferenceError: document is not defined`

```
ReferenceError: document is not defined
```

```
> 15 | const element = document.getElementById('root')
```

原因: テスト環境が `node` に設定されているが、ブラウザ API (`document`, `window` など) を使用しています。

解決方法: テストファイルの先頭に `@vitest-environment jsdom` を追加します。

```
/**
 * @vitest-environment jsdom
 */

import { render, screen } from '@testing-library/react'
import { MyComponent } from '@components/MyComponent'

test('renders correctly', () => {
  render(<MyComponent />)
  expect(screen.getByText('Hello')).toBeInTheDocument()
})
```

テスト対象	環境	指定方法
コンポーネント	jsdom	デフォルト
カスタムフック	jsdom	デフォルト
Server Actions	node	<code>@vitest-environment node</code>
API Routes	node	<code>@vitest-environment node</code>
ユーティリティ関数	どちらでもOK	デフォルト
Prisma 操作	node	<code>@vitest-environment node</code>

エラー: `TypeError: Cannot read properties of undefined (reading 'fn')`

```
TypeError: Cannot read properties of undefined (reading 'fn')
```

```
> 5 | const mockFunction = vi.fn()
```

原因: テストファイルの拡張子が `.ts` / `.tsx` ではなく `.js` / `.jsx` になっている、または Vitest の設定が正しくない場合に発生します。

解決方法:

1. テストファイルの拡張子を `.ts` / `.tsx` に変更
2. `vitest.config.ts` の `include` パターンを確認

```
// vitest.config.ts
export default defineConfig({
  test: {
    include: [
      '**/__tests__/**/*.test.ts', // ← .ts, .tsx を含む
    ],
  },
})
```

エラー: `Warning: An update to Component inside a test was not wrapped in act(...)`

```
Warning: An update to MyComponent inside a test was not wrapped in act(...)
```

原因: テスト中にコンポーネントの状態更新が発生しましたが、`act()` でラップされていません。

解決方法: 状態更新を引き起こす操作を `act()` でラップするか、`waitFor` を使用します。

```
// ❌ 警告が出るコード
test('updates state', () => {
  render(<Counter />)
  fireEvent.click(screen.getByText('増やす'))
  // 状態更新が反映される前に検証している
  expect(screen.getByText('1')).toBeInTheDocument()
})

// ✅ 修正方法1: waitFor を使用
test('updates state', async () => {
  render(<Counter />)
  fireEvent.click(screen.getByText('増やす'))
  await waitFor(() => {
    expect(screen.getByText('1')).toBeInTheDocument()
  })
})

// ✅ 修正方法2: findByText を使用（内部で waitFor を使用）
test('updates state', async () => {
  render(<Counter />)
  fireEvent.click(screen.getByText('増やす'))
  expect(await screen.findByText('1')).toBeInTheDocument()
})
```

act() の仕組み:

通常のReact:

クリック → 状態更新 → 再レンダリング → 画面反映

↑ ブラウザが自動で処理

テスト環境:

クリック → 状態更新 → ???

↑ act() がないと再レンダリングが完了しない

act() / waitFor あり:

クリック → 状態更新 → 再レンダリング → 画面反映 → 検証

↑ act() が再レンダリング完了を待ってくれる

F.2 Playwright (E2E) 関連のエラー

エラー: `Timeout exceeded while waiting for selector`

```
Error: Timeout 30000ms exceeded.  
  waiting for locator('text=投稿する')
```

原因: 指定した要素がページに見つかりません。考えられる原因は複数あります。

解決方法:

```
// ❌ タイムアウトするコード  
test('投稿する', async ({ page }) => {  
  await page.goto('/feed')  
  await page.getByText('投稿する').click() // ← 要素が見つからない  
})  
  
// ✅ 修正方法1: ページの読み込みを待つ  
test('投稿する', async ({ page }) => {  
  await page.goto('/feed')  
  await page.waitForLoadState('networkidle') // ← ネットワーク待機  
  await page.getByText('投稿する').click()  
})  
  
// ✅ 修正方法2: 要素の可視性を先に確認  
test('投稿する', async ({ page }) => {  
  await page.goto('/feed')  
  const button = page.getByRole('button', { name: '投稿する' })  
  await expect(button).toBeVisible({ timeout: 10000 })  
  await button.click()  
})  
  
// ✅ 修正方法3: セレクタを見直す  
test('投稿する', async ({ page }) => {  
  await page.goto('/feed')  
  // getByText よりも getByRole の方が堅牢  
  await page.getByRole('button', { name: /投稿/i }).click()  
})
```

要素が見つからない主な原因:

1. ページがまだ読み込み中
→ `waitForLoadState('networkidle')` で待機
2. テキストが微妙に異なる
→ 正規表現で部分一致: `/投稿/i`
3. 要素が非表示 (`display: none`)
→ `toBeVisible()` で可視性を確認
4. `iframe`やShadow DOM内にある
→ `frame()` や `locator()` で範囲を指定
5. 認証が必要だがログインしていない
→ `storageState` で認証状態を再利用

エラー: `page.goto: net::ERR_CONNECTION_REFUSED`

Error: page.goto: net::ERR_CONNECTION_REFUSED at http://localhost:3000/

原因: 開発サーバーが起動していません。

解決方法: `playwright.config.ts` の `webServer` 設定を確認します。

```
// playwright.config.ts
export default defineConfig({
  webServer: {
    command: 'npm run dev',           // ← サーバー起動コマンド
    url: 'http://localhost:3000',     // ← サーバーのURL
    reuseExistingServer: !process.env.CI, // ← ローカルでは既存サーバーを再利用
    timeout: 120_000,                // ← 起動待機時間 (2分)
  },
})
```

webServer の動作:

テスト実行時:

1. `reuseExistingServer = true` の場合:
 - `localhost:3000` にアクセスを試みる
 - 応答があれば、既存のサーバーを使う
 - 応答がなければ、`command` を実行してサーバーを起動
2. `reuseExistingServer = false` の場合 (CI) :
 - 常に `command` を実行してサーバーを起動
 - `timeout` 内に応答がなければエラー

ヒント: ローカル開発時は、先に `npm run dev` でサーバーを起動してからテストを実行すると、テストの起動が速くなります。

エラー: `storageState: ENOENT: no such file or directory`

```
Error: storageState: ENOENT: no such file or directory, open 'e2e/.auth/user.json'
```

原因: 認証セットアップ (`auth.setup.ts`) が実行される前にテストが実行されています。

解決方法: `playwright.config.ts` の `projects` 設定で依存関係を確認します。

```
// playwright.config.ts
export default defineConfig({
  projects: [
    // 1. 認証セットアップ (最初に実行)
    { name: 'setup', testMatch: /\.*\setup\.ts/ },

    // 2. テスト (setup 完了後に実行)
    {
      name: 'chromium',
      use: {
        ...devices['Desktop Chrome'],
        storageState: 'e2e/.auth/user.json', // ← setup で保存したファイル
      },
      dependencies: ['setup'], // ← setup に依存
    },
  ],
})
```

認証フローの流れ:

1. setup プロジェクト実行
 - └─ auth.setup.ts
 - └─ /login ページにアクセス
 - └─ メール・パスワード入力
 - └─ ログインボタンクリック
 - └─ /feed にリダイレクト確認
 - └─ storageState を e2e/.auth/user.json に保存
2. chromium プロジェクト実行 (dependencies: ['setup'])
 - └─ 各テストファイル
 - └─ storageState から認証情報を読み込み
 - └─ 認証済み状態でテスト実行

F.3 モック関連のエラー

エラー: `vi.mock()` is not allowed in manual mock

```
vi.mock() is not allowed to be called inside a manual mock factory.
```

原因: `vi.mock()` のファクトリ関数内で別の `vi.mock()` を呼んでいます。

解決方法: `vi.mock()` はトップレベルでのみ呼び出してください。

```
// ✖ ファクトリ内で vi.mock を呼んでいる
vi.mock('@lib/auth', () => {
  vi.mock('@lib/db') // ← これはエラー
  return { auth: vi.fn() }
})

// ✔ それぞれ独立して宣言する
vi.mock('@lib/auth', () => ({
  auth: vi.fn(),
}))
vi.mock('@lib/db')
```

問題: モックが適用されない

```
// テストが失敗する例
vi.mock('@lib/db')
import { prisma } from '@lib/db'
import { createPost } from '@lib/actions/post'

test('投稿を作成する', async () => {
  // prisma.post.create がモックされていない!
  ;(prisma.post.create as ReturnType<typeof vi.fn>).mockResolvedValue({ id: '1' })
  await createPost(formData)
})
```

原因: `vi.mock()` は巻き上げ（ホイスティング）されますが、モック関数の戻り値設定はテスト実行時に行われます。しかし、`createPost` がインポート時に `prisma` を束縛している場合、タイミングの問題が起ることがあります。

解決方法: 動的インポートパターンを使用します。

```
// ✅ 動的インポートで確実にモックを適用
vi.mock('@lib/db')

test('投稿を作成する', async () => {
  const { prisma } = await import('@lib/db')
  const { createPost } = await import('@lib/actions/post')

  ;(prisma.post.create as ReturnType<typeof vi.fn>).mockResolvedValue({ id: '1' })
  const result = await createPost(formData)

  expect(result).toEqual({ success: true, postId: '1' })
})
```

モック適用のタイミング:

通常の import:

1. vi.mock() - モック宣言
(巻き上げ済み)

※ 3 の時点で prisma はモックだが、戻り値未設定

2. import prisma - モジュール読み込み

3. import createPost - モジュール読み込み (prisma を使用)

4. テスト実行 - mockResolvedValue 設定

動的 import:

1. vi.mock() - モック宣言

2. テスト実行

2a. import prisma - ここで
読み込み

2b. 戻り値設定 - すぐに設
定

2c. import action - 設定済
みの prisma を使用

2d. 検証

問題: `vitest.setup.tsx` のグローバルモックを解除したい

状況: `vitest.setup.tsx` で `@/lib/db` がグローバルにモックされているが、特定のテストで実際の実装をテストしたい。

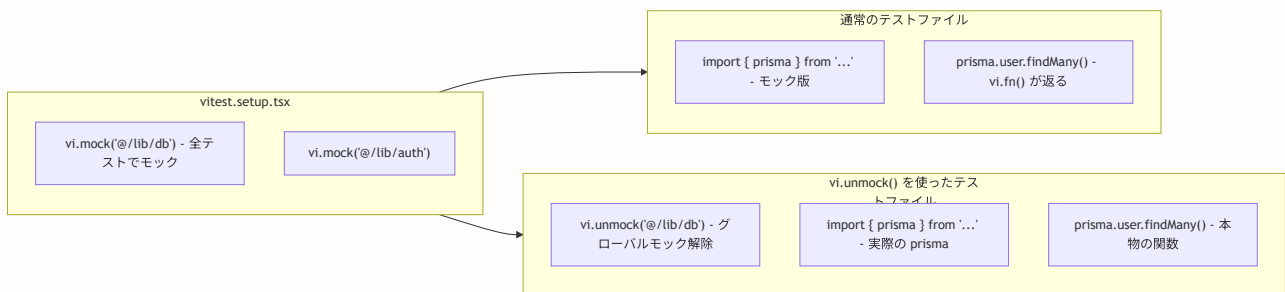
解決方法: テストファイルの先頭で `vi.unmock()` を使用します。

```
// テストファイルの先頭で unmock
vi.unmock('@lib/db')

// これで @lib/db は実際の実装が使われる
import { prisma } from '@lib/db'

test('Prismaクライアントが正しく初期化される', () => {
  expect(prisma).toBeDefined()
  expect(typeof prisma.user.findMany).toBe('function')
})
```

`vi.unmock()` の動作:



F.4 TypeScript 関連のエラー

エラー: `Type 'Mock' is not assignable to type 'xxx'`

```
Type 'Mock<any, any>' is not assignable to type '() => Promise<Session | null>'
```

原因: TypeScript の型チェックでモック関数の型が合いません。

解決方法: 型アサーションを使用します。


```
// ❌ 型エラー
const { auth } = await import('@lib/auth')
auth.mockResolvedValue({ user: { id: '1' } })
// Error: Property 'mockResolvedValue' does not exist

// ✅ 修正方法1: as ReturnType<typeof vi.fn>
;(auth as ReturnType<typeof vi.fn>).mockResolvedValue({
  user: { id: '1', name: 'Test', email: 'test@example.com' },
})

// ✅ 修正方法2: vi.mocked() ヘルパー
const mockedAuth = vi.mocked(auth)
mockedAuth.mockResolvedValue({
  user: { id: '1', name: 'Test', email: 'test@example.com' },
})
```

型アサーションの違い:

as ReturnType<typeof vi.fn>:

- └ メリット: シンプル、どんな関数にも使える
- └ デメリット: 戻り値の型情報が失われる (any)

vi.mocked():

- └ メリット: 元の関数の型情報を保持
 - └ → mockResolvedValue の引数に型チェックが効く
- └ デメリット: なし (Vitest ではネイティブサポート)

F.5 よくある問題パターンと対処法

テストが不安定 (Flaky) になる

テストの不安定さの主な原因:

原因	対処法
タイミング依存	waitFor / findBy を使用
テスト間の状態共有	beforeEach でリセット
外部 API 依存	モックに置き換え
日時依存	vi.useFakeTimers()
ランダムデータ	シード値を固定
環境依存	Docker で統一

具体的な対処例:

```
// ❌ 不安定: setTimeout に依存
test('3秒後にメッセージが消える', async () => {
  render(<Toast message="保存しました" />)
  await new Promise(resolve => setTimeout(resolve, 3000))
  expect(screen.queryByText('保存しました')).not.toBeInTheDocument()
})

// ✅ 安定: Fake Timer を使用
test('3秒後にメッセージが消える', async () => {
  vi.useFakeTimers()
  render(<Toast message="保存しました" />)

  // 時間を進める
  vi.advanceTimersByTime(3000)

  await waitFor(() => {
    expect(screen.queryByText('保存しました')).not.toBeInTheDocument()
  })

  vi.useRealTimers()
})
```

テストの実行が遅い

テスト高速化のチェックリスト:

- ☐ 不要な waitForTimeout を削除したか
- ☐ beforeAll で共通セットアップをまとめたか

- ☐ テストの並列実行を有効にしているか
- ☐ 重いモジュールの import を最小限にしたか
- ☐ カバレッジ収集を必要時のみにしているか

実行速度の目安:

テストの種類	目安時間
ユニットテスト1件	< 100ms
コンポーネント1件	< 500ms
Server Action1件	< 200ms
E2E テスト1件	< 10s
全テスト (7800件)	< 5min

▶ 理解度チェック（トラブルシューティング）（クリックで回答を確認）

付録G: テスト関連コマンドクイックリファレンス

この付録の目的: 日常的によく使うテスト関連のコマンドを一覧にまとめました。手元に置いて、必要なときにすぐ参照できるリファレンスです。

G.1 Vitest コマンド

```
# 基本的なテスト実行
npx vitest run                # 全テストを1回実行
npx vitest                    # ウォッチモードで実行（ファイル変更を監視）
npx vitest run --reporter=verbose # 詳細な出力で実行

# 特定のテストを実行
npx vitest run post           # "post" を含むテストファイルのみ
npx vitest run __tests__/lib/actions/ # 特定ディレクトリのテストのみ
npx vitest run -t "投稿を作成" # テスト名で絞り込み

# カバレッジ
npm run test:coverage         # カバレッジレポート生成
npx vitest run --coverage     # カバレッジ付きで実行

# デバッグ
npx vitest run --reporter=verbose # 詳細な出力
npx vitest --ui                  # ブラウザ UI でテスト管理
npx vitest run --bail 1          # 最初の失敗で停止
```

よく使うコマンドの組み合わせ：

開発中（1つのファイルを編集）：

```
npx vitest post                # "post" 関連のみウォッチモード
```

プルリクエスト前（全体チェック）：

```
npm run test:coverage
```

CI (GitHub Actions)：

```
npx vitest run --coverage
```

デバッグ（特定のテストが失敗する場合）：

```
npx vitest run --reporter=verbose --bail 1
```

G.2 Playwright コマンド

```
# 基本的な E2E テスト実行
npm run test:e2e                # 全E2Eテストを実行
npx playwright test             # Playwright 直接実行
npx playwright test --headed    # ブラウザを表示して実行
npx playwright test --ui        # Playwright UI モードで実行

# 特定のテストを実行
npx playwright test auth.spec.ts # 特定ファイル
npx playwright test --grep "ログイン" # テスト名で絞り込み
npx playwright test --project=chromium # 特定ブラウザのみ

# デバッグ
npx playwright test --debug      # デバッガー付きで実行
npx playwright show-report       # テストレポートを表示
npx playwright codegen localhost:3000 # テストコード自動生成

# セットアップ
npx playwright install           # ブラウザをインストール
npx playwright install chromium # Chromium のみインストール
```

Playwright コマンドの使い分け:

場面	コマンド
初めてのテスト作成	<code>npx playwright codegen</code>
テスト開発中	<code>npx playwright test --ui</code>
失敗原因の調査	<code>npx playwright test --debug</code>
結果の確認	<code>npx playwright show-report</code>
CI での実行	<code>npx playwright test</code>
特定ブラウザだけテスト	<code>--project=chromium</code>

G.3 Docker + テスト

```
# テスト用データベースの起動
docker compose up -d postgres      # PostgreSQL を起動
npx prisma db push                 # スキーマを反映
npx prisma db seed                 # シードデータを投入

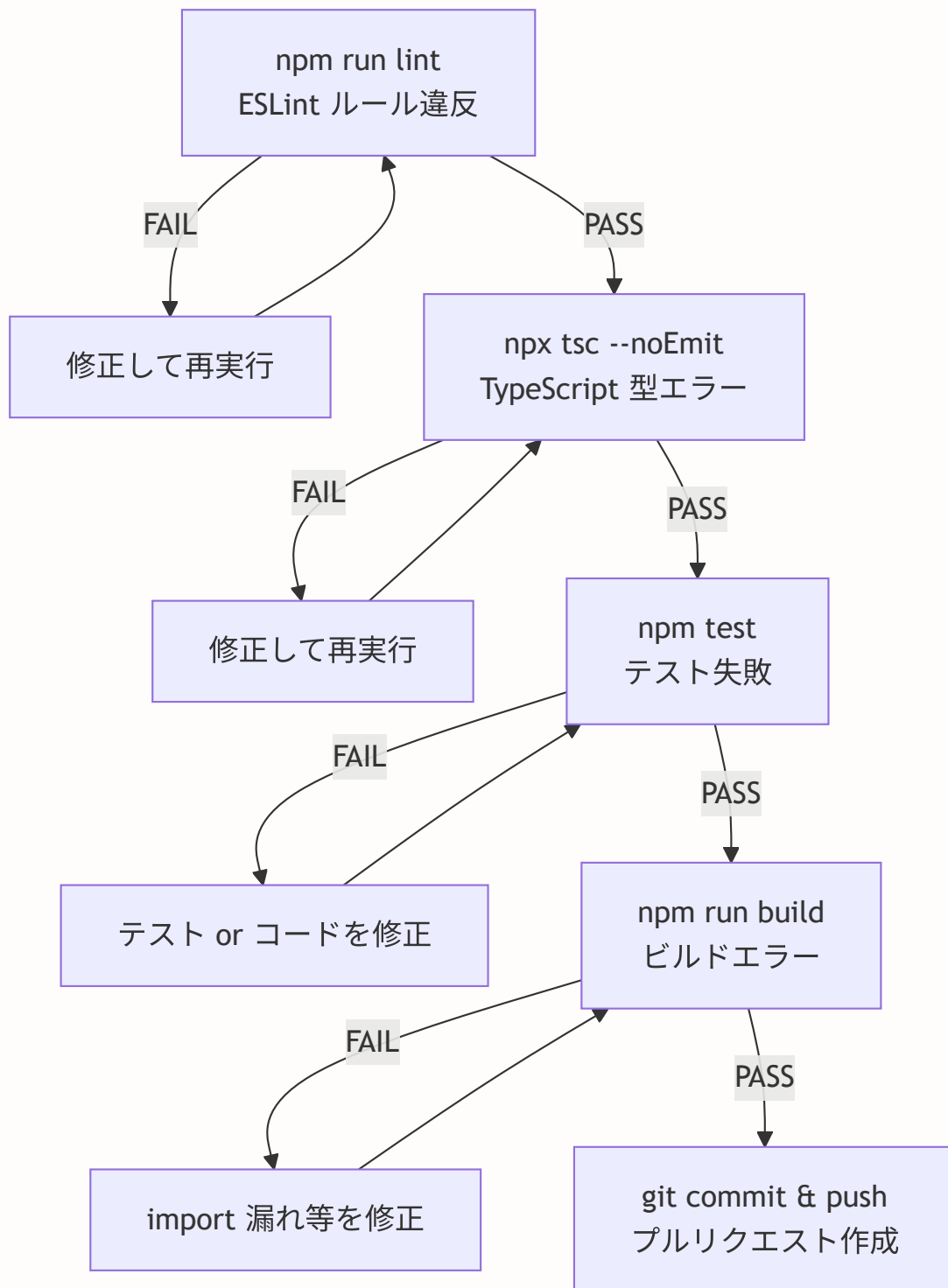
# テスト用データベースのリセット
docker compose down -v             # データ含めて削除
docker compose up -d postgres      # 再起動
npx prisma db push                 # スキーマ再作成
npx prisma db seed                 # シード再投入

# 全テストの一括実行
npm run test:all                   # ユニット + E2E
```

G.4 Git + テスト（プルリクエスト前のチェック）

```
# プルリクエスト前に実行すべきコマンド（順番に）
npm run lint                       # 1. リント
npx tsc --noEmit                   # 2. 型チェック
npm test                           # 3. ユニットテスト
npm run build                      # 4. ビルド確認
npm run test:e2e                   # 5. E2Eテスト（任意）
```

プルリクエスト前のチェックフロー:



► 理解度チェック（コマンドリファレンス）（クリックで回答を確認）

これでテストに関する全ての内容をカバーしました。このチュートリアルの内容を段階的に学習し、実践することで、BON-LOG プロジェクトに対して自信を持ってテストを書けるようになります。テストは「品質の保証」であると同時に「開発者の安心」でもあります。Happy Testing!

[前の章へ: 第20章 セキュリティ対策](#) | [次の章へ: 第22章 デプロイ](#)